

rubrapack 매뉴얼

MSI·MSIX 패키지를 만들고 서명하기 - 그리고 그 형식이 어떻게 짜였는가

판 0.7.0

<https://github.com/rubidus-api/rubrapack>
<https://rubidus-api.github.io/rubrapack/>

차례

제1부 - 따라 하며 배우기

1. 시작하기 전에	2
1.1 설치 파일이 하는 일	2
1.2 무엇을 쓰고 무엇이 만들어지나	2
1.3 rubrapack 받기	3
1.4 터미널을 처음 쓰는 사람을 위해	3
1.5 작업 폴더	3
1.6 이 튜토리얼에서 쓰는 말	4
2. 첫 설치 파일: 프로그램 하나	5
2.1 원본	5
2.2 빌드	6
2.3 설치하고 지우기	6
2.4 안에서 무슨 일이 일어났나	6
2.5 빌드하지 않고 검사하기	7
2.6 편집기 대신 질문으로	7
3. 판과 업그레이드	8
3.1 모든 판에 원본 하나	8
3.2 Windows 는 두 판을 어떻게 다루나	8
3.3 판 번호	9
3.4 두 식별자	9
3.5 사용자보다 먼저 업그레이드를 확인하기	9
3.6 다른 도구로 만든 옛 판	10
4. 더 많은 파일과 폴더	11
4.1 폴더	11
4.2 원본	11
4.3 폴더: [dir.ID]	12
4.4 파일 하나: [file.ID]	13
4.5 여러 파일: [files.ID]	13
4.6 빈 폴더: [folder.ID]	13
4.7 치우기: [remove.ID]	13
4.8 두 번째 사본: [copy.ID]	13
4.9 만날 수 있는 키 두 개 더	13
4.10 안에서 무슨 일이 일어났나	14
5. 바로가기, 아이콘, "설치된 앱" 항목	15
5.1 원본	15
5.2 바로가기: [shortcut.ID]	16
5.3 "설치된 앱" 항목: [arp]	16
5.4 다른 글자로 된 이름	16
5.5 해 보기	16
5.6 안에서 무슨 일이 일어났나	16
6. 대화창, 약관, 그리고 프로그램 시작하기	18
6.1 원본	18
6.2 페이지 세트 고르기: ui	19
6.3 약관: license	19
6.4 설치 폴더	19
6.5 위쪽의 그림: banner	19
6.6 내 말로: [ui-text.ID]	19
6.7 끝에 프로그램 시작하기: launch	19
6.8 해 보기	20

6.9 안에서 무슨 일이 일어났나	20
7. 여러 언어	21
7.1 원본	21
7.2 언어 더하기: <code>languages</code>	22
7.3 언어마다 약관: <code>license-xx</code>	22
7.4 언어마다 글: <code>text-xx</code>	22
7.5 영어와 한국어 말고 다른 언어	22
7.6 번역되지 않는 것	22
7.7 해 보기	22
7.8 안에서 무슨 일이 일어났나	22
8. 선택 구성요소: 기능과 사용자의 선택	24
8.1 기능	24
8.2 원본	24
8.3 기능 선언하기: <code>[feature.ID]</code>	25
8.4 파일을 기능에 넣기: <code>feature</code>	26
8.5 사용자가 보는 것	26
8.6 명령줄에서 고르기	26
8.7 나중에 바꾸기	26
8.8 안에서 무슨 일이 일어났나	27
9. 조건과 검색	28
9.1 속성	28
9.2 원본	28
9.3 내 속성: <code>[property.NAME]</code>	29
9.4 이럴 때만 설치하기: <code>when</code>	30
9.5 설치 거부하기: <code>[require.ID]</code>	30
9.6 먼저 둘러보기: <code>[search.ID]</code>	30
9.7 설치 폴더 기억하기	30
9.8 안에서 무슨 일이 일어났나	30

제2부 - 참조

10. 원본 파일과 <code>rubrapack</code> 명령	33
10.1 시작하기	34
10.2 예	36
10.3 TOML 부분집합	37
10.4 표	37
10.5 명령줄	47
10.6 진단 코드	50

제3부 - 파일 형식

11. 구현하는 사람을 위한 패키지 형식	53
11.1 사실을 읽는 법	53
11.2 표기	54
11.3 흐름을 한 그림으로	54
12. 복합 파일(Compound File Binary, CFB)	55
12.1 섹터	55
12.2 머리(처음 512바이트) <code>[spec]</code>	55
12.3 FAT, DIFAT, 미니 FAT <code>[spec]</code>	56
12.4 디렉터리 <code>[spec]</code>	56
12.5 단순하고 결정적인 작성기	57
12.6 안전하게 읽기	57
13. 복합 파일 속의 MSI 데이터베이스	58

13.1 스트림	58
13.2 문자열 풀: <code>_StringPool</code> 과 <code>_StringData</code>	58
13.3 시스템 표	59
13.4 표 스트림	60
13.5 IDT 보관 파일(<code>MsiDatabaseExport</code> 가 쓰는 것)	60
13.6 안전하게 읽기	60
14. 요약 정보	61
14.1 배치 [spec] [observed]	61
14.2 설치 패키지가 정하는 속성	61
14.3 코드 페이지 함정 - 요약 문자열은 ASCII 로 [observed]	61
15. 설치·복구·업그레이드·제거가 되는 가장 작은 패키지	63
15.1 표와 열	63
15.2 Property	64
15.3 Directory	64
15.4 구성 요소, 기능, 파일	64
15.5 빈 폴더	64
15.6 Media 와 캐비닛	65
15.7 메이저 업그레이드와 다운그레이드 거부	65
15.8 설치한 프로그램으로 등록·해제하기	65
15.9 레지스트리 값	65
15.10 바로가기	66
15.11 파일 지우기와 복제	66
15.12 환경 변수	66
15.13 INI 파일	66
15.14 검색과 설치 조건	67
15.15 서비스, 글꼴, 권한	67
15.16 캐비닛, 관리 이미지, 광고	67
15.17 사용자별 패키지와 겸용 패키지	67
15.18 사용 중인 파일	67
15.19 순서	68
15.20 대화창	68
15.21 이 조리법이 아직 다루지 않는 것	69
16. 프로그램 파일: 머신 형식과 판	70
16.1 머리 찾기 [spec]	70
16.2 판 정보 리소스 [spec]	70
16.3 패키지에 드는 것 [observed]	70
16.4 안전하게 읽기	71
17. 캐비닛, MSZIP, deflate	72
17.1 캐비닛 배치 [spec: MS-CAB]	72
17.2 MSZIP [spec: MS-MCI]	72
17.3 deflate 인코더 간단히 [spec: RFC 1951]	73
18. 결정적인 정체와 재현 가능한 패키지	74
18.1 이름에서 만드는 GUID(UUID 8판) [spec: RFC 9562]	74
18.2 키	74
18.3 재현 가능한 패키지	74
19. Authenticode 서명	75
19.1 PE 파일에서 서명이 사는 곳	75
19.2 PE 다이제스트	75
19.3 CMS SignedData	75
19.4 MSI 패키지	76
19.5 타임스탬프	77

19.6 ECDSA	77
19.7 Windows 가 알려 주는 것	77
20. 출력을 Windows 와 대조하기	78
20.1 데이터베이스가 열리고 풀린다	78
20.2 패키지가 설치된다	78
20.3 쓰기 전에 하는 검사	79
20.4 Microsoft 의 ICE 규칙 [관찰]	79
20.5 멀쩡해 보이지만 아닌 것	79
21. MSIX 패키지	80
21.1 ZIP 압축 파일	80
21.2 블록 맵(AppxBlockMap.xml)	80
21.3 [Content_Types].xml	81
21.4 매니페스트(AppxManifest.xml)	81
21.5 가상 레지스트리(Registry.dat, User.dat)	81
21.6 가상 파일 시스템(VFS\...)	82
21.7 확장	82
21.8 묶음(.msixbundle)	83
21.9 서명(AppxSignature.p7x)	83
21.10 서명 없는 패키지 설치하기	84
22. 레지스트리 하이브 파일(REGF)	85
22.1 배치	85
22.2 기본 블록	85
22.3 키(nk)	86
22.4 하위 키 목록(lh, ri)	86
22.5 값(vk)과 값 목록	86
22.6 보안 칸(sk)	86
22.7 재현 가능한 하이브	86

제1부

따라 하며 배우기

1 시작하기 전에

이 장의 차례

1.1	설치 파일이 하는 일	2
1.2	무엇을 쓰고 무엇이 만들어지나	2
1.3	rubrapack 받기	3
1.4	터미널을 처음 쓰는 사람을 위해	3
1.5	작업 폴더	3
1.6	이 튜토리얼에서 쓰는 말	4

매뉴얼의 이 부분은 따라 하며 배우는 튜토리얼이다. 가장 작은 설치 파일 - 프로그램 하나를 Program Files 에 복사하는 것 - 에서 시작해, 장마다 하나씩 더해 가며 끝에는 rubrapack 의 모든 기능을 한 번씩 써 본다. 장마다 앞 장에 이어서 쌓아 가므로 차례대로 읽는 것이 좋다. 컴퓨터를 쓸 줄 안다는 것 말고는 아무것도 가정하지 않는다. 바깥의 개념(GUID, 해시, 레지스트리 키 같은 것)이 필요하면 그 자리에서 설명하거나 "기초 지식" 부를 가리킨다.

1.1 설치 파일이 하는 일

내 컴퓨터에 있는 프로그램은 그저 폴더 안의 파일들이다. 이것을 다른 사람에게 주려면 폴더를 그대로 보내도 된다 - 하지만 그러면 받는 사람이 둘 곳을 정하고, 시작 메뉴 항목을 직접 만들고, 나중에는 모든 것을 손으로 찾아 지워야 한다. *설치 파일*이 그 일을 대신한다:

- 파일을 알맞은 자리(보통 C:\Program Files\<프로그램>)에 복사한다.
- 바로가기, 레지스트리 값, 파일 형식 연결 등 프로그램이 필요로 하는 것을 만든다.
- 설정 > 앱 > 설치된 앱에 프로그램을 등록해, 거기서 지울 수 있게 한다.
- 2판을 내면 1판을 깨끗하게 바꿔 넣는다.
- 도중에 무언가 잘못되면 모든 것을 원래대로 되돌린다.

Windows 에는 이 일을 하는 엔진이 들어 있다: **Windows Installer**. 설치하는 프로그램을 짜는 것이 아니라, 무엇을 설치할지 기술한 파일인 *패키지*를 만들면 Windows Installer 가 그대로 실행한다. 패키지 형식은 둘이다:

형식	파일	설치하는 것	주로 쓰는 곳
Windows Installer	.msi	Windows Installer(msiexec), Windows 2000 부터	모든 종류의 데스크톱 프로그램; 회사는 관리 도구로 MSI 를 배포한다
MSIX	.msix, .msixbundle	Windows 10·11 의 앱 설치 관리자	새 방식의 패키지 앱, Microsoft Store

rubrapack 은 같은 기술에서 둘 다 만든다. 튜토리얼은 대부분 MSI 를 다루고, 17장에서 같은 예제를 MSIX 로 바꾼다.

1.2 무엇을 쓰고 무엇이 만들어지나

짧은 글 파일 하나, *원본*을 TOML 로 쓴다 - 키 = 값 꼴의 간단한 형식이다. rubrapack 이 그것을 읽어 패키지를 쓴다:

```
hello.toml (내가 쓴다)
|
|   rubrapack build hello.toml -o hello.msi
v
```

```

hello.msi  (사용자에게 준다)
|
|  두 번 누르기, 또는: msexec /i hello.msi
v
C:\Program Files\Hello\hello.exe  (Windows Installer 가 거기 둔다)

```

MSI 파일은 그 자체로 표 수집 개와 식별자와 압축 보관 파일이 든 작은 데이터베이스다 - 이 책의 제4부가 그것을 바이트 단위로 보여 준다. 그 표를 직접 쓸 일은 없다: rubrapack 이 원본에서 알아서 만든다.

1.3 rubrapack 받기

rubrapack 은 따로 설치할 것 없는 프로그램 파일 하나다. 릴리스 페이지 <https://github.com/rubidus-api/rubrapack/releases> 에서 받는다:

- Windows: rubrapack-**<판>**-windows-x64.exe. 이름을 rubrapack.exe 로 바꾼다.
- Linux: rubrapack-**<판>**-linux-x86_64. 이름을 rubrapack 으로 바꾸고 실행할 수 있게 한다: `chmod +x rubrapack`.

PATH 에 들어 있는 폴더에 두면(어디서나 명령이 된다) 편하고, 작업 폴더에 그냥 두었다면 rubrapack 대신 .\rubrapack.exe(Windows)나 ./rubrapack(Linux)으로 친다. 이 튜토리얼은 그냥 rubrapack 으로 쓴다.

패키지는 Linux 에서도 만들 수 있지만, 설치해 보려면 Windows 가 있어야 한다. Windows 가상 머신이면 충분하다.

1.4 터미널을 처음 쓰는 사람을 위해

rubrapack 에는 창이 없다: 명령을 쳐서 쓴다. Windows 에서 Windows 키를 누르고 terminal(또는 cmd)을 쳐서 Enter 를 누르면 프롬프트가 있는 창이 열린다. 예를 들어:

```
C:\Users\you>
```

> 앞의 부분이 *지금 폴더*다. 경로를 주지 않으면 명령은 그 폴더의 파일을 대상으로 한다. 필요한 명령은 셋이다:

명령	하는 일
<code>cd C:\work\hello</code>	C:\work\hello 폴더로 들어간다
<code>dir (Linux: ls)</code>	지금 폴더의 파일을 보인다
<code>mkdir dist</code>	여기에 dist 폴더를 만든다

해 보자: rubrapack version 은 판을 찍고, rubrapack help 는 명령 목록을 보인다.

```

C:\work\hello> rubrapack version
rubrapack 0.7.0 (proven_c_lib-v0.1.1)

```

1.5 작업 폴더

모든 장이 같은 배치를 쓴다. 프로젝트용 폴더를 만들고, 그 안에 배포할 파일만 담은 dist("distribution", 배포) 폴더를 둔다:

```

C:\work\hello\
  dist\
    hello.exe      프로그램 - 자기 프로그램 아무것이나 된다
    hello.toml     원본(다음 장에서 쓴다)

```

마땅한 프로그램이 없으면 아무 .exe 로 배워도 된다. 설치 파일은 그것을 실행하지 않는다.

1.6 이 튜토리얼에서 쓰는 말

말	뜻
패키지	사용자에게 주는 .msi(또는 .msix) 파일
원본	내가 쓰는 .toml 파일; rubrapack 이 여기서 패키지를 만든다
설치 폴더	사용자 컴퓨터에서 파일이 들어갈 곳, 예: C:\Program Files\Hello
제품	Windows 가 보는 내 프로그램: 이름, 판, 제조사
업그레이드 코드	제품의 모든 판을 하나로 묶는 고정된 식별자(2장)
기능(feature)	사용자가 설치할지 말지 고를 수 있는 제품의 한 부분(8장)
컴포넌트 (component)	Windows Installer 의 설치 단위; rubrapack 이 알아서 만든다

2 첫 설치 파일: 프로그램 하나

이 장의 차례

2.1	원본	5
2.2	빌드	6
2.3	설치하고 지우기	6
2.4	안에서 무슨 일이 일어났나	6
2.5	빌드하지 않고 검사하기	7
2.6	편집기 대신 질문으로	7

목표: hello.exe 를 C:\Program Files\Hello 에 복사하고, "설치된 앱"에 나타나고, 깨끗하게 지워지는 설치 파일.

2.1 원본

C:\work\hello 폴더(앞 장 참고)에 아무 편집기로 - 메모장이면 된다 - hello.toml 이라는 글 파일을 만들고 이렇게 쓴다:

```
# tutorial 02: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "1.0.0"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"
```

한 줄씩 보면:

- # 으로 시작하는 줄은 주석이다. rubrapack 은 무시한다.
- [package] 는 *표*를 시작한다: 그 뒤의 줄은 다음 [...] 까지 이 표에 속한다. package 표는 제품이 무엇인지 말한다.
 - ▶ name 은 사용자가 보는 이름이다: "설치된 앱"에서, 설치 창의 제목에서.
 - ▶ manufacturer 는 나나 회사다. "설치된 앱"이 이름 아래에 보인다.
 - ▶ version 은 수 셋(또는 넷)이다: 주판, 부판, 빌드. 3장이 판 바꾸기를 다룬다.
 - ▶ arch 는 프로그램이 만들어진 프로세서다: x64(요즘 거의 모든 PC), x86(32비트 프로그램), arm64. rubrapack 은 hello.exe 가 정말 x64 프로그램인지 확인한다.
 - ▶ upgrade-code 는 GUID - 16진수 32자리를 중괄호로 쓴 128비트 수 - 로, 내 제품에만 있는 것이다. rubrapack guid 로 자기 것을 만들어(아래) 여기에 한 번 쓰고 **절대 바꾸지 않는다**. 이후 모든 판이 같은 코드를 지녀야 한다. 그러지 않으면 Windows 가 다른 제품으로 보고 옛것 옆에 따로 설치한다.
- [dir.INSTALLDIR] 는 ID 가 INSTALLDIR 인 폴더를 선언한다. ID 는 다른 표에서 그것을 가리키려고 내가 정하는 이름이다(영문자, 숫자, _). path 는 폴더가 어디인지 말한다: ProgramFiles/Hello 는 영어판이든 한국어판이든 Windows 에서 C:\Program Files\Hello 다 - 알려진 폴더 ProgramFiles 는 사용자 컴퓨터에서 정해진다.

- `[file.Hello]` 는 파일 하나를 ID Hello 로 설치한다. `dir` 은 들어갈 폴더를 ID 로 가리키고, `source` 는 지금 파일이 있는 곳으로, `hello.toml` 기준의 상대 경로를 / 로 쓴다.

자기 업그레이드 코드를 받는다 - 칠 때마다 새로 나온다:

```
C:\work\hello> rubrapack guid
{F1B1ED21-3061-4A65-8D77-FBB5ACCB1642}
```

예제의 코드 자리에 붙여 넣는다. (예제의 코드를 그대로 쓰면 Windows 가 내 제품과 이 책을 읽은 다른 사람의 제품을 "같은 제품"으로 본다.)

2.2 빌드

```
C:\work\hello> rubrapack build hello.toml -o hello.msi
```

잘되면 아무것도 찍지 않고, 원본 옆에 `hello.msi`(약 28 KB)가 생긴다. 무언가 틀리면 `rubrapack` 은 어디가 왜 틀렸는지 말하고 아무것도 쓰지 않는다. 예를 들어 `source` 에 오타가 있으면:

```
hello.toml:14:1: error[RP1507]: source file 'dist/hello.ex' not found
```

수는 줄과 열이고, 대괄호 안의 코드(RP1507: RP15xx 는 "설치할 파일")는 참조 부의 진단 코드가 설명한다. 그 줄을 고치고 다시 빌드한다.

2.3 설치하고 지우기

`hello.msi` 를 Windows 컴퓨터에 복사해 두 번 누른다. 모든 사용자의 것인 Program Files 에 설치하므로 Windows 가 관리자 허락을 묻는다(사용자 계정 컨트롤 창). 그다음 진행 막대가 있는 작은 창이 잠깐 나타난다 - 이 패키지에는 아직 자기 대화창이 없다(6장에서 더한다). 끝나면:

- `C:\Program Files\Hello\hello.exe` 가 있다.
- 설정 > 앱 > 설치된 앱에 **Hello**, 판 1.0.0, Example Software 가 보인다.

"설치된 앱"에서(... 메뉴, 제거) 지우거나 터미널에서 지운다:

```
msiexec /x hello.msi
```

파일과 폴더가 다시 사라진다. 같은 패키지를 창 없이 설치할 수도 있는데, 회사가 소프트웨어를 배포하는 방식이다(/qn 은 "조용히, 사용자 화면 없이"라는 뜻; 터미널을 관리자로 연다):

```
msiexec /i hello.msi /qn
msiexec /x hello.msi /qn
```

Windows 에서 무언가 잘못되면 기록이 이유를 알려 준다: `msiexec /i hello.msi /l*v install.log` 는 Windows Installer 가 한 일을 모두 `install.log` 에 쓴다.

2.4 안에서 무슨 일이 일어났나

`rubrapack inspect` 는 설치하지 않고 패키지 안을 보여 준다. `--files` 는 파일과 그 자리를 보인다:

```
C:\work\hello> rubrapack inspect hello.msi --files
[ProgramFiles64Folder]\Hello\hello.exe 17920 Hello C_185f8db32271fe25f561 1.2.3.4 1033
```

ProgramFiles64Folder 는 64비트 Program Files 의 Windows Installer 이름이고, 17920 은 바이트 단위 크기다. 1.2.3.4 와 1033(미국 영어)은 `hello.exe` 안에 적힌 판과 언어다(Windows 는 파일을 바꿔 넣을 때 이것을 비교한다). MSI 는 데이터베이스이고 `inspect` 는 어떤 표든 찍을 수 있다. File 표에는 파일 하나에 행 하나가 있다:

```
C:\work\hello> rubrapack inspect hello.msi File
File Component_ FileName FileSize Version Language Attributes Sequence
s72 s72 l255 i4 S72 S20 I2 i4
File File
Hello C_185fdb32271fe25f561 hello.exe 17920 1.2.3.4 1033 512 1
```

첫 줄은 열 이름, 둘째 줄은 열의 형식(s72 는 72자까지의 문자열, i4 는 4바이트 정수, 대문자는 비어 있어도 되는 열), 셋째 줄은 표 이름과 키 열이다. Hello 는 내 파일 ID, C_185f... 는 rubrapack 이 이 파일을 위해 만든 컴포넌트, 512 는 "필수"(이 파일을 쓰지 못하면 설치가 실패한다)라는 뜻이다. 제품의 정체는 Property 표에 있다:

```
C:\work\hello> rubrapack inspect hello.msi Property
...
ProductCode {98BFA9A2-2741-81F9-A1CA-2C6626AE1304}
ProductName Hello
ProductVersion 1.0.0
UpgradeCode {3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}
...
```

ProductCode 는 *이 판*을 가리키는 또 하나의 GUID 다. rubrapack 이 업그레이드 코드, 아키텍처, 판에서 끌어내므로 판이 바뀌면 저절로 바뀐다. 직접 쓰지 않는다. 이 표들이 파일에 어떻게 저장되는지는 제4부가 설명한다.

2.5 빌드하지 않고 검사하기

rubrapack lint hello.toml 은 빌드가 하는 검사를 모두 하되 아무것도 쓰지 않는다 - 고친 뒤 확인하기 좋다. rubrapack lint hello.msi 는 다 만든 패키지를 검사하며, 다른 도구로 만든 것도 된다:

```
C:\work\hello> rubrapack lint hello.msi
hello.msi: 0 errors, 0 warnings
```

2.6 편집기 대신 질문으로

rubrapack new hello.toml 은 이름, 판, 파일이 든 폴더 등을 물은 뒤 위와 같은 원본을 쓴다(업그레이드 코드도 만든다). 19장이 이 명령과 짝인 rubrapack edit 를 설명한다. 여기서처럼 원본을 한 번 손으로 써 보는 것이 각 줄이 하는 일을 익히는 가장 좋은 방법이다.

3 판과 업그레이드

이 장의 차례

3.1	모든 판에 원본 하나	8
3.2	Windows 는 두 판을 어떻게 다루나	8
3.3	판 번호	9
3.4	두 식별자	9
3.5	사용자보다 먼저 업그레이드를 확인하기	9
3.6	다른 도구로 만든 옛 판	10

목표: Hello 1.1.0 을 내서, 설치하면 1.0.0 을 바꿔 넣고, 그 뒤에 1.0.0 을 다시 설치하려 하면 거부되게 한다.

3.1 모든 판에 원본 하나

릴리스 전마다 version 을 손으로 고쳐도 되지만 잊기 쉽다. 대신 판을 기본값이 있는 *변수*로 두고, 빌드할 때 진짜 번호를 준다:

```
# tutorial 03: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
downgrade-message = "더 새 Hello 가 이미 설치되어 있습니다. 이 판을 원하면 먼저 그것을 지우십시오."

[define]
VERSION = "1.0.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"
```

- [define] 에는 변수를 둔다. 문자열 안 어디서든 \${VERSION} 은 변수 값으로 바뀐다. 여기서 기본값은 1.0.0 이다.
- 명령줄의 -D 이름=값 은 그 빌드 한 번에서 변수를 덮어쓴다:

```
C:\work\hello> rubrapack build hello.toml -o hello-1.0.0.msi
C:\work\hello> rubrapack build hello.toml -o hello-1.1.0.msi -D VERSION=1.1.0
```

- downgrade-message 는 사용자가 더 새 판 위에 옛(또는 같은) 판을 설치하려 할 때 보는 안내문이다. 없으면 rubrapack 은 "The same or a newer version of [ProductName] is already installed." 를 쓴다 ([ProductName] 은 이름으로 바뀐다).

3.2 Windows 는 두 판을 어떻게 다루나

hello-1.0.0.msi 를 설치하고 이어서 hello-1.1.0.msi 를 설치한다. Windows Installer 는 새 패키지가 같은 업그레이드 코드에 더 높은 판임을 보고, 1.0.0 을 지운 뒤 1.1.0 을 설치한다 - "설치된 앱"에는 항목 하나가 남

고 이제 1.1.0 을 보인다. 이것을 *메이저 업그레이드*라고 하며, rubrapack 이 만드는 업그레이드는 이것뿐이다. 새 판은 늘 완전한 패키지이고, 새 판을 설치하는 데 옛 판이 필요하지 않다.

이제 hello-1.0.0.msi 를 다시 설치해 본다: Windows 가 다운그레이드 안내문을 보이고 아무것도 바꾸지 않는다. hello-1.1.0.msi 를 한 번 더 실행하는 것도 오류가 아니다. 설치된 판과 제품 코드가 같으므로(아래 참고) Windows 는 제품이 이미 설치되어 있다고 알아보고 두 번 설치하지 않는다. 설치된 판을 고치려면 - 지워진 파일을 되살리는 등 - msixexec /fa hello-1.1.0.msi 를 쓴다.

판을 올리지 않고 바뀐 파일로 다시 빌드한 패키지는 설치된 것과 제품 코드는 같지만 같은 패키지가 아니어서, Windows 가 자체 안내문으로 거부한다: "Another version of this product is already installed"(오류 1638). 그러니 파일이 바뀌면 판을 올린다 - 설치된 것을 바꿔 넣는 것은 더 높은 판뿐이다.

rubrapack 이 이를 어떻게 꾸미는지는 Upgrade 표에 보인다:

```
C:\work\hello> rubrapack inspect hello-1.1.0.msi Upgrade
UpgradeCode VersionMin VersionMax Language Attributes Remove ActionProperty
s38 S20 S20 S255 i4 S255 s72
Upgrade UpgradeCode VersionMin VersionMax Language Attributes
{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D} 1.1.0 1 RP_OLDER_FOUND
{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D} 1.1.0 258 RP_NEWER_FOUND
```

첫 행은 1.1.0 보다 낮은 설치된 판을 찾고(지우려고), 둘째 행은 1.1.0 이상을 찾아 다운그레이드 안내문과 함께 설치를 멈춘다. 속성 수는 비트 플래그다 - 258 은 256 + 2, "최솟값 포함"에 "찾기만 하고 지우지 않음"을 더한 것이다. 비트 플래그는 제3부, 이 표는 제4부가 설명한다.

3.3 판 번호

- 수 셋이나 넷: 주판.부판.빌드 또는 주판.부판.빌드.리비전. Windows Installer 는 앞의 셋에 255.255.65535 까지 허용한다.
- **Windows 는 앞의 셋만 비교한다.** 1.0.0.1 과 1.0.0.2 는 같은 판이다: 둘째는 "같은 것"으로 거부된다. 릴리스마다 셋째 수(또는 그 위의 수)를 올린다.
- 틀린 판은 빌드를 멈춘다:

```
hello.toml:5:1: error[RP1308]: version '1.0' must be a.b.c or a.b.c.d with a, b <= 255 and c, d <= 65535
```

3.4 두 식별자

	업그레이드 코드	제품 코드
가리키는 것	모든 판에 걸친 제품	판 하나
직접 쓰나	upgrade-code 에 한 번; 절대 바꾸지 않는다	아니다: rubrapack 이 끌어낸다
바뀌나	바뀌지 않는다	판(앞의 수 셋)이 바뀔 때마다

```
1.0.0: ProductCode {98BFA9A2-2741-81F9-A1CA-2C6626AE1304}
1.1.0: ProductCode {9CC17977-2023-8B8F-8BB0-2FA82C5B248F}
```

끌어낸 제품 코드는 같은 판을 빌드할 때마다, 어느 컴퓨터에서나 같다. [package] 의 product-code 는 이것을 손으로 정한다 - 앞선 판을 다른 도구가 자기 제품 코드로 만든 제품을 이어 갈 때만 필요하다.

3.5 사용자보다 먼저 업그레이드를 확인하기

어떤 실수는 빌드를 멈추지 않지만 사용자 컴퓨터에서 업그레이드를 망가뜨린다: 바뀐 업그레이드 코드, 높은 판, Windows 가 따라갈 수 없게 자리를 옮긴 파일. 새 패키지를 그 앞의 것과 대조한다:

```
C:\work\hello> rubrapack lint hello-1.1.0.msi --previous hello-1.0.0.msi
hello-1.1.0.msi: 0 errors, 0 warnings (against hello-1.0.0.msi)
```

실수가 어떻게 보이는지 보려고 같은 패키지를 자기 자신과 대조하면:

```
C:\work\hello> rubrapack lint hello-1.0.0.msi --previous hello-1.0.0.msi
hello-1.0.0.msi: error[RP2302]: ProductVersion 1.0.0 is not higher than the previous 1.0.0 in its first three
fields: Windows Installer does not upgrade
hello-1.0.0.msi: warning[RP2303]: the ProductCode is the previous version's: that is a minor upgrade
(REINSTALLMODE), not the major upgrade rubrapack builds
hello-1.0.0.msi: 1 error, 1 warning (against hello-1.0.0.msi)
```

낸 .msi 는 모두(예: releases 폴더에) 보관해, 다음 판을 그것과 대조할 수 있게 한다.

3.6 다른 도구로 만든 옛 판

내 제품이 전에 다른 도구의 패키지로 설치되었더라도, 업그레이드 코드가 같으면 rubrapack 의 업그레이드가 그것을 바꿔 넣는다. 그것이 잘 안 될 때 도움이 되는 키가 둘 있다:

- `refuse-upgrade-below = "0.60.0"`: 이 판보다 낮은 판은 업그레이드하지 않고 거부하며, 사용자에게 먼저 지우는 방법을 알려 준다 - 업그레이드 도중에 지우면 망가지는 것으로 알려진 옛 패키지를 위한 것이다.
- `refuse-upgrade-message = "..."`: 그 안내문의 앞부분(지우는 명령은 뒤에 붙는다).

4 더 많은 파일과 폴더

이 장의 차례

4.1	폴더	11
4.2	원본	11
4.3	폴더: [dir.ID]	12
4.4	파일 하나: [file.ID]	13
4.5	여러 파일: [files.ID]	13
4.6	빈 폴더: [folder.ID]	13
4.7	치우기: [remove.ID]	13
4.8	두 번째 사본: [copy.ID]	13
4.9	만날 수 있는 키 두 개 더	13
4.10	안에서 무슨 일이 일어났나	14

목표: 프로그램 폴더 전체를 설치한다 - 프로그램, 다른 이름으로 둘 읽어보기, 사용자가 고칠 수 있는 설정 파일, 문서 폴더, 하위 폴더에 든 예제, 데이터를 위한 빈 폴더 - 그리고 프로그램이 쓰는 기록 파일을 치운다.

4.1 폴더

```
C:\work\hello\
  dist\
    hello.exe
    readme.txt
    settings.ini
    docs\
      guide.txt
    samples\
      sample1.txt
      sub\
        sample2.txt
  hello.toml
```

4.2 원본

```
# tutorial 04: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"

[define]
VERSION = "1.2.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[dir.DocsDir]
path = "INSTALLDIR/docs"

[dir.SamplesDir]
```

```

path = "INSTALLDIR/samples"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[file.Readme]
dir = "INSTALLDIR"
source = "dist/readme.txt"
name = "Read me.txt"
vital = false

[file.Settings]
dir = "INSTALLDIR"
source = "dist/settings.ini"
keep = true

[files.DocFiles]
dir = "DocsDir"
glob = "dist/docs/**"

[files.SampleFiles]
dir = "SamplesDir"
glob = "dist/samples/**/*.txt"

[folder.Data]
dir = "INSTALLDIR"
name = "data"
keep = true

[remove.Logs]
dir = "INSTALLDIR"
name = "*.log"
on = "uninstall"

[copy.ReadmeInDocs]
source = "file:Readme"
dir = "DocsDir"
name = "readme.txt"

```

빌드하고 파일을 본다:

```

C:\work\hello> rubrapack build hello.toml -o hello.msi
C:\work\hello> rubrapack inspect hello.msi --files
[ProgramFiles64Folder]\Hello\Read me.txt 8 Readme
[ProgramFiles64Folder]\Hello\docs\guide.txt 6 F_6d1508f52f6615459567
[ProgramFiles64Folder]\Hello\hello.exe 17920 Hello
[ProgramFiles64Folder]\Hello\samples\sample1.txt 7 F_1dc2b5e85973ee91d955
[ProgramFiles64Folder]\Hello\samples\sub\sample2.txt 7 F_c9c5c4da73214a6abf37
[ProgramFiles64Folder]\Hello\settings.ini 23 Settings

```

(실제 출력에는 열이 더 있다. 여기서는 경로, 크기, ID 만 보였다.)

4.3 폴더: [dir.ID]

dir 의 path 는 *알려진 폴더*나 다른 dir 에서 시작해 그 아래 폴더 이름을 잇는다:

path	사용자 컴퓨터에서(흔한 경우)
ProgramFiles/Hello	C:\Program Files\Hello (x86 패키지는 C:\Program Files (x86)\Hello)
INSTALLDIR/docs	C:\Program Files\Hello\docs - 사용자가 어디에 두든 dir INSTALLDIR 아래

path	사용자 컴퓨터에서(흔한 경우)
CommonAppData/Hello	C:\ProgramData\Hello - 모든 사용자가 함께 쓰는 데이터
LocalAppData/Hello	C:\Users\<이름>\AppData\Local\Hello

ProgramFiles/Hello 를 되풀이하지 않고 INSTALLDIR 위에 쌓는 것은 사용자가 설치 폴더를 고를 수 있게 되면(6장) 중요해진다: 그러면 docs 가 사용자가 고른 폴더를 따라간다. 알려진 폴더 전체 목록은 참조 부의 표에 있다.

4.4 파일 하나: [file.ID]

- name 은 파일을 다른 이름으로 설치한다 - 여기서는 readme.txt 가 Read me.txt 가 된다. 이름에는 어느 언어의 글자든 쓸 수 있다. Windows 가 파일 이름에 금지하는 것(< > : " / \ | ? *)은 거부한다.
- vital = false: 이 파일을 쓰지 못해도 설치하는 그 파일 없이 이어진다. 기본은 모든 파일이 필수다.
- keep = true: 제품을 지워도 파일이 남고, 사용자가 한 번 고친 뒤에는 복구나 다음 판이 덮어쓰지 않는다. 사용자가 고치는 설정에 쓴다.

4.5 여러 파일: [files.ID]

glob 은 와일드카드가 든 경로다: * 는 폴더 이름 하나 안의 아무 글자들, ? 는 글자 하나, ** 는 몇 단계든 폴더다. 맞는 파일이 모두 설치되고, 첫 와일드카드 아래의 폴더는 dir 아래에 다시 만들어진다:

glob	dist\samples\sub\sample2.txt 가 가는 곳
SamplesDir 로 dist/samples/**/*.txt	...\Hello\samples\sub\sample2.txt
SamplesDir 로 dist/samples/*.txt	설치되지 않음: * 는 sub 안으로 들어가지 않는다

맞은 파일은 이름 순으로 정렬되므로, 파일 시스템이 어떤 순서로 보이든 패키지는 같다. 아무것도 맞지 않는 그룹은 빌드를 멈춘다(RP1503) - 대개 경로의 오타다. rubrapack 은 맞은 파일마다 ID 를 따로 준다(F_ 와 경로에서 끌어낸 수).

4.6 빈 폴더: [folder.ID]

[folder.Data] 는 파일이 하나도 들어가지 않아도 INSTALLDIR 안에 data 를 만든다 - 프로그램이 쓸 자리다. keep = true 는 제품을 지울 때 그 폴더(와 프로그램이 거기 쓴 것)를 남긴다.

4.7 치우기: [remove.ID]

프로그램이 스스로 만드는 파일 - 기록, 캐시 - 은 설치 파일의 것이 아니어서 Windows Installer 가 남겨 두고, 그 때문에 폴더도 남는다. [remove.Logs] 는 제품을 지울 때(on = "uninstall") INSTALLDIR 의 *.log 를 지운다. on = "install" 은 설치할 때(옛 판이 남긴 파일), on = "both" 는 둘 다에서 지운다. name 이 없으면 폴더가 비었을 때 폴더 자체를 지운다. 설치 전부터 있던 폴더는 절대 지우지 않고, 설치가 실패하면 지운 파일을 되살린다.

4.8 두 번째 사본: [copy.ID]

[copy.ReadmeInDocs] 는 파일 Readme 의 사본을 하나 더 DocsDir 에 readme.txt 로 설치한다. 사본은 원래 파일과 함께 오고 함께 간다.

4.9 만날 수 있는 키 두 개 더

- 파일의 any-arch = true: rubrapack 은 모든 프로그램 파일(.exe, .dll)이 패키지의 arch 로 만들어졌는지 확인한다. 실수로 32비트 DLL 을 복사하는 x64 설치 파일은 망가진 프로그램을 설치하기 때문이다. x64 제품 안의 32비트 도우미처럼 일부러 그런 것이면 any-arch = true 로 밝힌다.
- 파일의 component-guid = "{...}": rubrapack 은 모든 컴포넌트의 GUID 를 제품과 파일 자리에서 끌어내므로, 판이 바뀌어도 같게 유지되고 업그레이드는 그것에 기댄다. 앞선 패키지가 다른 GUID 로 만든 컴포넌트를 이어 갈 때만 손으로 준다.

4.10 안에서 무슨 일이 일어났나

Windows Installer 는 *컴포넌트*를 설치한다: 함께 설치되고 함께 지워지는 작은 자원 묶음으로, 저마다 GUID 와 키 경로(그것이 있으면 "설치됨"을 뜻하는 것)를 가진다. rubrapack 은 파일마다 컴포넌트 하나를 만드는데 - 안전한 규칙이다 - 필요한 폴더, 지우기, 사본에도 하나씩 더 만든다:

```
C:\work\hello> rubrapack inspect hello.msi Component
...
C_74a883a037bc227f9189 {51A68DD4-96FA-83C0-86AA-F20D51339238} INSTALLDIR 272 Settings
C_cec3a9b89b2e391393d0 {615660C7-986E-8FB8-87AF-0616B544A6F9} Data 272
...
```

속성 열의 272 는 $256 + 16$ 이다: 256 은 64비트 컴포넌트, 16 은 "영구" - keep = true 다. 제4부가 이 표들을 설명한다. RemoveFile, CreateFolder, DuplicateFile 에 지우기, 빈 폴더, 사본이 들어 있다.

5 바로가기, 아이콘, "설치된 앱" 항목

이 장의 차례

5.1	원본	15
5.2	바로가기: [shortcut.ID]	16
5.3	"설치된 앱" 항목: [arp]	16
5.4	다른 글자로 된 이름	16
5.5	해 보기	16
5.6	안에서 무슨 일이 일어났나	16

목표: 시작 메뉴에 "Example Software" 폴더를 만들어 Hello 와 그 읽어보기를 넣고, 자기 아이콘을 가진 바탕화면 바로가기를 두고, "설치된 앱"의 Hello 에 아이콘과 링크를 단다.

원본 옆에 아이콘 파일 hello.ico 가 있어야 한다(아무 .ico 나 된다. 많은 그림 편집기가 저장할 수 있다).

5.1 원본

```
# tutorial 05: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"

[define]
VERSION = "1.3.0"

[arp]
icon = "hello.ico"
help = "https://example.com/hello/help"
about = "https://example.com/hello"
no-repair = true

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[dir.MenuFolder]
path = "Programs/Example Software"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[file.Readme]
dir = "INSTALLDIR"
source = "dist/readme.txt"

[shortcut.StartMenu]
dir = "MenuFolder"
name = "Hello"
target = "file:Hello"
description = "Says hello"
working-dir = "INSTALLDIR"
```

```
[shortcut.ReadmeLink]
dir = "MenuFolder"
name = "Hello - read me"
target = "file:Readme"

[shortcut.Desktop]
dir = "Desktop"
name = "Hello"
target = "file:Hello"
args = "--greet \"[ProductName] user\""
icon = "hello.ico"
```

5.2 바로가기: [shortcut.ID]

- dir 은 바로가기가 들어갈 곳이다: 내 dir, 또는 바로 Programs(시작 메뉴의 프로그램 목록), StartMenu, Desktop, Startup(로그인할 때마다 실행). 여기서 dir MenuFolder 는 시작 메뉴 프로그램 안의 "Example Software" 폴더라서, 시작 메뉴에 "Example Software" 와 그 안의 항목 둘이 보인다.
- name 은 바로가기 아래의 글이다.
- target 은 바로가기가 여는 파일이다: file: 과 이 패키지의 [file.*] ID.
- description 은 도움말 풍선, working-dir(dir ID)은 프로그램이 시작하는 폴더다.
- args 는 프로그램의 인자다. MSI *서식 문자열*이어서 바로가기를 만들 때 [ProductName] 이 "Hello" 가 된다. TOML 문자열 안의 큰따옴표는 \" 로 쓴다.
- icon 은 바로가기에 자기 .ico 를 준다. 없으면 Windows 는 프로그램 자체의 아이콘을 보인다.

rubrapack 이 바로가기를 위해 만든 시작 메뉴 폴더는 바로가기와 함께 제품을 지울 때 사라진다.

5.3 "설치된 앱" 항목: [arp]

("ARP" 는 이 목록의 옛 이름 Add or Remove Programs(프로그램 추가/제거)다.)

키	효과
icon = "hello.ico"	목록의 아이콘
help = "https://..."	Windows 가 제품과 함께 보이는 지원 링크
about = "https://..."	제품의 웹 페이지
no-repair = true	이 제품에 복구가 없다
no-modify = true	수정/변경이 없다(패키지에 바꿀 것이 없을 때 쓸모 있다. 8장이 바꿀 것을 더한다)

5.4 다른 글자로 된 이름

name 은 어느 언어로든 쓸 수 있다(name = "안녕"). 그런데 .msi 파일 자체의 속성(자세히 탭)은 ASCII 글자만 보일 수 있어서, 그런 이름에는 ASCII 형태도 준다: [package] 의 summary-name = "Annyeong". 없으면 rubrapack 이 경고하고(RP1203) 속성에는 "rubrapack package" 가 보인다.

5.5 해 보기

빌드하고 설치한 뒤 시작 메뉴를 연다: "Example Software" 에 "Hello" 와 "Hello - read me" 가 있고, 바탕 화면에는 준 아이콘을 단 "Hello" 가 있다. "설치된 앱"에는 아이콘과 함께 Hello 가 보인다. 지우면 바로가기와 시작 메뉴 폴더도 사라진다.

5.6 안에서 무슨 일이 일어났나

```
C:\work\hello> rubrapack inspect hello.msi Shortcut
Shortcut Directory_ Name Component_ Target Arguments Description Hotkey
```

```
...
Desktop DesktopFolder Hello C_185f8db32271fe25f561 [#Hello] --greet "[ProductName] user"
ReadmeLink MenuFolder HELLO~1\Hello - read me C_44ff26388205705fb806 [#Readme]
StartMenu MenuFolder Hello C_185f8db32271fe25f561 [#Hello] Says hello
```

- 바로가기는 대상 파일의 컴포넌트에 속한다: 파일과 함께 설치되고 지워진다.
 - [#Hello] 는 "파일 Hello 가 설치된 경로"를 뜻하는 Windows Installer 의 표기다.
 - HELLO~1\Hello - read me 에는 이름이 둘 들어 있다: 옛 방식의 짧은 이름(8자, 점, 3자 - MS-DOS 시절부터)과 긴 이름. Windows Installer 는 모든 이름에 둘 다를 원하고, 짧은 이름은 rubrapack 이 지어낸다.
- 아이콘은 패키지 안(Icon 표)에 저장되고, "설치된 앱" 설정은 Property 표에 들어 있다:

```
ARPHHELPLINK https://example.com/hello/help
ARNOREPAIR 1
ARPPRODUCTICON RpIcon1.ico
ARPURLINFOABOUT https://example.com/hello
```

6 대화창, 약관, 그리고 프로그램 시작하기

이 장의 차례

6.1	원본	18
6.2	페이지 세트 고르기: ui	19
6.3	약관: license	19
6.4	설치 폴더	19
6.5	위쪽의 그림: banner	19
6.6	내 말로: [ui-text.ID]	19
6.7	끝에 프로그램 시작하기: launch	19
6.8	해 보기	20
6.9	안에서 무슨 일이 일어났나	20

목표: 페이지가 있는 설치 파일 - 환영, 사용자가 동의해야 하는 약관, 폴더 고르기, 진행, 완료 - 을 만들고, 끝에 Hello 를 시작하겠냐고 묻게 한다.

6.1 원본

원본 옆에 LICENSE.txt 와 banner.bmp(약 493 x 58 픽셀 그림)가 있다.

```
# tutorial 06: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
ui = "installdir"
license = "LICENSE.txt"

[define]
VERSION = "1.4.0"

[ui]
banner = "banner.bmp"
launch = "file:Hello"
launch-args = "--first-run"
launch-checked = true

[ui-text.WelcomeText]
text = "This will install [ProductName] [ProductVersion]. Close Hello if it is running."

[ui-text.ExitText]
text = "[ProductName] is ready. Thank you for installing it."

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[shortcut.StartMenu]
```

```
dir = "Programs"
name = "Hello"
target = "file:Hello"
```

6.2 페이지 세트 고르기: ui

[package] 의 ui 는 내장 대화창 세트 하나를 고른다:

ui	설치할 때	설치 뒤 다시 실행할 때
(없음)	Windows Installer 자체의 작은 진행 창만	같음
basic	진행, 그다음 완료(또는 오류)	진행, 완료
minimal	환영, 약관(있으면), 진행, 완료	복구 또는 제거
installldir	환영, 약관, 설치 폴더(폴더 찾아보기 포함), 준비, 진행, 완료	복구 또는 제거
features	installldir 과 같고, 기능 트리과 그것이 필요로 하는 디스크 공간이 더해진다(8장)	복구, 변경, 제거

어느 세트에나 무슨 일이 생겼을 때 Windows 가 필요로 하는 페이지도 있다: "정말 취소하겠습니까?", 오류 메시지, "이 프로그램들이 바뀌어야 할 파일을 쓰고 있습니다"(사용 중인 파일), "디스크 공간 부족". 대화창이 있는 패키지도 /qn 으로 조용히 설치된다: 모든 페이지는 기본값이 있는 값만 모으기 때문이다.

6.3 약관: license

license = "LICENSE.txt" 는 환영 페이지 뒤에 약관 페이지를 더한다. "I accept the terms of the license agreement"(사용권 계약에 동의합니다)에 표시하기 전에는 다음 단추가 꺼져 있다. 파일은:

- .txt 또는 .md - 평문으로 보인다. 한국어, 다른 글자, 이모지도 그대로다.
- .rtf - 글꼴과 서식을 그대로 보인다(워드패드나 Word 로 만든다).

약관 글은 빌드할 때 패키지 안에 저장되므로 사용자에게 그 파일이 필요하지 않다.

6.4 설치 폴더

installldir 이나 features 에서는 설치 폴더와, 폴더 찾아보기를 여는 변경 단추가 있는 페이지가 보인다. 사용자가 고른 폴더가 INSTALLDIR 이 되고, INSTALLDIR 위에 쌓은 dir 은 모두 함께 옮겨 간다. 다른 dir 을 바꾸게 하려면 이름을 낸다: [ui] 의 install-dir = "AppDir". 명령줄에서는 폴더를 속성으로 준다:

```
msiexec /i hello.msi INSTALLDIR="D:\Tools\Hello"
```

6.5 위쪽의 그림: banner

banner = "banner.bmp" 는 안쪽 페이지마다 위쪽 띠에 그림을 넣는다(Windows Installer 는 BMP 를 그린다. 약 493 x 58 픽셀). 없으면 띠는 그냥 흰색이다.

6.6 내 말로: [ui-text.ID]

내장 페이지의 모든 글에는 ID 가 있고 [ui-text.ID] 가 하나를 바꾼다. ID 목록은 참조 부의 대화창에 있다. WelcomeText 와 ExitText 는 첫 페이지와 마지막 페이지의 문장이다. 글은 서식 문자열이다: [ProductName], [ProductVersion], [Manufacturer] 는 바뀌고, 대괄호 자체는 [\] 로 쓴다. 단추 글에서 & 는 Alt 와 함께 쓰는 글자를 표시한다: &Next 는 Alt+N 이다.

6.7 끝에 프로그램 시작하기: launch

launch = "file:Hello" 는 완료 페이지에 "Launch Hello"(Hello 실행) 확인란을 넣는다(launch-checked = false 가 아니면 표시된 채로). launch-args 는 프로그램의 인자다. 표시되어 있으면 마침 단추가 Hello 를 시작한다 - 설치 파일의 관리자 권한이 아니라 설치를 실행한 사용자의 권한으로 - 첫 설치나 업그레이드 뒤에만이고, 복구나 제거 뒤에는 아니며, 조용한 설치에서는 절대 아니다.

6.8 해 보기

hello.msi 를 두 번 누른다: 환영(내 문구와 배너), 약관(확인란을 표시할 때까지 다음이 회색), 설치 폴더, 준비, 진행, "Launch Hello" 가 표시된 완료. 설치한 뒤 다시 실행하면 복구와 제거를 원하는 페이지가 나온다.

6.9 안에서 무슨 일이 일어났나

대화창이 있는 패키지는 그것을 표로 지닌다 - Dialog, Control, ControlEvent 등 - 이 표들이 모든 창, 단추, 글상자를 위치와 눌렀을 때 일어나는 일과 함께 기술한다. rubrapack 은 내장 세트에서 이것을 쓴다:

```
C:\work\hello> rubrapack inspect hello.msi Dialog
...
RpBrowseDlg RpCancelDlg RpErrorDlg RpExitDlg RpFatalDlg RpInstallDirDlg RpLicenseDlg
RpMaintenanceDlg RpOutOfDiskDlg RpProgressDlg RpReadyDlg RpUserExitDlg RpWelcomeDlg FilesInUse
```

(여기서는 첫 열만 보였다.) InstallUISequence 표는 페이지가 언제 나오는지 말한다:

```
RpWelcomeDlg NOT Installed 1230
RpMaintenanceDlg Installed AND NOT RESUME AND NOT Preselected 1240
RpProgressDlg 1280
RpExitDlg -1
RpUserExitDlg -2
RpFatalDlg -3
```

가운데 열은 *조건*이다: 환영 페이지는 제품이 아직 설치되지 않았을 때만, 유지 관리 페이지는 설치되어 있을 때 나온다. 음수는 특별하다: 성공한 끝(-1), 사용자가 취소한 끝(-2), 실패한 끝(-3)에 보이는 페이지다. 9장에서 내 조건을 쓴다.

7 여러 언어

이 장의 차례

7.1	원본	21
7.2	언어 더하기: languages	22
7.3	언어마다 약관: license-xx	22
7.4	언어마다 글: text-xx	22
7.5	영어와 한국어 말고 다른 언어	22
7.6	번역되지 않는 것	22
7.7	해 보기	22
7.8	안에서 무슨 일이 일어났나	22

목표: 영어나 한국어로 말하는 패키지 하나를 만든다. 약관도 언어마다 따로 두고, 한국어 Windows에서는 한국어가 미리 골라진다.

7.1 원본

한국어 약관 LICENSE-ko.txt 가 LICENSE.txt 옆에 있다.

```
# tutorial 07: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${(VERSION)}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
ui = "installdir"
license = "LICENSE.txt"
downgrade-message = "A newer Hello is already installed."

[define]
VERSION = "1.5.0"

[ui]
languages = ["ko"]
license-ko = "LICENSE-ko.txt"
launch = "file:Hello"

[ui-text.WelcomeText]
text = "This will install [ProductName] [ProductVersion].\"
text-ko = "[ProductName] [ProductVersion]을(를) 설치합니다.\"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[shortcut.StartMenu]
dir = "Programs"
name = "Hello"
target = "file:Hello"
```

7.2 언어 더하기: languages

페이지는 늘 영어다. [ui] 의 languages = ["ko"] 는 같은 패키지에 한국어를 더하고, 그러면:

- 첫 페이지가 어떤 언어를 쓸지 묻는다 - English 또는 한국어.
- 지역 형식(없으면 시스템 언어)이 한국어인 Windows 에서는 한국어가 이미 골라져 있다. 다른 곳에서는 English.
- 그 뒤의 모든 페이지 - 환영, 약관, 폴더, 준비, 진행, 완료, 그리고 취소, 오류, 사용 중인 파일, 디스크 공간, 유지 관리 페이지 - 가 고른 언어로 말한다.

영어만 있으면 언어 페이지가 없다. 조용한 설치(/qn)는 페이지를 보이지 않으므로 고를 필요가 없다. 명령줄의 RPLANGUAGE=ko 는 페이지 없이 고른다:

```
msiexec /i hello.msi RPLANGUAGE=ko
```

7.3 언어마다 약관: license-xx

license-ko = "LICENSE-ko.txt" 는 한국어를 골랐을 때 보이는 약관이다. 영어(와 자기 약관이 없는 더한 언어) 는 [package] license 를 보인다. .txt / .md / .rtf 규칙은 같다.

7.4 언어마다 글: text-xx

[ui-text.ID] 에서 text 는 모든 언어의 글, text-ko 는 한국어 글이다. 내장 글은 모두 이미 영어와 한국어로 있다. text 를 준 곳에만 text-ko 를 주어 두 언어가 같은 말을 하게 한다.

7.5 영어와 한국어 말고 다른 언어

rubrapack 의 내장 페이지는 영어와 한국어로 쓰여 있다. 다른 언어 - ja, de, fr 등 - 는 모든 글을 text-xx 로 직접 준다. 빠진 것은 빌드가 알려 준다:

```
hello.toml:16:1: error[RP1202]: language 'ja' has no built-in texts: give [ui-text.ID] text-ja for all of them (64 missing, the first is Back)
```

그런 언어에는 [ui] 의 키 셋이 더 도움이 된다: name-ja(언어 페이지에 보일 이름), font-ja(그 페이지들의 글꼴), langid-ja(그 언어를 미리 고르게 하는 Windows 언어 번호 - 수 하나나 목록). 흔한 언어(ja, zh, de, fr, es, it, pt, nl, pl, ru, uk, tr, vi, th)에는 이 셋이 내장되어 있다. 한국어 페이지는 맑은 고딕, 영어 페이지는 Segoe UI 글꼴을 쓴다.

7.6 번역되지 않는 것

- Windows Installer 자체의 말 - 기능 트리의 크기와 메뉴, "남은 시간", 오류 메시지 - 은 패키지가 아니라 Windows 에서 온다.
- 다운그레이드 안내문(3장)과 refuse-upgrade-below 의 제거 안내문은 글이 하나씩이다. 두 언어가 다 필요하면 두 언어로 쓴다("A newer Hello is already installed. / 더 새 Hello 가 설치되어 있습니다.").
- [package] language = "ko-KR" 는 패키지 자체의 언어 코드만 정한다. 페이지를 한국어로 만드는 것은 languages 다.

7.7 해 보기

한국어 Windows 에서 hello.msi 를 두 번 누른다: 언어 페이지에 한국어가 골라져 있고, 그 뒤 모든 페이지가 한국어 약관과 함께 한국어다. English 를 고르면 같은 페이지가 LICENSE.txt 와 함께 영어로 나온다. 영어 Windows 에서 msiexec /i hello.msi RPLANGUAGE=ko 도 해 본다.

7.8 안에서 무슨 일이 일어났나

Windows Installer 패키지에는 "여러 언어"라는 개념이 따로 없다: 페이지의 글은 그 표에 고정되어 있다. 그래서 rubrapack 은 모든 글을 속성 - 이름 붙은 값 - 으로 만들고 페이지에는 그 속성을 보인다:

```
C:\work\hello> rubrapack inspect hello.msi Property
...
RpT_WelcomeText This will install Hello 1.5.0.
RpT_WelcomeText_en This will install Hello 1.5.0.
RpT_WelcomeText_ko Hello 1.5.0을(를) 설치합니다.
...
```

첫 페이지 전에 작은 동작들이 고른 언어의 글을 보이는 속성에 복사한다. 각 동작은 조건이 참일 때만 돈다:

```
C:\work\hello> rubrapack inspect hello.msi InstallUISequence
...
RpL_ko_26 RPLANGUAGE = "ko" 21
...
C:\work\hello> rubrapack inspect hello.msi CustomAction
...
RpL_ko_26 51 RpT_WelcomeText [RpT_WelcomeText_ko]
```

형식 51 은 "속성 정하기"다: RpT_WelcomeText 가 RpT_WelcomeText_ko 의 값이 된다. 글은 패키지에 UTF-8(코드 페이지 65001)로 저장된다. 글자 인코딩은 제3부, 문자열 저장 방식은 제4부가 설명한다.

8 선택 구성요소: 기능과 사용자의 선택

이 장의 차례

8.1	기능	24
8.2	원본	24
8.3	기능 선언하기: [feature.ID]	25
8.4	파일을 기능에 넣기: feature	26
8.5	사용자가 보는 것	26
8.6	명령줄에서 고르기	26
8.7	나중에 바꾸기	26
8.8	안에서 무슨 일이 일어났나	27

목표: 무엇을 설치할지 사용자가 고르게 한다. 프로그램 자체는 늘 설치하고, 문서는 보여 주며 표시된 채로 두고, 예제는 보여 주되 표시하지 않고, 진단 부분은 숨겨 두었다가 요청할 때만 설치한다.

8.1 기능

*기능(feature)*은 사용자가 고를 수 있는 제품의 한 부분이다: 설치 파일의 트리에서 제목, 설명, 표시가 있는 한 줄이다. 모든 파일은 정확히 한 기능에 속하고, 한 기능에는 파일이 몇 개든 들어갈 수 있다.

8.2 원본

```
# tutorial 08: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
ui = "features"
license = "LICENSE.txt"

[define]
VERSION = "1.6.0"

[feature.Main]
title = "Hello"
description = "The program itself."
required = true

[feature.Documentation]
title = "Documentation"
description = "The guide, in the docs folder."

[feature.Samples]
title = "Samples"
description = "Example files to try Hello with."
level = 2

[feature.MoreSamples]
title = "More samples"
description = "Samples in sub folders."
parent = "Samples"
```

```

follow-parent = true

[feature.Diagnostics]
title = "Diagnostics"
hidden = true
level = 3

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"
feature = "Main"

[dir.DocsDir]
path = "INSTALLDIR/docs"
feature = "Documentation"

[dir.SamplesDir]
path = "INSTALLDIR/samples"
feature = "Samples"

[dir.MoreSamplesDir]
path = "INSTALLDIR/samples/sub"
feature = "MoreSamples"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[files.DocFiles]
dir = "DocsDir"
glob = "dist/docs/**"

[files.SampleFiles]
dir = "SamplesDir"
glob = "dist/samples/*.txt"

[files.MoreSampleFiles]
dir = "MoreSamplesDir"
glob = "dist/samples/sub/*.txt"

[file.Readme]
dir = "INSTALLDIR"
source = "dist/readme.txt"
feature = "Diagnostics"

[shortcut.StartMenu]
dir = "Programs"
name = "Hello"
target = "file:Hello"

```

8.3 기능 선언하기: [feature.ID]

키	뜻
title	트리의 줄
description	그 줄을 골랐을 때 보이는 글
level	1(기본): 기본으로 설치. 2 이상: 보여 주지만 고를 때만 설치
required = true	늘 설치: 트리가 "사용할 수 없음"을 제안하지 않는다
parent = "Samples"	트리에서 다른 기능 아래의 줄
follow-parent = true	부모가 설치될 때 정확히 함께 설치

키	뜻
hidden = true	트리에 아예 보이지 않음

기능 ID 는 dir, 파일과 이름 공간을 함께 쓴다: 기능 Documentation 과 폴더 DocsDir 는 ID 가 달라야 한다(아니면 rubrapack 이 RP1301 로 알린다).

8.4 파일을 기능에 넣기: feature

- dir 에: 그 폴더의 모든 파일이 그 기능에 속한다 - DocsDir 는 guide.txt 를 Documentation 에, SamplesDir 는 sample1.txt 를 Samples 에 넣는다.
- 파일이나 파일 묶음에: 그 파일(묶음)이 폴더와 상관없이 그 기능에 속한다 - Readme 는 Main 의 폴더인 INSTALDIR 에 있지만 Diagnostics 에 속한다.

dir 은 자기 기능을 아래 dir 에 물려주지 않는다: MoreSamplesDir 는 자기 기능을 따로 낸다. 원본이 기능을 하나라도 선언하면 설치되는 모든 것에 기능이 필요하다. 기능이 없는 것은 rubrapack 이 RP1202 로 짚고 멈춘다.

8.5 사용자가 보는 것

ui = "features" 에서는 약관과 설치 폴더 다음에 "Choose features"(기능 고르기)가 나온다: 트리 하나

```
[x] Hello                (끌 수 없음)
[x] Documentation
[ ] Samples
    [ ] More samples      (Samples 를 따라감)
```

줄마다 메뉴가 있다: "Will be installed on local hard drive"(로컬 하드 드라이브에 설치), "Entire feature will be installed on local hard drive"(하위까지 모두 설치), "Entire feature will be unavailable"(모두 사용할 수 없음). 줄을 고르면 설명과 필요한 디스크 공간이 보이고, "Disk Usage" 는 드라이브마다 공간을 보인다. (이 글들은 Windows 에서 옴으로 영어로 남는다.) Diagnostics 는 보이지 않는다.

기본 설치(두 번 누르고 다음, 다음, ...)는 Hello 와 문서를 설치한다.

8.6 명령줄에서 고르기

기능은 속성으로 고르므로 조용한 설치도 고를 수 있다:

명령	설치하는 것
msiexec /i hello.msi /qn	기본: Main, Documentation
msiexec /i hello.msi /qn ADDLOCAL=Samples	기본과 예제(따라가는 More samples 포함)
msiexec /i hello.msi /qn ADDLOCAL=ALL	숨긴 것까지 모든 기능
msiexec /i hello.msi /qn INSTALLLEVEL=3	level 이 3 이하인 모든 기능: 여기서는 전부
msiexec /i hello.msi /qn REMOVE=Documentation	문서를 뺀 기본

기능 ID 는 대소문자를 가리고 쉼표로 나눈다: ADDLOCAL=Samples,Diagnostics.required 는 트리만 묶는다: 명령줄의 REMOVE=Main 은 여전히 따른다(Windows Installer 가 그렇게 동작한다).

8.7 나중에 바꾸기

설치한 뒤 "설치된 앱"은 Hello 에 수정을 보인다. 설치 파일의 유지 관리 페이지가 열리고 변경, 복구, 제거가 있다. 변경은 설치된 것이 표시된 같은 트리를 보이고, 사용자는 부분을 더하거나 뺀다. 설치된 제품에 명령줄로는:

```
msiexec /i hello.msi ADDLOCAL=Samples
msiexec /i hello.msi REMOVE=Samples
```

8.8 안에서 무슨 일이 일어났나

```
C:\work\hello> rubrapack inspect hello.msi Feature
Feature Feature_Parent Title Description Display Level Directory_ Attributes
...
Diagnostics Diagnostics 0 3 0
Documentation Documentation The guide, in the docs folder. 3 1 0
Main Hello The program itself. 1 1 16
MoreSamples Samples More samples Samples in sub folders. 7 1 2
Samples Samples Example files to try Hello with. 5 2 0
```

- Display 는 트리의 순서다(원본 순서의 홀수). 0 은 기능을 숨긴다.
- Level 은 수준이다. Windows 는 수준이 속성 INSTALLLEVEL(1)보다 높지 않은 기능을 기본으로 설치한다.
- Attributes 는 비트 플래그다: 16 은 "없앨 수 없음"(required), 2 는 "부모를 따라감".

FeatureComponents 표는 컴포넌트 - 파일 - 마다 그 기능을 잇는다:

```
C:\work\hello> rubrapack inspect hello.msi FeatureComponents
...
Documentation C_a10be66379e69940a2e0
Main C_185f8db32271fe25f561
Samples C_a5674982edf0797da30f
...
```

9 조건과 검색

이 장의 차례

9.1	속성	28
9.2	원본	28
9.3	내 속성: [property.NAME]	29
9.4	이럴 때만 설치하기: when	30
9.5	설치 거부하기: [require.ID]	30
9.6	먼저 둘러보기: [search.ID]	30
9.7	설치 폴더 기억하기	30
9.8	안에서 무슨 일이 일어났나	30

목표: 무엇을 설치할지 사용자 컴퓨터에서 정한다. 64비트 Windows 에서만 설치하고, 메모장이 있는 곳에서만 한 부분을 권하고, 바탕화면 바로가기 사용자를 끌 수 있게 하고, 다음 판은 사용자가 이번 판에 고른 폴더에 설치한다.

9.1 속성

Windows Installer 는 실행하는 동안 *속성*이라는 이름 붙은 값을 지닌다: ProductName, INSTALLDIR, VersionNT64(64비트 Windows 에서 정해짐), 그리고 내 것들. 이름이 모두 대문자인 속성은 *공개*다: 명령줄 (msiexec /i hello.msi DESKTOP_SHORTCUT=0)과 대화창이 정할 수 있다. *조건*은 속성에 대한 작은 식으로, 참이거나 거짓이다:

조건	참일 때
VersionNT64	속성이 정해져 있을 때(비어 있지 않음): 여기서는 64비트 Windows
NOT Installed	제품이 아직 설치되지 않았을 때(첫 설치)
DESKTOP_SHORTCUT = "1"	속성 값이 정확히 1 일 때
VersionNT >= 603	Windows 판 번호가 6.3 이상일 때
A AND (B OR NOT C)	생각하는 대로 조합

9.2 원본

```
# tutorial 09: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${(VERSION)}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
ui = "features"

[define]
VERSION = "1.7.0"

[property.DESKTOP_SHORTCUT]
value = "1"
secure = true

[require.Windows64]
```

```

condition = "VersionNT64"
message = "[ProductName] needs 64-bit Windows."

[search.Notepad]
property = "NOTEPAD_PATH"
kind = "file"
path = "Windows"
file = "notepad.exe"

[search.PreviousDir]
property = "INSTALLDIR"
kind = "registry"
root = "HKLM"
key = "Software\\Example Software\\Hello"
name = "InstallDir"

[feature.Main]
title = "Hello"
required = true

[feature.NotepadHelper]
title = "Open notes in Notepad"
description = "Only offered where Notepad is installed."
when = "NOTEPAD_PATH"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"
feature = "Main"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[file.Settings]
dir = "INSTALLDIR"
source = "dist/settings.ini"
feature = "NotepadHelper"

[registry.RememberDir]
root = "HKLM"
key = "Software\\Example Software\\Hello"
name = "InstallDir"
value = "[INSTALLDIR]"
feature = "Main"

[shortcut.StartMenu]
dir = "Programs"
name = "Hello"
target = "file:Hello"

[shortcut.Desktop]
dir = "Desktop"
name = "Hello"
target = "file:Hello"
when = "DESKTOP_SHORTCUT = \"1\""

```

9.3 내 속성: [property.NAME]

[property.DESKTOP_SHORTCUT] 은 기본값 1 로 속성을 만든다. 공개가 되려면 이름이 모두 대문자여야 한다. Windows 나 rubrapack 이 스스로 쓰는 이름(ALLUSERS, ARP..., MSI... 등)은 거부한다.

- `secure = true`: 모든 사용자를 위한 설치의 두 부분으로 돈다 - 사용자 부분(대화창)과, 컴퓨터를 바꾸는 권한 높은 부분. 첫 부분이나 명령줄에서 정한 값은 속성이 *보안(secure)*일 때만 둘째 부분에 닿는다. `when` 조건이 보는 속성은 모두 보안으로 한다. (내 대화창 페이지의 속성과 검색 결과는 rubrapack 이 알아서 그렇게 한다.)
- `hidden = true`: 값을 설치 기록에 쓰지 않는다 - 암호와 키를 위한 것이다.

9.4 이럴 때만 설치하기: `when`

기능, 파일이나 파일 묶음, 바로가기, 레지스트리 값, 환경 변수, INI 값에 `when` 을 달면 첫 설치 때 조건이 참일 때만 설치한다:

- 바탕화면 바로가기의 `when = "DESKTOP_SHORTCUT = \"1\""`: 기본으로는 만들고, `msiexec /i hello.msi DESKTOP_SHORTCUT=0` 이면 만들지 않는다. 14장에서 이 속성을 내 페이지의 확인란으로 바꾼다.
- 기능의 `when = "NOTEPAD_PATH"`: 조건이 거짓인 곳에서는 그 기능을 설치하지도 트리에 보이지도 않는다.

복구는 설치된 것을 그대로 두고, 메이저 업그레이드(다음 판)는 다시 정한다.

9.5 설치 거부하기: `[require.ID]`

요구 조건은 `condition` 이 거짓이면 첫 설치를 `message` 와 함께 멈춘다 - 여기서는 32비트 Windows 에서: "Hello needs 64-bit Windows." 복구와 제거는 절대 막지 않는다.

9.6 먼저 둘러보기: `[search.ID]`

검색은 다른 무엇보다 먼저 돌고, 찾은 것을 속성에 넣는다(못 찾으면 비움). 조건이 그것을 볼 수 있다:

kind	찾는 것	키
file	파일의 전체 경로	path(알려진 폴더와 상대 경로), file, min-version(프로그램용)
dir	폴더	path
registry	레지스트리의 값	root, key, name, view(32비트 보기는 32)
component	다른 제품 컴포넌트의 키 파일	component-guid

`[search.Notepad]` 는 Windows 폴더에서 `notepad.exe` 를 찾아 경로를 `NOTEPAD_PATH` 에 넣는다.

9.7 설치 폴더 기억하기

사용자가 Hello 를 `D:\Tools\Hello` 에 설치했다면 다음 판도 Program Files 로 돌아가지 말고 그리 가야 한다. 두 조각으로 한다:

1. `[registry.RememberDir]` 는 설치할 때 폴더를 레지스트리에 적는다: 값 안의 `[INSTALLDIR]` 은 폴더로 바뀐다 (레지스트리는 11장).
2. `[search.PreviousDir]` 는 그것을 다시 읽는다. `property` 가 `dir ID INSTALLDIR` 이므로 찾은 것이 그 `dir` 의 *기본값*이 된다 - 그 폴더가 아직 있을 때만이고, 명령줄로 준 폴더가 여전히 이긴다.

9.8 안에서 무슨 일이 일어났나

```
C:\work\hello> rubrapack inspect hello.msi LaunchCondition
Installed OR (VersionNT64) [ProductName] needs 64-bit Windows.
C:\work\hello> rubrapack inspect hello.msi AppSearch
NOTEPAD_PATH Notepad
RpFound_PreviousDir PreviousDir
C:\work\hello> rubrapack inspect hello.msi Condition
NotepadHelper 0 NOT Installed AND NOT (NOTEPAD_PATH)
```

(머리줄은 뺐다.) rubrapack 은 요구 조건에 `Installed OR` 를 붙여, 설치된 제품은 늘 복구하거나 지울 수 있게 한다. 기능의 `when` 은 `Condition` 표의 한 행이 되어 조건이 거짓이면 수준을 0 - "설치 안 함, 안 보임" - 으로 만

든다. NOT Installed 는 속성이 사라진 제거 때 이 일이 일어나지 않게 막는다. 바탕화면 바로가기는 컴포넌트를 따로 받았고, 그 Condition 열에 when 이 들어 있다:

```
C_2ea6d5bfd9b4a9073b30 {BCBFF063-...} DesktopFolder 260 DESKTOP_SHORTCUT = "1" R_7f1795ed8aa7ac1a1c60
```

제2부

참조

10 원본 파일과 rubrapack 명령

이 장의 차례

10.1	시작하기	34
	첫 패키지	34
	약관 페이지와 선택 구성요소	35
	Windows 에서 시험하기	36
	무언가 잘못됐을 때	36
	자동화하기	36
	AI 도우미와 함께	36
10.2	예	36
10.3	TOML 부분집합	37
10.4	표	37
	MSIX 패키지	38
	설치한 프로그램으로 등록하기: [action.ID]	39
	레지스트리 값: [registry.ID]	40
	바로가기: [shortcut.ID]	40
	파일 형식과 링크: [assoc.ID], [protocol.ID]	40
	MSIX 전용: [msix-extension.ID]	41
	지우기와 복사: [remove.ID], [copy.ID]	41
	환경 변수: [env.ID]	41
	INI 파일: [ini.ID]	41
	검색과 요구: [search.ID], [require.ID]	41
	서비스, 글꼴, 권한	42
	사용자별 패키지와 검용 패키지: scope	42
	캐비닛	42
	설치된 앱 항목과 속성	42
	명령줄로 설치하기	42
	창 없이 설치하기	43
	대화창: ui	43
	여러 언어	43
	아이콘, 완료 페이지, 설치 범위 페이지	44
	나만의 대화창 페이지: [dialog.ID], [dialog-control.ID]	44
	아키텍처와 업그레이드 계열	45
	먼저 지워야 하는 판	45
	와일드카드: [files.ID]	46
	빈 폴더: [folder.ID]	46
	기능(feature)	46
	조건: when	46
	제거할 때 파일 남기기: keep	46
	변수	46
	프로그램 파일	47
	설치 폴더 지키기	47
	경로	47
10.5	명령줄	47
10.6	진단 코드	50

원본 파일 하나가 패키지 하나를 기술한다. 형식은 TOML 1.0의 엄격한 부분집합이고 이름은 *.toml 이라서, 편집기와 GitHub, AI 도우미가 TOML 로 알아본다(예전 이름 *.rpk 도 똑같이 읽는다). 모든 원본은 올바른 TOML 이지만, 부분집합 밖의 TOML 기능은 무시하지 않고 오류로 거부한다.

10.1 시작하기

rubrapack 은 프로그램 파일 하나다. 릴리스 페이지에서 rubrapack-<판>-windows-x64.exe(원하면 rubrapack.exe 로 이름을 바꾼다)나 rubrapack-<판>-linux-x86_64(chmod +x 한다)를 내려받아 그 자리에서 실행하거나 PATH 에 둔다. 따로 설치할 것은 없다: 런타임도, SDK 도, 라이브러리도 필요 없다. 같은 프로그램이 Windows 에서도 Linux 에서도 같은 패키지를 만든다.

첫 패키지

배포할 것을 dist/ 폴더에 둔다: 프로그램, 그리고 그것이 필요로 하는 파일은 dist/files/ 에(하위 폴더도 그대로 간다). 그다음 rubrapack 에게 시작할 원본을 쓰게 한다:

```
rubrapack new app.toml      # 묻고 나서 app.toml 을 쓰고 검사한다
```

제품 이름, 판, 파일이 든 폴더, 주 프로그램(아키텍처는 파일에서 읽는다), Program Files 아래 폴더, 누구를 위해 설치하는지, 어떤 하위 폴더가 선택 구성요소인지, 대화창, 약관(옆에 LICENSE.txt, .md, .rtf 가 있으면 권한다), 한국어 대화창, 바로가기를 묻는다. Enter 는 대괄호 안의 값을 쓴다. 끝에는 같은 답을 한 줄 명령(rubrapack new app.toml --dist dist --ui features ...)으로 보여 주는데, 스크립트나 AI 도우미는 묻지 않고 이것을 돌리면 된다. 이름 없이 rubrapack new 만 주면 파일 이름까지 묻고, rubrapack new app(확장자 없이)은 대신 고정된 틀을 쓴다.

어느 쪽이든 app.toml 은 읽고 고칠 수 있는 평문이다 - 아무 편집기로나, 또는 rubrapack edit app.toml 으로. edit 는 같은 질문을 지금 값을 기본으로 보여 주는 메뉴다(판, 설치 폴더, 대화창과 약관, 선택 구성요소, 바로가기, 그리고 프로그램 폴더가 바뀐 뒤 파일 목록 맞추기). 그 값만 바꾸고 다른 줄, 주석, 표는 그대로 둔다. 스크립트에서는: rubrapack edit app.toml --set define.VERSION=1.1.0 --sync. 아래 원본은 프로그램과 파일을 Program Files\My App 에 설치하고, 시작 메뉴에 넣고, 사용자가 폴더를 바꿀 수 있는 대화창을 보인다:

```
[package]
name = "My App"
manufacturer = "My Company"          # 설정 > 설치된 앱에 보인다
version = "${VERSION}"
arch = "x64"                          # x64, arm64, x86
upgrade-code = "{E8C1815C-CCD7-4F3F-B914-92A4E3F3A317}" # `new` 가 준 것; 영원히 간직한다
ui = "installdir"

[define]
VERSION = "1.0.0"

[dir.INSTALLDIR]
path = "ProgramFiles/My App"

[file.App]
dir = "INSTALLDIR"
source = "dist/app.exe"

[files.Rest]
dir = "INSTALLDIR"
glob = "dist/files/**"

[shortcut.StartMenu]
dir = "Programs"
name = "My App"
target = "file:App"
```

```
rubrapack build app.toml -o app-1.0.0.msi
rubrapack build app.toml -o app-1.0.1.msi -D VERSION=1.0.1    # 다음 판
```

이것으로 완전한 설치 파일이다. "설치된 앱"에 나타나고, 스스로 복구하고, 설치한 것을 모두 지운다. 더 높은 판은 설치된 판을 대체하고, 같거나 낮은 판은 안내문과 함께 거부된다. 컴포넌트 GUID, 파일 키, 캐비닛과 표는 원본에서 끌어내므로, 판에서 판으로 기억해야 할 것은 업그레이드 코드뿐이다.

약관 페이지와 선택 구성요소

설치 파일이라면 흔히 바라는 두 가지: 설치 전에 사용자가 약관에 동의하는 것, 그리고 일부 구성요소를 고를 수 있는 것. license 는 약관 페이지를 더하고("동의합니다"에 표시하기 전에는 다음 단추가 꺼져 있다), ui = "features" 는 사용자가 설치할 기능을 고르는 트리를 더한다. 아래 원본은 프로그램을 늘 설치하고, 예제는 보여 주되 고르지 않은 상태로 둔다:

```
[package]
name = "My App"
manufacturer = "My Company"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{E8C1815C-CCD7-4F3F-B914-92A4E3F3A317}"
ui = "features"                # welcome, license, folder, feature tree, ready
license = "LICENSE.txt"       # Next stays off until "I accept" is ticked

[define]
VERSION = "1.0.0"

[feature.Main]
title = "My App"
description = "The program itself."
required = true                # always installed: the tree does not offer to leave it out

[feature.Samples]
title = "Samples"
description = "Example documents to try the program with."
level = 2                      # offered in the tree, not selected by default

[dir.INSTALLDIR]
path = "ProgramFiles/My App"
feature = "Main"

[dir.SamplesDir]
path = "INSTALLDIR/samples"
feature = "Samples"

[file.App]
dir = "INSTALLDIR"
source = "dist/app.exe"

[files.Rest]
dir = "INSTALLDIR"
glob = "dist/files/**"

[files.Samples]
dir = "SamplesDir"
glob = "dist/samples/**"

[shortcut.StartMenu]
dir = "Programs"
name = "My App"
target = "file:App"
```

약관 글은 LICENSE.txt(.txt, .md, .rtf)에, 예제 파일은 dist/samples/ 에 둔다. level = 2 인 기능은 사용자가 표시하지 않으면 설치되지 않고, required 는 트리가 그 기능을 빼자고 제안하지 못하게 한다. 나중에 사용하는 "설치된 앱"의 "변경"에서 고른 것을 바꿀 수 있고, 명령줄에서도 대화창 없이 같은 일을 한다: `msiexec /i app.msi /qn ADDLOCAL=Samples` 는 예제를 더하고, `REMOVE=Samples` 는 뺀다(required 는 트리에만 적용되고 명령줄은 묶지 않는다). 나머지는 기능(feature)과 조건: when에 있다.

Windows 에서 시험하기

```
:: 대화창과 함께 설치
msiexec /i app-1.0.0.msi
:: 조용히 업그레이드(관리자로)
msiexec /i app-1.0.1.msi /qn
:: 제거
msiexec /x app-1.0.1.msi
:: 문제가 있을 때 전체 기록
msiexec /i app-1.0.0.msi /l*v install.log
```

명령줄로 설치하기에 대화창 없이 기능, 폴더, 언어를 고르는 속성이 모여 있다.

무언가 잘못됐을 때

`rubrapack lint app.toml` 은 빌드가 하는 검사를 모두 하고 아무것도 쓰지 않는다. 모든 문제는 자리와 코드를, 흔히 고칠 방법까지 알려 준다:

```
app.toml:18:1: error[RP1201]: unknown key 'glb' in [files.Rest] (did you mean 'glob'?)
app.toml:25:1: error[RP1301]: ID 'App' is already used (line 17); IDs must differ across all tables
```

패키지 안을 보려면: `rubrapack inspect app.msi` File 은 표를 글로 보이고, `rubrapack extract app.msi -d out` 은 설치되는 모습대로 파일을 풀고, `rubrapack lint app-1.0.1.msi --previous app-1.0.0.msi` 는 새 판이 옛 판을 깨끗하게 업그레이드하는지 본다. 종료 코드는 정해져 있다: 0 성공, 1 원본 오류, 2 사용법, 3 입출력, 4 서명, 5 lint, 6 네트워크.

자동화하기

빌드는 원본 밖에 상태가 없는 명령 하나라서 어떤 스크립트나 CI 작업에도 맞는다. `--reproducible` 을 주면 같은 원본은 어느 컴퓨터에서나 같은 바이트가 된다. Linux 러너에서:

```
V=0.7.0 # 쓸 릴리스
curl -sLo rubrapack "https://github.com/rubidus-api/rubrapack/releases/download/v$V/rubrapack-$V-linux-x86_64"
chmod +x rubrapack
./rubrapack lint app.toml
./rubrapack build app.toml -o "app-$VERSION.msi" -D VERSION="$VERSION" --reproducible \
--key signer.pfx --pass-env SIGN_PASS --timestamp http://timestamp.digicert.com
```

서명은 선택이다. 서명 없는 패키지라면 `--key` 부터 뒤를 뺀다. 토큰이나 Windows 인증서 저장소의 키는 명령줄(`sign`, `--pkcs11`, `--key-store`)을 본다.

AI 도우미와 함께

원본은 짧은 평문이고 모든 오류가 어디서 왜인지 말하므로, AI 코딩 도우미가 일 전체를 할 수 있다: 이 매뉴얼 (또는 책, <https://rubidus-api.github.io/rubrapack/>)을 주고 무엇을 설치할지 말한다. 도우미는 `app.toml` 을 쓰고, 깨끗해질 때까지 `rubrapack lint app.toml` 을 돌리고, 빌드하고, `inspect` 와 `extract` 로 결과를 확인하고, 그 과정을 릴리스 스크립트나 위와 같은 CI 작업으로 만들 수 있다. 두 가지는 사람의 몫으로 남는다: 첫 판의 업그레이드 코드를 간직하는 것, 그리고 내보내기 전에 한 번은 Windows 에 패키지를 설치해 보는 것.

10.2 예

```
[define]
VERSION = "1.4.0"
```

```
[package]
name = "Example App"
manufacturer = "Example"
version = "${VERSION}"
arch = "x64" # x64, arm64, x86 가운데 하나 - 기본값 없음
upgrade-code = "{0B9A6C1E-3D2F-4A5B-8C7D-6E5F4A3B2C1D}" # 한 번 만들어 영원히 쓴다
language = "en-US" # 또는 "ko-KR"

[dir.INSTALLDIR]
path = "ProgramFiles/Example App"

[dir.Docs]
path = "INSTALLDIR/docs"

[file.MainExe]
dir = "INSTALLDIR"
source = "dist/app.txt"

[file.Guide]
dir = "Docs"
source = "dist/guide.txt"
name = "User guide.txt"
```

```
rubrapack build example.toml -o example.msi -D VERSION=1.4.1
rubrapack inspect example.msi File
```

10.3 TOML 부분집합

받는 것: [kind]·[kind.ID] 표, 맨 키(A-Z a-z 0-9 _ -), TOML 이스케이프가 되는 기본 문자열 "...", 리터럴 문자열 '...' (이스케이프 없음 - 백슬래시와 따옴표를 쓸 때 편하다: 'SOFTWARE\Example'), 십진수와 0x 정수, true/false, 한 가지 형만 담은 배열, # 주석. BOM 이 있거나 없는 UTF-8, 또는 BOM 이 있는 UTF-16LE. 줄 끝은 LF 나 CRLF.

오류로 거부하는 것: 여러 줄 문자열, 인라인 표, 표 배열, 점으로 이은 키와 따옴표 키, 세 부분 이상의 표 이름, 부동소수, 날짜, 숫자 속 _, 8진·2진 수, 빈 배열이나 섞인 배열, 첫 표보다 앞의 키. 표와 키의 순서는 아무 의미가 없다.

10.4 표

표	키(굵은 것은 반드시)
[package]	name , manufacturer , version (a.b.c 또는 a.b.c.d), arch , upgrade-code , upgrade-code-x64 / -arm64 / -x86, product-code, summary-name(ASCII), language, scope(machine, user, dual), ui(none, basic, minimal, installdir, features), license(.txt, .md, .rtf), reboot(suppress/allow), downgrade-message, compress(none, mszip, mszip:0..mszip:9; 기본 mszip:6), cab(embed 또는 external), cab-max-size(MiB), refuse-upgrade-below, refuse-upgrade-message
[define]	변수: NAME = "value"
[feature.ID]	title , description, level(1-32767), hidden, parent, required, follow-parent, when
[dir.ID]	path = 기준/상대/경로, feature, guard(true: 설치 폴더 지키기 참고)
[file.ID]	dir , source , name, vital(기본 true), any-arch, feature, component-guid, keep, when
[files.ID]	dir , glob , vital, any-arch, feature, keep, when
[folder.ID]	dir , name , keep, feature
[arp]	no-modify, no-repair, help(URL), about(URL), icon(.ico) - "설치된 앱"에 제품이 어떻게 보이는가

표	키(굵은 것은 반드시)
[property.ID]	value , secure, hidden - 대문자 이름의 공개 속성
[action.ID]	run (이 패키지의 .exe 를 가리키는 file:ID), do , undo , check
[registry.ID]	root (HKLM, HKCU, HKCR, HKMU), key , name, value, type, remove, keep, view, with, feature, when
[remove.ID]	dir , name(* 와 ?; 없으면 폴더 자체), on (install, uninstall, both), feature
[ini.ID]	dir , file , section , key , value, mode(set, add, remove), feature, when
[require.ID]	condition , message
[search.ID]	property (또는 dir ID), kind (registry: root, key, name, view; file: path, file, min-version; dir: path; component: component-guid)
[service.ID]	file (.exe 를 가리키는 file:ID), name , display-name, description, start(auto, demand, disabled), account(LocalSystem, LocalService, NetworkService), args, start-on-install
[assoc.ID]	extension (.ext, 소문자), prog-id , target (.exe 를 가리키는 file:ID), description, icon(file: ID), args(기본 "%1")
[protocol.ID]	name (스킴, 소문자), target (.exe 를 가리키는 file:ID), description, args(기본 "%1")
[font.ID]	file (path = "Fonts" 인 dir 에 드는 파일의 file:ID), title
[permission.ID]	target (dir:ID, file:ID, registry:ID), sddl
[env.ID]	name , value , mode(set, append, prepend), keep, feature, when
[copy.ID]	source (file:ID), dir , name(기본: 원본 파일 이름)
[ui]	install-dir(dir ID; 기본 INSTALLDIR), banner(.bmp), launch(file:ID), launch-args, launch-checked, languages(영어에 덧붙일 언어, 예 ["ko"]), license-xx, name-xx, font-xx, langid-xx - 여러 언어 참고
[ui-text.ID]	text 또는 text-xx - 내장 대화창 문구 하나를 바꾼다
[dialog.ID]	after (내장 페이지 또는 다른 [dialog.*]), title, description, title-xx, description-xx
[dialog-control.ID]	dialog , type (text, checkbox, edit, radio, combo), x, y, width , height , text, property, values, labels, text-xx, labels-xx
[shortcut.ID]	dir (dir ID, 또는 Programs, Desktop, StartMenu, Startup), name , target (file:ID), args, description, working-dir(dir ID), icon(.ico), when
[msix]	identity-name , publisher , publisher-display-name, min-version - MSIX 패키지 참고
[msix-app.ID]	executable ([file.*] ID), display-name, description, logo-150, logo-44, store-logo
[msix-extension.ID]	kind (alias: alias ; startup-task: task-id, display-name, enabled), app([msix-app.*] ID; 기본은 첫째) - MSIX 전용

dir 경로의 기준은 다른 dir ID 이거나 다음 가운데 하나다: ProgramFiles(x64/arm64 는 64비트, x86 은 32비트), ProgramFiles32, CommonFiles, AppData, LocalAppData, CommonAppData, StartMenu, Programs, Desktop, Startup, Windows, System, Fonts, Temp. 알려진 폴더 하나만 쓴 경로(path = "Fonts")는 그 폴더 바로 안에 드는 파일에 쓴다.

ID 는 [A-Za-z_][A-Za-z0-9_]* 콜이고(최대 72자, feature 는 38자) dir·파일·feature 를 통틀어 서로 달라야 한다.

MSIX 패키지

출력 이름이 .msix 로 끝나면 같은 원본이 MSIX 패키지가 된다: 완전 신뢰로 도는 데스크톱 앱 하나, 아키텍처 하나. MSIX 에만 필요한 것은 표 두 개가 더 말한다:

```
[msix]
identity-name = "Example.App"           # 3~50자: A-Z a-z 0-9 . -
publisher = "C=KR, O=Example, CN=Example" # 서명 인증서의 주체, 마지막 부분부터
publisher-display-name = "Example"       # 기본: [package] manufacturer
```

```

min-version = "10.0.17763.0"          # 설치되는 가장 오래된 Windows(이것이 기본값)

[msix-app.Main]
executable = "MainExe"                # 앱을 시작하는 [file.*]
display-name = "Example App"          # 기본: [package] name
description = "An example"            # 기본: 표시 이름
logo-150 = "assets/Square150x150.png" # PNG, 150x150
logo-44 = "assets/Square44x44.png"    # PNG, 44x44
store-logo = "assets/StoreLogo.png"   # PNG, 50x50

```

- 판은 [package] version 을 네 부분으로 늘린 것(1.2.3 은 1.2.3.0), 아키텍처는 [package] arch, 언어는 [package] language 다.
- 패키지 자신의 폴더는 첫 [msix-app.*] 실행 파일이 드는 폴더 - ProgramFiles/Example App 처럼 알려진 자리에 붙은 dir - 다. 다른 알려진 자리의 파일은 패키지의 가상 파일 시스템(VFS)에 들어가고, 앱은 그것을 늘 보던 자리에서 본다: Program Files(VFS\ProgramFilesX64, x86 패키지와 ProgramFiles32 는 X86), CommonFiles, System, Windows, CommonAppData(ProgramData). 사용자의 AppData·LocalAppData 와 Temp 에는 가상 폴더가 없어 거기 드는 파일은 오류다(RP1609). Fonts 의 글꼴은 그 [font.*] 로 들어가고, 시작 메뉴·Programs·바탕 화면 폴더는 바로가기([shortcut.*])로, Startup 은 시작 작업([msix-extension.*])으로 쓴다.
- [msix-app.*] 표가 여럿이면 한 패키지에 항목이 여럿 생긴다(최대 100개). 패키지 로고는 첫째 것이다.
- [registry.*] 값은 패키지의 가상 레지스트리에 들어간다. 앱은 그것을 실제 레지스트리와 합쳐 보고, 컴퓨터의 레지스트리는 건드리지 않는다: Software 아래의 HKLM(과 HKMU)은 Registry.dat (view = "32" 면 32비트 보기)에, Software 아래의 HKCU 는 User.dat 에 든다. MSIX 가 담을 수 없는 것은 오류다(RP1612): HKCR(파일 형식과 프로토콜은 [assoc.*]·[protocol.*] 로 쓴다), Software 밖의 키, remove 와 keep, 그리고 Windows Installer 가 설치 때 채우는 부분이 든 값 ([INSTALLDIR], [#File] 등. 이스케이프 [\] 와 [\] 는 괜찮다).
- 로고는 셋 다 주거나 하나도 주지 않는다. 주지 않으면 한 가지 색의 로고가 들어간다. 로고는 크기가 정확해야 한다(RP1608).
- MSIX 가 할 수 없는 것은 조용히 빠지 않고 오류로 알린다(RP1605): 사용자 지정 동작, 서비스, 환경 변수, INI 파일, 권한, 설치 조건과 검색, 설치 때 지우거나 복사하는 파일, 빈 폴더. 그런 표(또는 [file.*]/[files.*])에 msi-only = true 를 달면 MSI 에는 들어가고 MSIX 는 그것 없이 만들어진다. feature·속성·대화창[arp] 은 Windows Installer 에만 해당하므로 MSIX 에는 쓰지 않는다.
- --unsigned-test 는 서명 없는 패키지를 시험 삼아 설치할 때 Windows 가 요구하는 속성을 더한다 (Add-AppxPackage -AllowUnsigned, 프로그램이 들어 있으면 관리자). 그런 패키지는 배포용이 아니고 정체(identity)도 서명판과 다르다. 대신 --key 를 주면 패키지(또는 묶음과 그 안의 패키지)에 서명한다. 아래 sign 참고.
- 파일은 압축한다(--msix-compress store 로 끈다). 그림, 압축 파일처럼 이미 압축된 파일은 저장만 한다. MSIX 에는 시작이 들어가지 않아, 같은 원본은 Linux 와 Windows 에서 같은 바이트가 된다.
- 출력 이름이 .msixbundle 로 끝나면 묶음이다: --arch 의 아키텍처마다 원본을 한 번씩 짓고 (목록, --arch x64,x86,arm64; 없으면 원본의 아키텍처), 각 패키지는 안에서 <identity-name>_<version>_<arch>.msix 라는 이름을 갖는다. Windows 는 그 가운데 자기 아키텍처의 패키지를 설치한다(x64 컴퓨터는 x86 보다 x64 를 고른다). \$(ARCH) 로 빌드마다 제 프로그램을 준다: source = "bin/\$(ARCH)/app.exe". 묶음의 판은 패키지들의 판이다.

설치한 프로그램으로 등록하기: [action.ID]

```

[action.Tip]
run = "file:MainExe"      # 이 패키지가 설치하는 .exe
do = "--register"          # 파일을 설치한 뒤, 그리고 복구할 때 다시 실행
undo = "--unregister"      # 제거할 때, 파일을 지우기 전에 실행

```

rubrapack 은 이 한 쌍을 관리자 권한의 지연(deferred) 동작과 그 되돌림(rollback) 짝으로 바꾼다. 그래서 제거와 업그레이드는 전부 되거나 전혀 되지 않는다: 뒤에서 무엇이 실패하면(또는 do/undo 자체가 0 아닌 값으로 끝나면) 파일이 되돌아오고 다른 명령이 등록을 되살린다. 두 명령은 두 번 실행해도 안전해야 하고, 아무것

도 묻지 않고 끝나야 한다 - 창 없이 실행되고 사용자를 기다리는 것은 없다. check 는 등록이 되어 있으면 0 으로 끝나는 명령을 (필요하면) 적는다.

레지스트리 값: [registry.ID]

```
[registry.InstallDir]
root = "HKLM"
key = 'SOFTWARE\Example'      # 리터럴 문자열은 백슬래시를 그대로 둔다
name = "InstallDir"          # 없으면 키의 기본값
value = "[INSTALLDIR]"        # MSI 서식 문자열: [PROPERTY], [#FileID], "[" 는 [\[
```

type 은 string(기본), expand, dword(0~0xFFFFFFFF 정수), qword(정수, 또는 "0x" 와 16자 이하의 16진수), binary(16진수), multi(문자열 배열)이다. Windows Installer 는 REG_QWORD 를 스스로 쓰지 못한다. 그래서 rubrapack 이 작은 도우미 DLL 을 패키지에 넣어 쓰게 하고, 설치가 실패하면 그 DLL 이 이전 값을 되살린다. 값 하나가 구성 요소(component) 하나이고 제거할 때 지워진다(keep = true 면 남는다). with = "file:ID" 는 대신 그 파일의 구성 요소에 넣는다. 64비트 패키지에서 값은 64비트 레지스트리 보기로 가고, view = "32" 는 32비트 보기에 쓴다. remove = true(value 없이)는 설치 중에 이름 붙은 값을 - name 이 없으면 키 전체를 - 지운다.

바로가기: [shortcut.ID]

```
[dir.Menu]
path = "Programs/Example"      # 시작 메뉴 > Example

[shortcut.Settings]
dir = "Menu"
name = "Example settings"      # ".lnk" 는 붙여 준다
target = "file:MainExe"
args = "--settings \"[INSTALLDIR]\""
```

바로가기는 대상 파일(과 그 feature)에 딸린다. 바로가기 때문에 만든 폴더는 제거할 때 지운다. 컴퓨터 전체 패키지에서 Programs 와 Desktop 은 모든 사용자의 시작 메뉴와 공용 바탕화면이다.

MSIX 에서 바로가기는 앱을 시작한다(대상은 [msix-app.*] 의 실행 파일이어야 한다): Programs 나 StartMenu 의 것은 그 앱 자신의 시작 메뉴 항목이고(args 없음), Desktop 의 것은 매니페스트에 적히며 min-version = "10.0.19645.0" 이상이 필요하다(RP1614). 다른 폴더, working-dir, args 속 [...] 는 거기서 오류다(RP1613). Startup 은 시작 작업으로 쓴다.

파일 형식과 링크: [assoc.ID], [protocol.ID]

```
[assoc.Doc]
extension = ".exdoc"
prog-id = "Example.Document"   # 같은 prog-id 의 표들은 한 종류의 문서를 뜻한다
description = "Example document"
target = "file:MainExe"
args = "--open \"%1\""         # 기본값은 "%1"

[protocol.Link]
name = "example"               # example:... 가 프로그램을 연다
target = "file:MainExe"
```

MSI 에서는 프로그램의 구성 요소에 든 HKEY_CLASSES_ROOT 아래 레지스트리 값이다: 확장자의 기본값은 prog-id, prog-id 에는 설명과 DefaultIcon(icon, 없으면 프로그램의 첫 아이콘), shell\open\command 가 있고, 스킴에는 URL Protocol 이 붙는다. HKEY_CLASSES_ROOT 는 설치 방식을 따른다: 컴퓨터 전체면 HKLM\Software\Classes, 사용자별이면 HKCU\Software\Classes 에 들고, 제거하면 없어진다. 이 프로그램만 맡겠다는 형식은 곧바로 이 프로그램으로 열리고, 사용자가 다른 프로그램을 골라 두었으면 Windows 는 그 선택을 지킨다.

MSIX 에서는 대상 실행 파일을 가진 앱의 매니페스트에 들어가고(파일 형식 연결, 프로토콜) args 는 평문이어야 한다. icon 은 쓰지 않는다: 앱의 로고가 파일 형식을 나타낸다.

MSIX 전용: [msix-extension.ID]

```
[msix-extension.Cli]
kind = "alias"
alias = "example.exe"           # 콘솔에서 이 이름을 치면 앱이 시작된다

[msix-extension.Boot]
kind = "startup-task"          # 앱을 한 번 실행한 뒤로 Windows 와 함께 시작된다
display-name = "Example"       # 작업 관리자에 보이는 이름; task-id 기본값은 표 ID
enabled = true
```

MSI 빌드는 이것들을 뺀다. MSIX 에서 글꼴([font.*])은 패키지의 Fonts 폴더에서 다른 앱과 나눠 쓰며 (uap4:SharedFonts) title 은 쓰지 않는다.

지우기와 복사: [remove.ID], [copy.ID]

[remove.ID] 는 dir 에서 name 에 맞는 파일을 지운다 - 예를 들어 옛 판이 남긴 *.log 를 on = "install" 로, 프로그램이 실행 중에 만드는 파일을 on = "uninstall" 로. name 이 없으면 비어 있는 폴더 자체를 지운다. 설치가 실패하면 지운 파일은 되돌아온다. [copy.ID] 는 이 패키지의 파일을 다른 폴더에 한 벌 더 설치하고, 원본과 함께 들어오고 함께 나간다. 설치 전부터 있던 폴더는 절대 지우지 않는다.

환경 변수: [env.ID]

시스템(컴퓨터 전체) 변수다. mode = "set"(기본)은 값을 바꾸고, append/prepend 는 기존 값 끝에 ;value 를, 앞에 value; 를 붙인다. 제거는 정확히 그만큼 되돌린다: set 한 변수는 지우고, 붙인 부분은 떼어 내고 나머지는 둔다. keep = true 면 남긴다.

INI 파일: [ini.ID]

mode = "set"(기본)은 파일의 [section] 에 key=value 를 쓰고, add 는 섹션 목록에 값을 덧붙이며 (a 가 a,b 로), remove 는 설치 중에 키를 지운다. 제거하면 set 과 add 가 쓴 것을 뺀다. remove 는 파일 설치보다 먼저 실행되므로 옛 판이 남긴 INI 파일을 위한 것이다.

검색과 요구: [search.ID], [require.ID]

검색은 무엇보다 먼저 실행되고 찾은 것을 공개 속성에 넣는다(못 찾으면 비어 있다): 레지스트리 값의 데이터, 파일의 전체 경로(path = 알려진 폴더와 상대 경로, 예 System 이나 ProgramFiles/Example; 프로그램 파일은 min-version), 폴더, 또는 다른 제품 구성 요소의 키 파일. 요구 사항은 condition 이 거짓이면 그 메시지로 첫 설치를 멈춘다(복구와 제거는 막지 않는다). 조건은 Windows Installer 의 문법을 쓰고 (VersionNT >= 603, FOUND_TOOL, NOT OLDSETTING) 검색 결과를 볼 수 있다.

```
[search.Tool]
property = "FOUND_TOOL"
kind = "file"
path = "System"
file = "tool.exe"

[require.Tool]
condition = "FOUND_TOOL"
message = "[ProductName] needs tool.exe."
```

registry 나 dir 검색은 속성 대신 dir 을 가리킬 수 있다. 그러면 찾은 폴더가 그 dir 의 기본값이 된다. 사용자가 고른 폴더를 패키지가 기억하는 방법이 이것이다: 폴더를 적어 두고, 다음 판에서 찾아본다. 레지스트리 값은 그 폴더가 아직 있을 때만 쓰이고, 명령줄에 준 폴더(msiexec /i app.msi INSTALLDIR=D:\Apps\Example\)가 여전히 이긴다.

```
[registry.RememberDir]
root = "HKLM"
key = "Software\\Example"
name = "InstallDir"
```

```
value = "[INSTALLDIR]"

[search.PreviousDir]
property = "INSTALLDIR"           # dir: 레지스트리에 있는 폴더가 실제로 있으면 그것이 기본값
kind = "registry"
root = "HKLM"
key = "Software\\Example"
name = "InstallDir"
```

서비스, 글꼴, 권한

[service.ID] 는 패키지의 .exe 가 돌리는 서비스를 설치한다: 파일이 바뀌기 전과 제거할 때 멈추고, 제거할 때 지우며, start-on-install = true 면 설치 뒤에 시작한다. [font.ID] 는 패키지가 Fonts 폴더에 설치하는 글꼴 파일을 등록한다. title 이 없으면 Windows 가 TrueType/OpenType 파일에서 이름을 읽는다. [permission.ID] 는 패키지가 만드는 폴더, 그 파일, 또는 그 레지스트리 값에 SDDL 보안 서술자를 건다(이것이 있으면 패키지는 Windows Installer 5.0 을 요구한다).

사용자별 패키지와 겸용 패키지: scope

scope = "machine"(기본)은 모든 사용자에게 설치하고 관리자 권한이 필요하다. scope = "user" 는 권한 상승 없이 현재 사용자에게 설치한다: ProgramFiles 는 %LOCALAPPDATA%\Programs 가 되고, Programs 와 Desktop 은 그 사용자의 것, 레지스트리 값은 HKCU(또는 HKMU), 환경 변수는 사용자의 것이며 do/undo 동작은 그 사용자로 실행된다. 컴퓨터 전체 설치를 요청하면 거부한다. scope = "dual" 은 기본으로 사용자별로 설치하고, 관리자 명령창에서 msixexec /i x.msi ALLUSERS=1 MSIINSTALLPERUSER="" 로 하면 컴퓨터 전체로 설치한다. 그 레지스트리 값은 설치된 방식에 따라 HKLM 이나 HKCU 가 되는 HKMU 를 쓴다. 서비스, 글꼴, 권한과 컴퓨터 폴더(Windows, System, Fonts, CommonAppData)는 scope = "machine" 이 필요하다.

캐비닛

파일은 패키지에 넣는 캐비닛 하나로 압축한다. cab-max-size = N 은 파일이 N MiB 를 넘을 때마다 새 캐비닛을 시작하고, cab = "external" 은 캐비닛을 패키지 옆에 <name>.cab(또는 <name>-1.cab, <name>-2.cab, ...)으로 쓴다. 그 캐비닛은 패키지와 함께 다녀야 하고, cab-max-size 가 없으면 캐비닛마다 2 GiB 전에 나눈다. Windows Installer 는 2 GiB 이상인 패키지를 열지 못하므로, 그만큼 큰 내장 캐비닛은 cab = "external" 을 원하는 오류(RP1516)다. rubrapack 은 있는 캐비닛을 덮어쓰지 않고 패키지를 맨 나중에 쓴다: 패키지는 <out>.rp-map 에서 만들어지고(바이트가 메모리가 아니라 그 파일로 바로 간다) 모든 일이 성공했을 때만 제 이름으로 바뀌므로, 실패한 빌드는 패키지를 남기지 않는다. 압축은 모든 프로세서에서 돈다(덜 쓰려면 --jobs N); 어느 쪽이든 바이트는 같다. 모든 패키지에는 관리 설치(msiexec /a, 압축을 푼 네트워크 이미지)와 광고(msiexec /jm) 순서도 들어 있다.

MSI 서식 문자열로 해석되는 곳은 이것뿐이다: 레지스트리 value(와 multi 항목), 바로가기 args, 환경 변수 value, INI value, 요구 사항 message, 서비스 args. 나머지는 적은 그대로 쓴다.

설치된 앱 항목과 속성

[arp] 는 제품이 "설정 > 설치된 앱"에 어떻게 보이는지 정한다(no-modify, no-repair, help, about). [property.NAME] 은 공개 속성을 더한다. secure = true 는 그 값이 설치의 관리자 권한 부분까지 가게 하고, hidden = true 는 로그에 값이 남지 않게 한다. 설치 엔진이나 rubrapack 이 스스로 쓰는 이름(ARP*, MSI*, RP_*, ALLUSERS, REBOOT 등)은 거부한다.

명령줄로 설치하기

대화창이 고르는 것은 모두 msiexec 에 줄 수 있다.

속성	하는 일
INSTALLDIR=D:\Apps\Example\	설치 폴더(대문자 ID 의 dir 모두)
ADDLOCAL=Core,Extra	이 기능들을 설치(ADDLOCAL=ALL: 모든 기능)
REMOVE=Extra	설치된 제품에서 이 기능들을 제거(REMOVE=ALL: 전부)

속성	하는 일
INSTALLLEVEL=3	level 이 3 이하인 기능을 모두 설치
RPLANGUAGE=ko	대화창 언어([ui] languages 가 있을 때)
ALLUSERS=1 MSIINSTALLPERUSER=""	검용 패키지를 모든 사용자에게(기본: 현재 사용자만)
DESK=1, APP_MODE=server	원본이 정한 속성: when 조건, 대화창 값

창 없이 설치하기

rubrapack 이 만드는 패키지는 모두 명령줄에서 사용자 화면 없이 설치·복구·업그레이드·제거된다:

```
msiexec /i example.msi /qn /l*v install.log
msiexec /x {ProductCode} /qn
```

종료 코드 0 은 성공, 3010 은 다시 시작이 필요한 성공(rubrapack 은 컴퓨터를 스스로 다시 시작하지 않는다), 그 밖의 값은 실패이며 그 뒤 컴퓨터는 설치 전과 같다.

대화창: ui

[package] 의 ui 가 내장 대화창 세트 하나를 고른다. 없으면(none) Windows Installer 자신의 진행 막대만 보인다.

ui	설치할 때	이미 설치되어 있을 때
basic	진행, 끝(또는 오류)	진행, 끝
minimal	환영, 사용권(있으면), 진행, 끝	복구 또는 제거
installdir	환영, 사용권, 설치 폴더(폴더 찾아보기 포함), 준비, 진행, 끝	복구 또는 제거
features	installdir 에 더해 기능 트리과 필요한 디스크 공간	복구 또는 제거

모든 세트에는 취소 확인, 오류 대화창, 사용 중인 파일 목록, 디스크 공간 부족 경고도 있다. 대화창은 모두 기본값이 있는 값만 모으므로 /qn 은 여전히 창 없이 설치한다.

```
[package]
ui = "installdir"
license = "LICENSE.txt"           # 동의할 때까지 설치/다음 단추가 꺼져 있다

[ui]
install-dir = "APPPDIR"           # 사용자가 바꿀 수 있는 dir(기본 INSTALLDIR)
banner = "banner.bmp"            # 위쪽 띠; 약 493 x 58 화소

[ui-text.WelcomeText]
text = "This will install [ProductName]. Close other programs first."
```

.txt 나 .md 사용권은 평문으로 보여 준다(한글, 이모지 같은 어떤 글자도 그대로 둔다). .rtf 사용권은 그대로 쓴다. banner 가 없으면 띠는 흰색이다. [ui-text.ID] 는 문구 하나를 바꾼다. 문구는 MSI 서식 문자 열이라 [ProductName] 은 치환되고 [] 자체는 [\] 로 쓴다. ID 는 다음과 같다: Back, Next, Cancel, Install, Finish, OK, Yes, No, Retry, Ignore, Abort, Exit, Browse, WelcomeTitle, WelcomeText, LicenseTitle, LicenseText, LicenseAccept, DirTitle, DirText, DirLabel, BrowseTitle, BrowseText, BrowseLookIn, BrowseFolder, BrowseUp, BrowseNew, CustomizeTitle, CustomizeText, Reset, DiskCost, DiskCostTitle, DiskCostText, ReadyTitle, ReadyText, ProgressTitle, ProgressText, ProgressStatus, ExitTitle, ExitText, UserExitTitle, UserExitText, FatalTitle, FatalText, CancelText, FilesInUseTitle, FilesInUseText, OutOfDiskTitle, OutOfDiskText, MaintTitle, MaintText, Repair, RepairText, Remove, RemoveText, LanguageTitle, LanguageText, DirGuardText(아래 가드의 메시지). 단추 문구에서 & 는 바로 가기 키를 표시한다(&Next 는 Alt+N).

여러 언어

대화창은 영어다. [ui] languages 는 같은 패키지에 다른 언어를 덧붙인다. 그러면 첫 페이지가 언어를 묻고, 그 뒤의 모든 페이지 - 환영, 사용권, 폴더, 사용자 페이지, 준비, 진행, 완료, 그리고 취소·오류·사용 중 파일·디스크

공간·유지보수 페이지 - 가 고른 언어로 나온다. 미리 골라 두는 언어는 사용자의 지역 형식 (UserLanguageID), 그다음 시스템 로캘(SystemLanguageID)이 LANGID 에 드는 첫 덧붙인 언어이고, 없으면 영어다. 명령줄의 RPLANGUAGE=ko 는 바로 고른다. 무인 설치(/qn)는 아무것도 보여 주지 않으니 고를 것도 없다. 영어만 있으면 언어 페이지도 없다.

```
[ui]
languages = ["ko"]           # 영어는 늘 있고 기본이다
license-ko = "LICENSE-ko.txt" # 언어별 사용권(기본: [package] license)

[ui-text.WelcomeText]
text-ko = "[ProductName]을(를) 설치합니다." # text = 모든 언어, text-xx = 한 언어

[dialog.Options]
after = "RpInstallDirDlg"
title = "Options"
title-ko = "선택 사항"

[dialog-control.Mode]
# ...
labels = ["&Typical", "&Portable"]
labels-ko = ["표준(&T)", "휴대용(&P)"]
```

- 내장 문구는 영어와 한국어(ko)로 있다. 다른 언어(ja, de, ...)는 내장 문구를 모두 text-xx 로 주고 (빠진 것은 lint 가 이름을 댄다), name-xx(언어 페이지에 보일 이름), font-xx(그 언어 대화창의 글꼴), langid-xx(그 언어를 고르게 하는 LANGID: 숫자 하나나 배열)를 줄 수 있다. 흔한 언어는 이것들이 내장되어 있다: ja, zh, de, fr, es, it, pt, nl, pl, ru, uk, tr, vi, th.
- 한국어 대화창은 맑은 고딕, 영어는 Segoe UI 를 쓴다.
- [ProductName], [Manufacturer], [ProductVersion] 밖의 속성을 쓰는 문구는 언어를 고를 때 서식이 폴리므로 255자 안이어야 한다.
- Windows Installer 자신의 문구 - 기능 트리 메뉴, 크기, 남은 시간, 오류 메시지 - 는 패키지 것이 아니다: 앞의 것들은 영어로 남고, 오류 메시지는 Windows 를 따른다.
- [package] language 는 패키지의 언어(ProductLanguage, 요약 정보)만 정한다. 0.2 부터는 대화창을 한국어로 만들지 않는다: languages = ["ko"] 를 넣는다(lint 가 RP1317 로 경고한다).

아이콘, 완료 페이지, 설치 범위 페이지

- [arp] icon = "app.ico" 는 "설치된 앱"에 보일 제품 아이콘이다. [shortcut] icon 은 바로가기에 따로 .ico 를 준다(없으면 프로그램 자신의 아이콘). 둘 다 빌드할 때 읽어 패키지 안에 넣는다.
- [ui] launch = "file:App" 은 완료 페이지에 "[ProductName] 실행"을 둔다(launch-checked = false 가 아니면 체크된 채로; launch-args 는 그 인자). "마침"을 누르면 첫 설치나 업그레이드 뒤에 설치를 실행한 사용자의 권한으로(설치 엔진의 권한이 아니라) 프로그램을 띄운다. 복구·제거 뒤에는 띄우지 않고, /qn 에서는 결코 띄우지 않는다.
- 대화창이 있는 scope = "dual" 패키지는 사용권 다음에 "나만"(기본) / "이 컴퓨터의 모든 사용자"(관리자 권한 필요) 페이지를 둔다. 설치 폴더도 그에 따라 %LOCALAPPDATA%\Programs 나 Program Files 로 옮긴다.

나만의 대화창 페이지: [dialog.ID], [dialog-control.ID]

minimal, installdir, features 에서는 내장 흐름에 페이지를 더할 수 있다. 더한 페이지도 다른 페이지와 같은 띄와 뒤로/다음/취소 단추를 갖고, 본문에 컨트롤을 놓는다.

```
[dialog.Options]
title = "Options"           # 띠의 제목; 없으면 [ProductName]
description = "Choose how to install."
after = "RpInstallDirDlg"   # 이 페이지 바로 다음에 나온다

[dialog-control.Modelabel]
dialog = "Options"
```

```

type = "text"
x = 20
y = 55
width = 330
height = 12
text = "Installation &mode:"

[dialog-control.Mode]
dialog = "Options"
type = "radio"
x = 20
y = 70
width = 200
height = 42          # 값 하나에 12 이상
property = "APP_MODE"
values = ["typical", "portable", "server"]
labels = ["&Typical", "&Portable", "&Server"]

[property.APP_MODE]
value = "typical"      # 기본값, 창 없는 설치에서도 쓰인다

```

- after 는 고른 세트의 페이지 - RpWelcomeDlg, RpLicenseDlg(사용권이 있을 때), RpInstallDirDlg(installdir, features), RpCustomizeDlg(features) - 또는 직접 만든 다른 페이지다. 같은 페이지 뒤에 여러 페이지가 오면 ID 순서를 따른다. minimal 에서는 마지막 페이지의 단추가 설치이고, 다른 세트에서는 준비 페이지가 늘 마지막이다.
- 좌표는 370 x 270 페이지 위의 대화창 단위다. 컨트롤은 본문, y = 45 에서 y = 234 사이에 놓는다. Tab 키는 컨트롤을 위에서 아래로, 그다음 왼쪽에서 오른쪽으로 옮겨 다닌다.
- text 는 글씨를 보인다(& 는 바로 가기 키를 표시하고, 글씨 컨트롤에서는 다음 컨트롤로 옮긴다). checkbox 는 체크하면 속성을 1 로, 풀면 속성을 없앤다. edit 는 사용자가 속성 값을 입력하게 한다. radio 와 combo 는 values(64바이트 이하 문자열 1~32개) 가운데 하나를 고르게 하고, 적은 순서대로 labels(기본: 값 자체)로 보인다.
- radio 와 combo 에는 그 값 가운데 하나를 값으로 가진 [property.*] 가 있어야 한다: 대화창은 값을 모을 뿐이고 창 없는 설치(/qn)는 기본값을 쓴다. 어떤 속성이든 명령줄에서 줄 수도 있다: msixexec /i example.msi /qn APP_MODE=server.
- 속성은 직접 정한 공개 이름(대문자)이고 컨트롤 하나에 하나이며, 설치의 관리자 권한 부분까지 가므로 [registry.*], [ini.*], [env.*], 조건 등에서 [APP_MODE] 로 쓸 수 있다.
- Rp 로 시작하는 페이지 ID 와 틀의 컨트롤 ID(Banner, Title, Description, BannerLine, BottomLine, Back, Next, Cancel)는 예약되어 있다. 페이지 하나에는 컨트롤이 64개까지 든다.

아키텍처와 업그레이드 계열

arch 는 원본 자신의 아키텍처, upgrade-code 는 그 업그레이드 계열이다. 같은 원본을 다른 아키텍처로 지으려면 (--arch) 그 아키텍처에 upgrade-code-x86, upgrade-code-arm64, upgrade-code-x64 로 제 계열을 주어야 하고, 없으면 빌드를 거부한다. Windows Installer 의 업그레이드 감지는 아키텍처를 가리지 못해서, 코드를 같이 쓰면 한 아키텍처를 설치할 때 다른 아키텍처가 지워진다. MSIX 에는 업그레이드 코드가 없으므로 .msix.msixbundle 빌드에는 필요 없다.

먼저 지워야 하는 판

refuse-upgrade-below = "1.0.0" 은 설치된 1.0.0 미만 판의 업그레이드를 거부하고 먼저 지우라고 알린다. 메시지(refuse-upgrade-message, 또는 기본 문구)는 늘 그 일을 하는 명령으로 끝난다: msixexec /x {ProductCode} /qn MSIRESTARTMANAGERCONTROL=Disable. 옛 판을 다른 도구가 MSIRESTARTMANAGERCONTROL=Disable 없이 만들었을 때 쓴다: 그런 판을 업그레이드 도중에 지우면 그 파일을 불러 쓰는 프로그램을 모두 닫으려 한다(formats/msi-package.md 의 "사용 중인 파일" 참고).

와일드카드: [files.ID]

glob 은 *(한 폴더 안의 아무 글자들), ?(한 글자), **(폴더 여러 단계)가 든 원본 경로다. 맞는 것은 이름순으로 정렬하므로 결과가 파일 시스템에 따라 달라지지 않는다. 첫 와일드카드 아래의 폴더는 dir 아래에 다시 만든다: glob = "dist/layouts/**/*.jmt" 는 dist/layouts/de/x.jmt 를 <dir>/de/x.jmt 로 설치한다. 맞는 것이 없거나, 심볼릭 링크이거나, 출력 파일 자신이 맞으면 오류다.

빈 폴더: [folder.ID]

파일이 하나도 들지 않아도 dir 안에 name 폴더를 만든다. keep = true 면 제거한 뒤에도 폴더가 남는다(프로그램이 쓰는 데이터를 위해).

기능(feature)

[feature.*] 표가 없으면 모든 것이 숨은 기능 하나에 든다. 기능을 하나라도 선언하면 모든 파일에 기능이 있어야 한다: 파일 자신의 feature 키, 또는 그 dir 의 feature. level 이 1 보다 큰 기능은 기본으로 설치하지 않는다. ui = "features" 면 사용자가 트리에서 기능을 고르고, 나중에 "설치된 앱"에서 바꿀 수 있다(유지보수 페이지의 "변경").

- required = true: 트리에서 "설치하지 않음"을 고를 수 없다.
- follow-parent = true: 부모가 설치되는 대로 따라간다.
- when = "<조건>": 조건이 참이 아니면 그 기능은 꺼진다 - 설치하지도 보여 주지도 않는다(아래).

조건: when

when 은 Windows Installer 조건(VersionNT64, DESK = "1", NOT OLDVERSION)을 받는다. 기능, 파일과 파일 묶음, 레지스트리 값(파일과 with 로 묶인 값은 안 된다: 파일 쪽에 둔다), 바로가기, 환경 변수, INI 값에 쓸 수 있다. 조건이 참일 때만 그것을 설치한다. 나만의 대화창 페이지에서 정한 속성, 명령줄에 준 속성, 검색 결과를 볼 수 있다. 조건은 그것을 처음 설치할 때(메이저 업그레이드 포함) 따진다 - 복구는 있는 그대로 둔다. MSIX 는 모두 설치하므로 when 을 거부한다.

```
[dialog-control.Desk]           # 나만의 페이지의 체크박스
dialog = "Options"
type = "checkbox"
x = 20
y = 60
width = 300
height = 16
text = "바탕화면 바로 가기 만들기(&D)"
property = "DESK"

[shortcut.Desk]
dir = "Desktop"
name = "Example"
target = "file:App"
when = "DESK"
```

제거할 때 파일 남기기: keep

파일(또는 파일 묶음)에 keep = true 를 두면 제품을 제거해도 그 파일은 남는다 - 사용자가 바꿨을 수 있는 설정 파일용이다. 한 번 바뀐 파일은 복구나 다음 판이 덮어쓰지 않는다(판 없는 파일에 대한 Windows Installer 의 규칙). MSIX 는 파일을 모두 지우므로 keep 을 거부한다.

변수

문자열 값 속 \$(NAME) 은 한 번 치환된다. 명령줄의 -D NAME=value 가 먼저, 그다음 [define] 이다. 치환 결과는 다시 읽지 않는다(\$(X) 가 든 값은 그대로 남는다). \$\$ 는 \$ 한 글자다. 정의되지 않은 이름은 오류다. \$(ARCH) 는 -D 나 [define] 이 정의하지 않는 한 내장이다: 짓고 있는 아키텍처(x64, arm64, x86) - 그래서 원본 하나가 source = "bin/\$(ARCH)/app.exe" 처럼 아키텍처마다 제 프로그램을 가리킬 수 있다.

프로그램 파일

PE 파일(.exe, .dll 등)은 검사한다: 머신 형식이 arch 와 맞아야 하고(x64 패키지의 x86 도우미는 any-arch = true 가 필요하다), 판 정보 리소스가 패키지 안 파일의 판이 된다. Windows Installer 는 그것을 보고 설치된 파일을 바꿀지 정한다.

설치 폴더 지키기

다른 프로그램이 그 파일을 불러 쓰는 프로그램 - 입력기, 셀 확장, 서비스 - 은 남이 미리 만들어 둔 폴더에 설치되면 안 된다: 거기 있던 파일은 그대로 남고, 프로그램 옆에 심어 둔 DLL 이 그 프로그램의 권한으로 실리기 때문이다. dir 에 guard = true 를 두면, 그 폴더가 다음과 같을 때 첫 설치가 파일을 하나도 놓기 전에 멈춘다.

- 이미 있고 주인이 SYSTEM, Administrators, TrustedInstaller 가 아니다.
- 연결 지점(junction)이나 다른 리파스 포인트를 거쳐 닿는다(폴더 자신이나 그 위의 어느 폴더든).

아직 없는 폴더는 통과한다(설치 엔진이 만들고, [permission.*] 로 잠글 수 있다). 복구, 제자리 업그레이드, 제거는 검사하지 않는다. 메이저 업그레이드는 새 판의 첫 설치라 똑같이 검사한다(이전 판이 만든 폴더는 통과한다). 설치는 DirGuardText 메시지([1] 은 그 폴더; 대화창에서 고른 언어) 뒤에 1603 으로 끝나고, /qn 에서도 같으며, 로그에 찾은 주인이 적힌다. 검사는 rubrapack 의 도우미 DLL 이 하고, 패키지가 그 DLL 을 실행한다(REG_QWORD 값과 같다). 보는 것은 주인이다: 관리자 소유라도 권한이 누구나 쓸 수 있게 되어 있는 폴더는 거부하지 않는다([permission.*] 로 잠근다).

```
[dir.INSTALLDIR]
path = "ProgramFiles/Example"
guard = true
```

경로

원본 경로는 원본 파일 기준의 상대 경로이고 / 를 쓴다. 절대 경로, \, 심볼릭 링크, 없는 파일은 오류다. 설치될 이름에는 Windows 가 금하는 것(< > : " / \ | ? *, 제어 문자, 끝의 점이나 공백, con 같은 장치 이름)만 빼고 어떤 유니코드 글자든 쓸 수 있고, 한 폴더 안의 두 이름이 대소문자만 달라서는 안 된다. Windows Installer 는 이름을 8.3 짧은 이름과 함께 UTF-16 255단위에 저장하므로, 긴 이름은 짧은 이름이 남긴 만큼만 쓸 수 있다: 세 글자 확장자의 name.ext 는 242단위, 확장자 없는 이름은 246단위(NTFS 는 255 를 받는다). 더 긴 이름은 RP1514 로 거부한다.

10.5 명령줄

```
rubrapack build <src.toml> -o <out.msi|out.msix|out.msixbundle> [-D NAME=VALUE]...
    [--arch x64|arm64|x86 | --arch <목록> (.msixbundle)]
    [--compress none|mszip|mszip:N] [--jobs N] [--nfc] [--reproducible]
    [<키>] [--cert <chain.pem>]
    [--timestamp <URL>] [--tsa-trust <인증서>] [--tls-trust <인증서>] [--system-roots]
    [--proxy <URL>]] [--allow-unsigned-cabs]]
    [--unsigned-test] [--msix-compress deflate|store]      (.msix, .msixbundle)
rubrapack inspect <file.msi> [table | --summary | --files | --streams]
rubrapack inspect <file.msix|file.msixbundle> [--files | --manifest]
rubrapack inspect <file.cab>
rubrapack new [msi] <name>
rubrapack new [<file>.toml] [-i]
rubrapack new <name> [--dist <folder>] [--name ...] [--ui ...] [--optional ...] ...
rubrapack edit <file>.toml [--set <table>.<key>=<value>]... [--unset <table>.<key>]... [--sync]
rubrapack guid [--from <text>]
rubrapack lint <src.toml> [-D NAME=VALUE]... [--arch x64|arm64|x86] [--target msi|msix] [--nfc] [--strict]
rubrapack lint <file.msi> [--previous <old.msi>] [--strict]
rubrapack lint <file.msix|file.msixbundle> [--strict]
rubrapack extract <file.msi|file.msix|file.msixbundle|file.cab> -d <새 폴더> [--limit-entries N] [--limit-bytes N]
rubrapack sign <file.exe|.dll|.msi|.msix|.msixbundle> <키> [--cert <chain.pem>]
    [--timestamp <URL>] [--tsa-trust <인증서>] [--tls-trust <인증서>] [--system-roots]
    [--proxy <URL>]] [--allow-unsigned-cabs] [-o <out>]
```

```
rubrapack keys list [--pkcs11 <모듈> [--token-label <이름>] [--pin-env VAR | --pin-file FILE]]
rubrapack verify <file.exe|.dll|.msi|.msix|.msixbundle> [--trust <인증서>]... [--system-roots] [--tsa-trust <인증서>]...
rubrapack version | help [command]
```

<키> 는 다음 가운데 하나:

--key <key.pfx .pem> [--pass-env VAR --pass-file FILE]	키 파일
--pkcs11 <모듈> --key-label <이름> [--token-label <이름>] [--pin-env VAR --pin-file FILE]	PKCS#11 토큰 안의 키
--key-store <SHA-1 지문> [--machine-store]	Windows 저장소의 키

- 명령줄 옵션은 원본보다 앞선다.
- --reproducible 은 패키지 코드를 내용에서 끌어낸다: 같은 원본은 Linux 에서든 Windows 에서든, 어느 폴더에서든, 옵션 순서가 어떻게 같은 바이트가 된다. 없으면 패키지 코드는 무작위다 - Windows Installer 가 서로 다른 패키지 파일에 기대하는 대로. MSI 에는 시각이 전혀 들지 않으므로(요약 정보에 만든·저장한 날짜가 없고 캐비닛 항목은 1980-01-01) SOURCE_DATE_EPOCH 는 결과를 바꾸지 않는다. 원본의 경로나 지은 컴퓨터·사용자의 이름도 들지 않는다. 원본 파일은 한 번만 읽고, 그 바이트를 해시하고 판을 읽고 담는다. 짓는 동안 바뀐 파일은 빌드를 멈춘다(RP1515).
- --nfc 는 패키지가 폴더·파일·바로가기에 붙이는 이름을 유니코드 NFC(조합형)로 바꾼다. macOS 에서 온 파일은 흔히 이름이 분해형이다 - 한글 음절 U+D55C 가 자모 셋 U+1112 U+1161 U+11AB 로 - 그리고 Windows 는 이름을 있는 그대로 설치하므로 같은 낱말이 서로 다른 두 파일이 될 수 있다. 원본 파일의 이름은 그대로 두고, 문구와 레지스트리 값도 적은 대로 둔다. lint 는 NFC 가 아닌 문구를 경고하고(RP2105), 다른 이름과 같아지는 이름은 거부한다(RP1511). 정규화는 Unicode 17.0 자료로 rubrapack 이 직접 한다.
- inspect <file.msi> <table> 은 표를 Windows Installer 의 IDT 형식으로 찍는다. --files 는 파일마다 탭으로 나눈 한 줄을 찍는다: 설치 경로, 크기, File 키, 구성 요소, 판, 언어, MD5(MsiFileHash 에서; 없으면 비어 있다). --streams 는 스트림과 그 크기를 늘어놓는다. inspect <file.cab> 은 캐비닛 속 파일을 늘어놓는다. inspect <file.msix> 는 (블록 해시를 모두 확인한 뒤) 정제, 실행 파일, 파일을 보인다. --files 는 경로, 크기, 압축 여부를, --manifest 는 AppxManifest.xml 을 찍는다. 묶음은 정제와 패키지(하나하나 패키지로 열어 확인한 것)와 AppxBundleManifest.xml 을 보이고, extract 는 패키지를 꺼내 쓴다.
- new <name> 은 <name>.toml 을 쓴다. 프로그램 파일을 dist/ 에 넣기만 하면 빌드되는 원본이고 upgrade-code 는 새로 만든다. new <file>.toml, 이름 없는 new(파일 이름도 묻는다), new <name> -i 는 대신 첫 패키지의 질문을 한다(질문은 표준 오류로, 답은 표준 입력에서 한 줄씩 받으므로 파이프로 줄 수 있다; 마지막 답 전에 입력이 끝나면 아무것도 쓰지 않는다). new <name> 에 옵션을 주면 묻지 않고 같은 답을 받는다: --name, --manufacturer, --version, --dist(기본 dist), --main <파일>|-, --arch, --install-dir, --scope, --optional <폴더,...>|-, --ui, --license <파일>|-, --languages ko|-, --shortcuts start,desktop|none; 빠진 것은 질문이 권했을 기본값을 쓴다. 원본은 폴더 최상위 파일을 하나씩 적고(바로가기는 파일을 가리키고, 그룹은 파일을 뺄 수 없다) 파일이 든 하위 폴더는 그룹으로 적는다. 선택 하위 폴더는 필수 Main 옆에 level = 2 기능이 된다. 쓴 뒤에는 lint 처럼 검사한다(문제가 있으면 종료 코드 1). 있는 파일은 절대 바꾸지 않는다. 이미 있는 app.toml 에 new app.toml 을 주면 edit 를 쓰라고 알린다.
- edit <file>.toml 은 원본을 그 자리에서 고친다. 옵션이 없으면 메뉴를 보인다: 1 이름, 제조사, 판(version = "\$ (VERSION)" 이면 [define] VERSION), 아키텍처; 2 Program Files 아래 폴더와 설치 범위; 3 대화창, 약관, 한국어 대화창; 4 선택 구성요소로 할 하위 폴더(처음 켜면 필수 기능 Main 을 더하고, 그때부터 기능이 필요한 모든 것에 준다); 5 시작 메뉴·바탕화면 바로가기와 그것이 여는 프로그램; 6 프로그램 폴더와 파일 목록 맞추기 - 사라진 파일과 하위 폴더는 빼고 새것은 더한다; 7 아무 표.키; v 는 글을 보이고, l 은 lint 처럼 검사하고, s 는 저장하고 검사하며, q 는 나간다(바뀐 것이 있으면 먼저 묻는다). 질문마다 지금 값을 권한다. 바꾼 값만 다시 쓰고 주석, 순서, 손으로 쓴 표는 그대로다. 구문이 깨질 변경은 하지 않는다. 옵션을 주면 묻지 않고 차례로 적용해 저장한다: --set package.version=1.2.0(TOML 값 - "글", 수, true, ["ko"]) - 이면 그대로, 아니면 문자열로 쓰고, 없는 표는 더한다), --unset package.license, --sync(묻지 않는 6). UTF-16 원본은 거부한다: 먼저 UTF-8 로 저장한다.
- guid 는 무작위 GUID(4판)를 찍는다. guid --from <text> 는 rubrapack 이 글에서 끌어내는 GUID 를 어느 컴퓨터에서나 같게 찍는다: guid 의 32비트 리틀엔디언 길이와 그 4바이트, 32비트 리틀엔디언 수 1, 글

의 32비트 리틀엔디언 길이와 UTF-8 바이트에 대한 SHA-256. 해시의 앞 16바이트에 판 니블을 8 로, 변 중 비트를 10 으로 넣어(RFC 9562 UUIDv8) 대문자 {XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX} 로 쓴다. 예를 들어 guid --from hello 는 {B52FE8EF-B68D-84B5-91AF-E6E01BEC2773} 이다.

- 진단은 example.toml:12:3: error[RP1201]: unknown key 'nmae' in [package] (did you mean 'name'?) 꼴이다.
- 쓰기 전에 build 는 완성된 표를 검사한다(RP20xx 진단, 종료 코드 5). 이 검사는 rubrapack 자신을 지키는 것이라, RP1xxx 검사를 통과한 원본은 여기에 걸리지 않아야 한다.
- lint <src.toml> 는 build 가 하는 검사를 모두 하고 아무것도 쓰지 않는다. 원본을 MSI 기준으로 검사하며, --target msix 를 주면 MSIX 기준으로 검사한다([msix] 표, 그리고 keep 이나 when 처럼 MSIX 가 담을 수 없는 것). lint <file.msi> 는 어떤 도구가 만든 패키지든 같은 표 규칙으로 검사한다: 설치를 멈추게 하는 것은 오류(종료 코드 5) - 열에 맞지 않는 값, 가리키는 행이 없음, 중복 키, 깨진 대화창 Tab 순서(RP2101), 대화창의 첫/기본/최소 컨트롤이 없음(RP2102), ListBox 표 없는 사용 중 파일 대화창(RP2103), ErrorText/ErrorIcon 없는 오류 대화창(RP2104) - 이고 나머지는 경고다. --strict 는 경고도 실패로 친다. 65001 이 아닌 코드 페이지의 패키지에는 알림(RP2100)이 붙는다: 그 글자는 UTF-8 로 검사할 수 없다. 표준 출력의 마지막 줄은 오류와 경고의 수다. lint <file.msix> 는 Windows 가 읽는 방식대로 - ZIP, 블록 맵, 블록마다의 해시 - 검사하고, 매니페스트에 정체가 있는지, 매니페스트가 가리키는 파일이 패키지에 있는지 본다(RP2201, RP2202). 묶음은 매니페스트의 블록 맵, 그리고 패키지마다 매니페스트가 말하는 자리·크기·정체와 패키지 자체를 본다. 아키텍처 하나에 패키지 하나다.
- lint new.msi --previous old.msi 는 new.msi 가 old.msi 를 깨끗하게 업그레이드하는지도 본다. 오류: 다른 UpgradeCode(RP2301: 새 패키지가 옛것을 대체하지 않는다), 앞의 세 자리가 높지 않은 버전(RP2302: Windows 는 그 세 자리만 비교한다). 경고: 같은 ProductCode(RP2303: 업그레이드에는 새 것이 필요하다), GUID 는 그대로인데 키 경로(파일, 폴더, 레지스트리 값)가 바뀐 컴포넌트(RP2304), 64비트 표시가 바뀐 컴포넌트(RP2305), 없어진 컴포넌트(RP2306: 그 자원은 옛 판과 함께 지워진다), 없어진 기능(RP2307: 패치나 설치된 기능 바꾸기에서 사라진다). 마지막 줄은 이전 패키지를 밝힌다.
- extract 는 패키지를 설치되는 모습대로 본다: 폴더는 Directory 트리의 긴 이름으로(ProgramFiles64Folder 같은 표준 폴더는 그 이름 그대로), 파일은 안팎의 캐비닛에서, 또는 압축하지 않은 패키지라면 옆의 원본 폴더에서 가져온다. 파일마다 크기와 MsiFileHash 를 확인한다. 대상은 새 폴더거나 비어 있어야 하고, 패키지 전체가 통과할 때까지 아무것도 쓰지 않는다: .., /, \, :, 드라이브, 예약 장치 이름(CON, COM1 등), 끝의 점이나 공백이 든 이름, 대소문자만 다른 두 경로는 거부한다. 기본 한도는 항목 100,000개와 16 GiB 다. .cab 은 안의 이름대로 본다. .msix 는 블록이 모두 해시와 맞은 뒤 파일(ZIP 자신의 [Content_Types].xml 과 AppxBlockMap.xml 은 빼고)을 본다.
- sign 은 PE 파일, MSI 패키지, MSIX 패키지나 묶음(안의 패키지를 먼저 서명한다)에 Authenticode 서명(SHA-256; RSA 또는 ECDSA P-256/P-384)을 제자리에서, 또는 -o 로 붙인다. build --key 는 짓는 김에(같은 코드로) 서명하고, 서명한 --reproducible 빌드도 매번 같은 바이트다(서명에 시각이 없다) - 타임스탬프가 붙지 않는 한: 타임스탬프에는 서버의 시각과 일련번호가 들어 매번 다르고, 서명 안의 패키지만 같다. 캐비닛이 밖에 있는 MSI 는 거부한다: 서명이 그것을 덮지 못한다 - 캐비닛을 안에 넣거나, --allow-unsigned-cabs 로 경고와 함께 .msi 만 서명한다. 키 파일은 PKCS#12 파일(.pfx/.p12: PBKDF2-HMAC-SHA256 의 AES, SHA-256 MAC - 요즘 Windows 와 OpenSSL 이 내보내는 것)이거나, 평문 또는 PBES2 로 암호화한 PKCS#8 PEM/DER 키다. --cert 는 키 파일에 없는 인증서를 더한다. 인증서는 코드 서명용이어야 하고, CA 가 아니어야 하며, 지금 유효해야 한다. 암호는 환경 변수나 파일에서 받고 명령줄에서는 절대 받지 않는다. 이미 서명된 파일은 거부한다. MSIX 의 [msix] publisher 는 Windows 가 쓰는 방식의 인증서 주체 - 부분을 마지막부터 처음으로, O=Example Ltd, CN=Example 처럼 - 와 같아야 하고, 다르면 서명을 멈추고 써야 할 주체를 알려 준다. --unsigned-test 와 키는 함께 쓰지 않는다.
- 하드웨어 밖으로 나가서는 안 되는 키 - 2023년부터 공개 신뢰 코드 서명 인증서에 요구되는 것 - 는 거기 머문다: rubrapack 이 해시, CMS 구조, 타임스탬프 요청을 스스로 계산하고 서명 연산 하나만 맡긴다. --pkcs11 <모듈> 은 토큰의 PKCS#11 라이브러리를 불러오고(rubrapack 이 불러오는 유일한 라이브러리고, 이름 준 것만 불러온다), --pin-env 나 --pin-file 의 PIN 으로(명령줄은 절대 아님) 로그인해, --key-label 이라는 이름표의 개인 키와 토큰에 같은 ID 로 저장된 인증서를 쓴다(토큰에 인증서가 없으면 --cert 가 그것이나 사슬의 나머지를 준다). 토큰이 여럿이면 --token-label 로 고른다. Windows 에서 --key-store <지문> 은 현재 사용자의 개인 저장소(--machine-store: 로컬 컴퓨터의 것)에 있고 개인 키에 NCrypt 로 닿을

수 있는 인증서 - 키 저장소 공급자를 거친 스마트카드·토큰·TPM, 또는 소프트웨어 키 - 를 쓰고, Windows 가 만들어 주는 사슬을 가져온다. RSA 키와 ECDSA 키 모두 된다. 서명은 모두 파일에 쓰기 전에 인증서로 확인한다.

- `keys list` 는 서명에 쓸 수 있는 키를 보인다: `--pkcs11` 이면 토큰의 개인 키마다 이름표, ID, 인증서를, Windows 에서 그것 없으면 개인 저장소에서 개인 키가 있는 인증서와 그 지문을 보인다. 인증서마다 그것이 서명하는 MSIX publisher 와 만료일을 찍고, 비밀은 아무것도 찍지 않는다.
- `--timestamp <URL>` 은 RFC 3161 타임스탬프 서버에 서명의 연서명을 청해, 인증서가 만료된 뒤에도 서명이 유효하게 한다. 없으면 `sign` 과 `build --key` 는 그렇지 않을 것이라고 경고한다. 기본 서버는 없고, 서버를 적지 않으면 네트워크에 아무것도 나가지 않는다. 공개 서버로는 <http://timestamp.digicert.com>, <http://timestamp.sectigo.com>, <http://time.certum.pl> 이 있다. `http` 여도 괜찮다: 답 자체가 서명되어 있고 확인하기 때문이다(nonce, 서명의 해시, 서버의 서명과 그 타임스탬프 인증서). `--tsa-trust` 는 서버 인증서의 경로가 준 인증서 가운데 하나에서 끝나야 한다는 조건을 더한다. `https` 서버와는 rubrapack 이 TLS 1.3 을 직접 말하고(AES-GCM, x25519 또는 P-256, RSA-PSS 또는 ECDSA 서버 키; TLS 1.2 는 아님) 서버 인증서를 확인한다 - `--tls-trust` 로 준 루트나 `--system-roots` 로 운영 체제 저장소에서 찾은 루트까지의 경로, 날짜, 서버 이름을 담았는지(subjectAltName; 점을 넘는 와일드카드 없음), TLS 서버용인지. 이 검사를 끄는 방법은 없다. 세 가지 신뢰는 섞이지 않는다: `--tls-trust` 와 `--system-roots` 는 연결을 누구 말로 믿을지, `--tsa-trust` 는 시각을 누구 말로 믿을지를 정하고, 어느 것도 다른 쪽에 쓰이지 않는다. Linux 에서 `--system-roots` 는 `$SSL_CERT_FILE` 이나 배포판의 CA 묶음을 읽고, Windows 에서는 ROOT 저장소를 읽는다. Windows 는 어떤 루트를 무언가가 처음 필요로 할 때에야 채우므로 그때까지 없을 수 있다. `--proxy http://host:port` 는 요청을 프록시로 보낸다(`https` 서버는 `CONNECT` 로, 그래서 프록시는 서버 이름만 본다). 없으면 `https_proxy` 나 `HTTPS_PROXY` (`https` 서버), `http_proxy`(`http`; 대문자 `HTTP_PROXY` 는 읽지 않는다 - CGI 환경에서는 요청 머리가 그것을 정할 수 있다)를 쓰되, `no_proxy` 나 `NO_PROXY` - 이름들, 각각 그 아래 이름까지 포함, 또는 * - 에 서버가 있으면 쓰지 않는다. 암호를 묻는 프록시와 `https://` 프록시는 지원하지 않는다(분명한 오류). 타임스탬프가 실패하면 - 답 없음, 거절, 잘못된 답, 서명 인증서의 유효 기간 밖의 시각 - 아무것도 서명하지 않고 아무것도 쓰지 않는다(종료 코드 6). 서명이 타임스탬프 없이 슬그머니 나가는 일은 없다. 한도: 연결 10초, 전체 60초, 답 4 MiB, 리디렉션 3번, 네트워크 오류나 5xx 답 뒤 재시도 2번.
- `verify` 는 무엇을 확인했는지 찍는다 - 구조, 다이제스트, 서명, `--trust` 로 준 인증서까지의 경로, 폐기 여부(찾아보지 않는다: not-checked), 타임스탬프 - 그리고 모두 맞고 경로가 신뢰하는 인증서에서 끝날 때만 성공한다. `--trust` 가 없으면 아무것도 신뢰한다고 하지 않는다(종료 코드 4). `--system-roots` 는 서명자의 경로에 대해(타임스탬프 쪽은 아님) `--trust` 에 운영 체제의 루트를 더한다. Windows 자신의 평판 검사에 대해서는 아무것도 말하지 않는다. 타임스탬프 줄은 not-present, <시각>, TSA not-checked (give `--tsa-trust`), <시각>, TSA trusted, <시각>, TSA untrusted, invalid, unsupported(Set-AuthenticodeSignature -TimestampServer 가 쓰는 옛 Authenticode 타임스탬프로, rubrapack 은 확인하지 않는다) 가운데 하나이고 뒤의 셋은 실패다. 서버가 `--tsa-trust` 로 신뢰되는 타임스탬프만 - `--trust` 는 여기에 치지 않는다 - 서명자 인증서가 유효해야 하는 시각을 지금에서 찍힌 시각으로 옮긴다. (시험용으로, 환경 변수 `RUBRAPACK_TEST_NOW` - 1970년부터의 초 - 가 `verify` 검사의 "지금"을 대신한다. 그래서 시계를 바꾸지 않고 만료된 인증서를 확인할 수 있다. 그 시각으로 맞춘 시계가 허용하지 않을 것은 아무것도 넓히지 않는다.)
- 종료 코드: 0 성공, 1 원본 오류, 2 사용법, 3 입출력, 4 서명, 5 lint, 6 네트워크.

10.6 진단 코드

rubrapack 이 알리는 모든 문제에는 파일, 줄, 열 다음에 코드가 붙는다: `error[RPnnnn]` 또는 `warning[RPnnnn]`. 앞의 두 자리가 문제의 종류를 말한다:

코드	무엇이 잘못됐나	볼 곳
RP00xx	명령줄: 모르는 명령이나 옵션, 읽거나 쓸 수 없는 파일	rubrapack help <명령>
RP10xx	원본 파일의 인코딩: UTF-8(또는 BOM 있는 UTF-16)이 아님, 짝 없는 캐리지 리턴	파일을 UTF-8 로 저장한다

코드	무엇이 잘못됐나	볼 곳
RP11xx	rubrapack 이 읽는 부분집합 밖의 TOML: 여러 줄 문자열, 인라인 표, 두 번 정의한 표	TOML 부분 집합
RP12xx	표와 키: 모르는 표나 키(제안과 함께), 빠진 필수 키나 표, 기능이 생긴 뒤 기능 없는 항목	표
RP13xx	값: ID(모든 표에 걸쳐 유일, 예약어 아님), GUID, 판, 범위를 벗어난 수, 없는 것을 가리키는 참조	그 표의 절
RP14xx	변수: 값 없는 \$(NAME), 닫히지 않은 \$(	변수
RP15xx	설치할 파일: 없음, 폴더임, 링크임, 맞는 것 없는 그룹, 다른 아키텍처의 프로그램, 너무 긴 이름, 빌드 중에 바뀐 파일, 너무 큰 패키지	경로, 프로그램 파일
RP16xx	MSIX: MSIX 패키지에 필요한 것, 답을 수 없는 것	MSIX 패키지
RP19xx	이 rubrapack 이 제공하지 않는 기능	-
RP20xx	완성된 표가 Windows Installer 규칙을 어김. RP1xxx 검사를 통과한 원본은 여기에 걸리지 않아야 한다: 알려 주기 바란다	-
RP21xx	패키지 lint: 대화창, 코드 페이지, 글자 정규화	명령줄의 lint
RP22xx	MSIX 패키지나 묶음의 lint	명령줄의 lint
RP23xx	lint --previous: 이 패키지가 앞 판을 깨끗하게 업그레이드하지 못함	판과 업그레이드

무엇을 고칠지는 메시지가 말한다. 매뉴얼에서 찾을 때는 위의 코드 범위를 쓴다.

제3부

파일 형식

11 구현하는 사람을 위한 패키지 형식

이 장의 차례

11.1	사실을 읽는 법	53
11.2	표기	54
11.3	흐름을 한 그림으로	54

이 문서들은 Windows 가 받아들이는 Windows Installer 패키지(.msi)와 MSIX 패키지·묶음을 만들려면 프로그램이 무엇을 써야 하는지, 그리고 그것들에 어떻게 서명하는지를 설명한다. 공개 명세와 Windows 자체에 대한 실험으로 썼으므로, rubrapack 의 코드를 읽지 않고도 이것만으로 자기 도구를 만들 수 있다 - 또는 rubrapack 의 코드를 이것과 대조해 볼 수 있다.

문서	다루는 것
복합 파일 (Compound File Binary, CFB)	복합 파일(Compound File Binary): 모든 .msi 의 그릇
복합 파일 속의 MSI 데이터베이스	스트림, 문자열 풀, 시스템 표, 표 인코딩, IDT 내보내기
요약 정보	요약 정보 스트림과 그 코드 페이지 함정
설치·복구·업그레이드·제거가 되는 가장 작은 패키지	설치·업그레이드·복구·제거를 하는 표들: 파일, 레지스트리, 바로가기, 서비스, 글꼴, 대화창, 사용자별 패키지, 순서
프로그램 파일: 머신 형식과 판	프로그램 파일: 머신 형식과 판 정보 리소스
캐비닛, MSZIP, deflate	캐비닛 파일, MSZIP 블록, deflate 인코더
결정적인 정체와 재현 가능한 패키지	결정적인 GUID 와 키, 재현 가능한 빌드
Authenticode 서명	PE 파일과 MSI 패키지의 Authenticode 서명: 다이제스트, Windows 가 받는 CMS 구조, ECDSA, RFC 3161 타임스탬프
출력을 Windows 와 대조하기	Windows 자신의 구성 요소로 결과를 확인하는 방법
MSIX 패키지	MSIX: ZIP 배치, 블록 맵, 매니페스트, 가상 레지스트리와 파일 시스템, 확장, 묶음, 서명
레지스트리 하이브 파일(REGF)	MSIX Registry.dat·User.dat 가 담는 레지스트리 하이브 파일(REGF)

11.1 사실을 읽는 법

모든 서술에는 출처 표시가 붙는다:

- **[spec]** - 공개 명세: [MS-CFB], [MS-OLEPS], [MS-CAB], [MS-MCI], RFC 1951, RFC 1321, RFC 3161, RFC 5652, RFC 9562, PKWARE ZIP APPNOTE, ECMA-376 Part 2(OPC), Authenticode PE 명세, 또는 Microsoft Learn 의 Windows Installer·MSIX 참고 문서.
- **[observed]** - Windows 11 에서 켜진 것: MSI 문서는 25H2(빌드 26200)와 Windows Installer 5.0.10011 에서 msi.dll 로 데이터베이스를 만들어 바이트 단위로 다시 읽고 패키지를 설치해서, MSIX·레지스트리 하이브·서명 문서는 빌드 26100 에서 Windows 자신의 패키징 API, 오프라인 레지스트리 라이브러리, 서명

기와 대조하고 패키지를 설치·실행해서 얻었다. 관찰은 그 빌드들에 대한 사실이다. Microsoft 가 형식을 문서로 내지 않은 곳(MSI 표 인코딩, 레지스트리 하이브, MSIX 서명의 해시 기록은 공개되지 않았다)에서 는 관찰이 유일한 출처다.

여기의 어떤 글이나 코드도 다른 MSI·MSIX·캐비닛·레지스트리 구현에서 오지 않았다.

11.2 표기

- 따로 말하지 않으면 모든 정수는 리틀엔디언이다.
- u16, u32, u64 는 부호 없는 수, i16, i32 는 2의 보수 부호 있는 수다.
- 오프셋은 이름 붙은 구조의 처음부터 센 바이트 수다.
- 16진 바이트열은 파일에 놓인 순서로 쓴다: D0 CF 11 E0.

11.3 흐름을 한 그림으로

```
원본 기술 --> 표(문자열/정수/스트림의 행)
            --> MSI 데이터베이스 인코딩(문자열 풀 + 표마다 스트림 하나)
            --> 캐비닛에 담은 파일(저장 또는 MSZIP) --> 스트림 하나 더
            --> 요약 정보 스트림
            --> 모든 스트림을 복합 파일 하나에 쓴다 = .msi
```

MSIX 패키지는 더 단순하다: 페이로드 파일(각각 독립된 64 KiB 조각으로 deflate), AppxManifest.xml, AppxBlockMap.xml(조각마다 해시), [Content_Types].xml 이 ZIP64 압축 파일 하나에 들고, 레지스트리 값은 Registry.dat/User.dat 하이브에 든다. 묶음은 패키지를 나란히 담고, 서명은 AppxSignature.p7x 를 더한다(MSIX 패키지, 레지스트리 하이브 파일(REGF)).

화살표 하나가 위의 문서 하나다. 어느 것도 만드는 데 Windows 가 필요하지 않다: rubrapack 은 Linux 와 Windows 에서 같은 바이트를 만든다.

12 복합 파일(Compound File Binary, CFB)

이 장의 차례

12.1	섹터	55
12.2	머리(처음 512바이트) [spec]	55
12.3	FAT, DIFAT, 미니 FAT [spec]	56
12.4	디렉터리 [spec]	56
12.5	단순하고 결정적인 작성기	57
12.6	안전하게 읽기	57

.msi 파일은 복합 파일("구조적 저장소")이다: 파일 하나 안에 이름 붙은 스트림들이 든 작은 파일 시스템이다. 형식은 [MS-CFB]가 정한다. 이 문서는 MSI 작성기에 필요한 부분과 Windows 자신의 작성기가 하는 일을 다룬다.

12.1 섹터

- 파일은 크기가 고정된 섹터의 연속이다. **3판**은 512바이트 섹터(섹터 시프트 9), **4판**은 4096바이트 섹터(시프트 12)를 쓴다. [spec]
- 머리(header)는 오프셋 0에 있다. 섹터 번호 N 은 두 판 모두 파일 오프셋 $(N + 1) * \text{sector_size}$ 에서 시작한다. 그래서 4판에서는 머리가 4096바이트 섹터 하나를 통째로 차지하고, 머리 뒤의 바이트는 0이다. [spec]
- msi.dll은 크기와 상관없이(20 KB에서 10 MB까지 해 보았다) 새 데이터베이스를 모두 **4판**으로, 부판(minor version) 0x003E로 쓴다. 설치할 때는 두 판 다 받는다. rubrapack은 기본으로 4판을 쓰고 3판도 쓸 수 있다. [observed]

체인과 표 속의 특별한 섹터 번호: [spec]

값	뜻
FFFFFFFA	가장 큰 보통 섹터 번호
FFFFFFFC	DIFAT 섹터(FAT 안에서)
FFFFFFFD	FAT 섹터(FAT 안에서)
FFFFFFFE	체인의 끝
FFFFFFF	빈 섹터 / 스트림 없음

12.2 머리(처음 512바이트) [spec]

오프셋	크기	필드	쓰는 값
0	8	서명	D0 CF 11 E0 A1 B1 1A E1
8	16	CLSID	0
24	2	부판	003E
26	2	주판	3 또는 4
28	2	바이트 순서	FFFE(바이트 FE FF)
30	2	섹터 시프트	9 또는 12
32	2	미니 섹터 시프트	6(64바이트 미니 섹터)
34	6	예약	0
40	4	디렉터리 섹터 수	3판은 0, 4판은 그 수

오프셋	크기	필드	쓰는 값
44	4	FAT 섹터 수	
48	4	첫 디렉터리 섹터	
52	4	트랜잭션 서명	0
56	4	미니 스트림 경계	4096
60	4	첫 미니 FAT 섹터	없으면 FFFFFFFE
64	4	미니 FAT 섹터 수	
68	4	첫 DIFAT 섹터	없으면 FFFFFFFE
72	4	DIFAT 섹터 수	
76	436	처음 109개 FAT 섹터 번호(DIFAT)	안 쓰는 칸은 FFFFFFFF

12.3 FAT, DIFAT, 미니 FAT [spec]

- FAT 는 섹터마다 u32 하나인 배열로, 그 섹터가 속한 체인의 다음 섹터를 준다. FAT 섹터 자신은 FFFFFFFD 로 표시한다.
- FAT 자신의 섹터 번호는 DIFAT 에 적는다: 처음 109개는 머리에, 나머지는 DIFAT 섹터에. DIFAT 섹터 하나는 $\text{sector_size} / 4 - 1$ 개의 번호를 담고, 마지막 u32 에 다음 DIFAT 섹터(끝이면 FFFFFFFE)를 담는다. DIFAT 섹터는 FAT 에서 FFFFFFFC 로 표시한다.
- 4096바이트 섹터에서는 FAT 섹터 109개가 이미 약 457 MB 를 덮으므로, 실제 4판 MSI 파일에는 DIFAT 섹터가 없다. 512바이트 섹터에서는 약 7 MB 부터 DIFAT 가 생긴다.
- 경계(4096바이트) 보다 작은 스트림은 미니 스트림에 산다. 64바이트 미니 섹터로 나뉘고 미니 FAT(FAT 와 같은 배치)로 이어진다. 미니 스트림 자체는 루트 디렉터리 항목의 보통 스트림으로 저장된다.

FAT 는 자기 섹터와 DIFAT 의 섹터까지 덮어야 한다. 작성기는 수가 더 바뀌지 않을 때까지 되풀이해 셈한다.

12.4 디렉터리 [spec]

디렉터리는 128바이트 항목을 담은 섹터 체인이다. 항목 0 이 루트다.

오프셋	크기	필드
0	64	이름, UTF-16LE, NUL 로 끝남(최대 31자 + NUL)
64	2	NUL 을 포함한 이름 길이(바이트)
66	1	종류: 0 안 씀, 1 저장소, 2 스트림, 5 루트
67	1	색: 0 빨강, 1 검정
68	4	왼쪽 형제(FFFFFFF = 없음)
72	4	오른쪽 형제
76	4	자식(저장소와 루트만)
80	16	CLSID
96	4	상태 비트
100	8	만든 시각
108	8	고친 시각
116	4	시작 섹터(크기에 따라 보통 또는 미니)
120	8	스트림 크기(3판에서는 아래 32비트만 센다)

- 루트 항목의 이름은 Root Entry 이고, 그 시작 섹터와 크기는 미니 스트림을 가리킨다.
- MSI 의 루트 항목은 CLSID {000C1084-0000-0000-C000-000000000046} 을 가지며, 바이트로는 84 10 0C 00 00 00 00 00 C0 00 00 00 00 00 46 이다. [spec] [observed]
- 안 쓰는 항목은 링크 필드 셋이 FFFFFFFF 인 것을 빼고 모두 0 이다. [spec]

- 저장소의 자식들은 이름 길이, 그다음 대문자로 바꾼 UTF-16 이름의 비교 순으로 늘어선 **레드-블랙 트리**를 이룬다. [MS-CFB]는 가장 단순한 올바른 트리도 허용한다: 모든 노드가 검정이고 트리는 그냥 균형 잡힌 이진 탐색 트리. rubrapack 은 그것 - 정렬한 이름 위의 균형 트리, 모든 노드 검정 - 을 만들고, msi.dll 은 그것을 연다. [spec] [observed]
- 시각은 0 이어도 된다. 시각을 0 으로 쓰는 것(그리고 시계에서 오는 다른 입력을 쓰지 않는 것)이 CFB 작성기를 결정적으로 만든다. [spec]

12.5 단순하고 결정적인 작성기

rubrapack 은 파일을 다음 순서로, 모든 구간을 이어서 놓는다:

머리 | FAT 섹터 | DIFAT 섹터 | 디렉터리 | 미니 FAT | 미니 스트림 | 큰 스트림들을 차례로

1. 스트림 크기를 "미니"(< 4096)와 "보통"으로 나눈다.
2. 미니 스트림들은 64바이트 단위로 이어 붙인다. 미니 스트림 길이는 그 합이다.
3. 디렉터리, 미니 FAT, 미니 스트림 섹터 수를 세고, 그다음 FAT 와 DIFAT 섹터 수를 센다(되풀이).
4. 이어진 구간마다 FAT 체인을 채우고, 디렉터리 트리, 미니 FAT, FAT, DIFAT, 머리를 쓴다.

12.6 안전하게 읽기

판독기는 다음을 따라가지 말고 거부해야 한다: 도는 체인(파일의 섹터 수 안에 FFFFFFFE 에 닿지 않는 체인), 파일 밖의 섹터 번호, 스트림 크기가 요구하는 것보다 짧은 체인, 범위 밖의 디렉터리 링크, 한 노드를 두 번 지나는 형제 트리, 상식적인 한도를 넘는 수. 어느 것도 비용이 크지 않고, 그렇지 않으면 조작된 .msi 가 판독기를 멈추게 하거나 죽인다.

13 복합 파일 속의 MSI 데이터베이스

이 장의 차례

13.1	스트림	58
	스트림 이름	58
13.2	문자열 풀: <code>_StringPool</code> 과 <code>_StringData</code>	58
	코드 페이지	59
13.3	시스템 표	59
	열 형식 비트(<code>_Columns.Type</code>)	59
13.4	표 스트림	60
13.5	IDT 보관 파일(<code>MsiDatabaseExport</code> 가 쓰는 것)	60
13.6	안전하게 읽기	60

Microsoft 는 Windows Installer 표의 *뜻*은 문서로 내지만(Microsoft Learn, "Database Tables"), 데이터베이스가 복합 파일 안에 어떻게 놓이는지는 내지 않는다. 이 문서에서 배치에 관한 모든 것은 `msi.dll` 로 데이터베이스를 만들고, 스트림을 바이트 단위로 다시 읽고, 이 규칙으로 쓴 데이터베이스가 열리고 내보내지고 설치되는지 확인해서 알아냈다. 따로 표시하지 않으면 [observed] 다.

13.1 스트림

표 하나가 루트 저장소 바로 아래의 스트림 하나다. 보통 패키지에는 하위 저장소가 없다.

스트림 이름

T 라는 표는 U+4840 뒤에 T 를 UTF-16 단위 하나에 두 글자씩 압축해 붙인 이름으로 저장한다:

1. 다음 64글자 알파벳의 각 글자를 0..63 으로 바꾼다: 0-9 -> 0..9, A-Z -> 10..35, a-z -> 36..61, . -> 62, _ -> 63.
2. 알파벳 글자 둘이 이어지면 단위 하나 $0x3800 + \text{first} + (\text{second} \ll 6)$ 가 된다.
3. 알파벳 글자 하나(마지막 글자, 또는 뒤에 알파벳 밖의 글자가 오는 글자)는 $0x4800 + \text{value}$ 가 된다.
4. 그 밖의 글자는 그대로 쓴다.

압축한 이름(표는 U+4840 접두 포함)은 CFB 이름 한도 31단위 안에 들어야 하므로, 표 이름은 알파벳 글자 60 개까지 쓸 수 있다. 풀 때는 이 단계를 거꾸로 밟는다.

예: `_Columns` -> U+4840 U+3B3F U+43F2 U+4438 U+45B1.

다른 스트림:

스트림	이름
Binary/Icon 칸 (OBJECT 열)	<Table>.<key> 를 같은 방식으로 압축하되 U+4840 접두 없이, 예 <code>Binary.Logo</code> (복합 키는 . 로 잇는다)
안에 넣은 캐비닛	<code>Media.Cabinet</code> 의 # 뒤 이름(예 <code>cab1.cab</code>), 압축, 접두 없음
요약 정보	<code>\x05SummaryInformation</code> - 압축하지 않는다(요약 정보 참고)

13.2 문자열 풀: `_StringPool` 과 `_StringData`

모든 표의 모든 문자열은 공유 풀에 한 번씩만 살고, 표의 칸은 문자열 id 를 담는다.

`_StringPool`:

u32 header 아래 31비트: 데이터베이스 코드 페이지; 비트 31: 문자열 참조가 3바이트
그다음 문자열 id 1, 2, 3, ... 마다(id 0 은 "문자열 없음"이고 저장하지 않는다):
u16 length _StringData 속 바이트 길이
u16 refs 비트 0-14: 참조 수; 비트 15: 문자열에 0x80 이상인 바이트가 있을 때만 켜짐

- **긴 문자열**(64 KiB 이상의 바이트): 항목은 length = 0, refs 는 위와 같고(0 아님) 그 뒤에 전체 바이트 길이를 담은 u32 가 하나 더 온다. 이 둘이 여전히 id 하나를 기술한다. 본 예: 한 번 쓰인 70002바이트 문자열 - > 0000 8001 00011172.
- length = 0 이고 refs = 0 인 항목은 빈 id 다.
- **참조 수** = 그 문자열을 담은 칸의 수, 더하기 표마다 _Tables 행 몫 1, 열마다 _Columns 의 표 이름 몫 1 과 열 이름 몫 1. (msi.dll 이 만든 모든 데이터베이스를 다시 세어 확인했다.)
- _StringData 는 모든 문자열의 바이트를 id 순서대로 이어 붙인 것이고, 데이터베이스 코드 페이지로 되어 있다. 길이는 글자 수가 아니라 **바이트** 수다.
- **빈 문자열은 저장하지 않는다**: 칸에 "" 를 쓰면 null 이 저장된다.
- 서로 다른 문자열이 65535개를 넘으면 머리의 비트 31 이 켜지고, 모든 표의 모든 문자열 참조가 3바이트(아래 16비트, 그다음 위 바이트)가 된다.

코드 페이지

데이터베이스 코드 페이지는 풀 머리의 아래 비트다: 한 번도 정하지 않았으면 0, 아니면 _ForceCodepage 가 쳐오기로 정한 값. **65001(UTF-8)은 요즘 Windows 에서 모든 표에 통한다**: 한국어, 일본어, 보조 평면 (예 U+20000, U+1F600)의 파일·폴더 이름, 레지스트리 키와 값, 바로가기 이름, 제품 이름이 모두 정확한 UTF-16 글자로 설치되고 대화창에도 보인다 - ANSI 코드 페이지가 한국어를 담지 못하는 시스템 로캘(en-US, 1252)에서도 그렇다. 그래서 따로 유니코드 설치 프로그램이 필요 없다. UTF-16 코드 페이지(1200)는 여기서 쓸 수 없다: 풀은 한 바이트 단위의 문자열을 담는다. [observed] Windows Installer 는 이름을 정규화하지 않는다: 분해형 한글(U+1100 U+1161)로 이름 붙인 폴더는 바로 그 코드 단위로 설치되고, U+AC00 으로 이름 붙인 폴더를 대신하지 않고 그 옆에 생긴다. [observed]

13.3 시스템 표

표	열	키
_Tables	Name(문자열)	Name
_Columns	Table(문자열), Number(i2, 1부터), Name(문자열), Type(i2)	Table, Number

_Tables 는 자기 자신과 _Columns 를 신지 않는다. 행이 없는 표는 스트림이 아예 없고 _Tables·_Columns 행만 있다. _Validation 은 있다면 보통 표다.

열 형식 비트(_Columns.Type)

비트	뜻
0x00FF	너비: 문자열 최대 길이(0 = 제한 없음), 또는 정수의 2 / 4
0x0100	유효
0x0200	지역화 가능
0x0400	비이진(글자면 켜짐, OBJECT 면 꺼짐)
0x0800	문자열(글자와 OBJECT 면 켜짐)
0x1000	null 허용
0x2000	기본 키의 일부

흔한 SQL 열 형식에 msi.dll 이 쓰는 값:

SQL	Type
CHAR(72) NOT NULL(키)	0x2D48

SQL	Type
CHAR(72)(null 허용)	0x1D48
CHAR(255) NOT NULL LOCALIZABLE	0x0FFF
LONGCHAR NOT NULL LOCALIZABLE	0x0F00
SHORT NOT NULL	0x0502
SHORT(null 허용)	0x1502
SHORT NOT NULL(키)	0x2502
LONG NOT NULL	0x0104
OBJECT NOT NULL	0x0900

정수 열에는 비이진 비트가 없다.

13.4 표 스트림

- 표 스트림은 **열 우선**이다: 모든 행의 1열 칸, 그다음 모든 행의 2열 칸, 이런 식이다. 행 수는 스트림 크기를 행 너비로 나눈 것이다.
- 칸 너비: 문자열 참조 2바이트(긴 참조면 3), 16비트 정수 2바이트, 32비트 정수 4바이트, OBJECT 2바이트.
- 정수는 부호 비트를 뒤집어 저장한다: $\text{stored} = \text{value} \wedge 0x8000$ (16비트) 또는 $\text{value} \wedge 0x80000000$ (32비트). 저장된 0 은 null 이다. 그래서 -32768 과 -2147483648 은 저장할 수 없다.
- OBJECT 칸은 스트림이 있으면 1, null 이면 0 을 담는다.
- 행은 기본 키 칸의 저장된 수 - 문자열 id, 뒤집은 정수 - 로, 키 열 순서대로 정렬한다. Windows Installer 는 이 순서에 기댄다.

문자열 id 를 문자열의 바이트 순서로 매기는 작성기는 행을 문자열 값 순으로 저절로 정렬하게 되고, 이것은 IDT 내보내기가 쓰는 순서이기도 하다.

13.5 IDT 보관 파일(MsiDatabaseExport 가 쓰는 것)

기준 출력으로 쓸모 있다: 바이트까지 같은 IDT 파일을 내보내는 판독기는 Windows 와 같은 방식으로 데이터베이스를 푸는 것이다.

1줄: TAB 으로 나눈 열 이름
 2줄: 열 형식: s/S(문자열), l/L(지역화 문자열), i/I(정수), v/V(이진),
 대문자 = null 허용, 뒤에 너비(0 = 제한 없음; 정수는 2 또는 4)
 3줄: [<코드 페이지> TAB] <표 이름> TAB <키 열> [TAB <키 열> ...]
 행 : TAB 으로 나눈 칸, 줄 끝은 CR LF

- 3줄의 코드 페이지 접두는 표에 ASCII 가 아닌 문자열이 있을 때만 나온다.
- 행은 기본 키 값(문자열은 바이트, 정수는 부호 있게)으로 정렬한다 - 저장 순서가 아니다.
- null 은 빈 필드로 찍는다. 이진 칸은 <key>.ibd 를 찍고, 바이트는 <Table>/<key>.ibd 로 간다.
- 값 속의 TAB, CR, LF 는 0x10, 0x11, 0x19 로 쓴다. [spec]
- _ForceCodepage.idt 는 빈 줄 둘 다음에 <코드 페이지> TAB _ForceCodepage 다.
- _SummaryInformation.idt 는 머리 줄 PropertyId TAB Value, i2 TAB 1255, _SummaryInformation TAB PropertyId 와 속성마다 한 행을 가진다. 날짜는 내보내는 컴퓨터의 지역 시각으로 yyyy/mm/dd hh:mm:ss 로 찍는다.

13.6 안전하게 읽기

풀 길이의 합이 _StringData 크기와 정확히 같은지, 표의 모든 문자열 참조가 쓰이는 id 를 가리키는지, 표 스트림 크기가 행 너비의 배수인지, _Columns 가 표마다 열을 1..n 으로 빈틈없이 번호 매기는지 확인한다.

14 요약 정보

이 장의 차례

14.1	배치 [spec] [observed]	61
14.2	설치 패키지가 정하는 속성	61
14.3	코드 페이지 함정 - 요약 문자열은 ASCII 로 [observed]	61

모든 패키지에는 \x05SummaryInformation 스트림이 있다(첫 글자는 U+0005, 이름은 압축하지 않는다). [MS-OLEPS]가 정한 OLE 속성 집합이고, Microsoft Learn("Summary Information Stream Property Set")이 설치 패키지가 어떤 속성을 쓰는지 알려 준다.

14.1 배치 [spec] [observed]

```

u16 바이트 순서      FFFE
u16 판               0 (msi.dll 은 0 을 쓴다)
u32 OS 필드         msi.dll 은 0x00020206 을 쓴다
16 CLSID             0
u32 구역 수          1
16 FMTID             F29F85E0-4FF9-1068-AB91-08002B27B3D9, 저장은 E0 85 9F F2 F9 4F 68 10 AB 91 08 00 2B 27 B3 D9
u32 구역 오프셋      48

```

구역:

u32 구역 크기

u32 속성 수

수만큼 (u32 속성 id, u32 구역 처음부터의 오프셋)

값들, 각각 4바이트 배수로 채움:

VT_I2 (2) u32 형식, i16 값, 채움 2바이트

VT_I4 (3) u32 형식, i32 값

VT_LPSTR (30) u32 형식, NUL 포함 u32 바이트 수, 바이트, NUL, 채움

VT_FILETIME (64) u32 형식, 1601-01-01 UTC 부터의 100ns 간격 수 u64

14.2 설치 패키지가 정하는 속성

Id	이름	형식	값
2	Title	LPSTR	예 Installation Database
3	Subject	LPSTR	제품 이름
4	Author	LPSTR	제조사
5	Keywords	LPSTR	예 Installer
7	Template	LPSTR	<플랫폼>;<언어 id>: x64;1033, Arm64;1042, Intel;1033
9	Revision Number	LPSTR	패키지 코드: GUID, 서로 다른 패키지 파일마다 새로
12, 13	Create / Last Save time	FILETIME	선택
14	Page Count	I4	최소 설치 엔진 판: x64 와 x86 은 200, Arm64 는 500
15	Word Count	I4	비트 1 = 파일이 (캐비닛에) 압축되어 있음; 보통 패키지는 2
18	Creating Application	LPSTR	아무것

14.3 코드 페이지 함정 - 요약 문자열은 ASCII 로 [observed]

속성 1(CodePage, VT_I2)은 LPSTR 바이트의 인코딩을 말한다. Windows 11 25H2 에서:

- **65001:** msi.dll 은 문자열을 UTF-8 로 제대로 쓰지만, 코드 페이지를 부호 있는 값 -535 로 다시 읽어 모 든 문자열 변환에 실패하고(설치 로그: "Failed to retrieve # 9 summary info property. GetLastError returned 87, the code page is -535") 오류 1620 으로 패키지를 거부한다.
- **1200(UTF-16):** 똑같이 거부한다(로그: "... the code page is 1200"). [MS-OLEPS]대로 완전한 UTF-16 문자열과 바이트 수를 써도 그렇다.
- **코드 페이지 속성 없음, ASCII 문자열:** 설치된다.

그러니 속성 1 은 빼고 요약 문자열은 모두 ASCII 로 쓴다. 지역화한 제품 이름은 데이터베이스 코드 페이지를 쓰는 Property 표(ProductName)에 둔다 - 거기서는 65001 이 통한다(복합 파일 속의 MSI 데이터베이스 참고).

15 설치·복구·업그레이드·제거가 되는 가장 작은 패키지

이 장의 차례

15.1	표와 열	63
15.2	Property	64
15.3	Directory	64
15.4	구성 요소, 기능, 파일	64
15.5	빈 폴더	64
15.6	Media 와 캐비닛	65
15.7	메이저 업그레이드와 다운그레이드 거부	65
15.8	설치한 프로그램으로 등록·해제하기	65
15.9	레지스트리 값	65
15.10	바로가기	66
15.11	파일 지우기와 복제	66
15.12	환경 변수	66
15.13	INI 파일	66
15.14	검색과 설치 조건	67
15.15	서비스, 글꼴, 권한	67
15.16	캐비닛, 관리 이미지, 광고	67
15.17	사용자별 패키지와 검용 패키지	67
15.18	사용 중인 파일	67
15.19	순서	68
15.20	대화창	68
15.21	이 조리법이 아직 다루지 않는 것	69

표의 뜻은 Microsoft Learn("Database Tables", "Standard Actions", "Suggested InstallExecuteSequence")에 있다. 이 문서는 시험을 거친 조리법이다: 정확히 이대로 지은 패키지는 Windows 11 에서 컴퓨터 전체로 설치되고, 복구를 견디고, 옛 판을 업그레이드하고, 다운그레이드를 거부하고, 깨끗이 제거된다. [observed]

15.1 표와 열

형식은 복합 파일 속의 MSI 데이터베이스의 표기를 쓴다(s72 = CHAR(72), 대문자 = null 허용, l = 지역화 가능, i2/i4 정수). 키는 굵게 쓴다.

표	열
Property	Property s72, Value l0
Directory	Directory s72, Directory_Parent S72, DefaultDir l255
Component	Component s72, ComponentId S38, Directory_ s72, Attributes i2, Condition S255, KeyPath S72
Feature	Feature s38, Feature_Parent S38, Title L64, Description L255, Display I2, Level i2, Directory_ S72, Attributes i2
FeatureComponent	Feature_ s38, Component_ s72
File	File s72, Component_ s72, FileName l255, FileSize i4, Version S72, Language S20, Attributes I2, Sequence i4
MsiFileHash	File_ s72, Options i2, HashPart1..HashPart4 i4

표	열
CreateFolder	Directory_ s72, Component_ s72
Media	DiskId i2, LastSequence i4, DiskPrompt L64, Cabinet S255, VolumeLabel S32, Source S72
Upgrade	UpgradeCode s38, VersionMin S20, VersionMax S20, Language S255, Attributes i4, Remove S255, ActionProperty s72
CustomAction	Action s72, Type i2, Source S72, Target S255
InstallExecuteSequence	Action s72, Condition S255, Sequence I2
InstallUISequence	

File.Sequence 와 Media.LastSequence 를 32비트 정수로 두면 패키지에 파일을 32767개 넘게 담을 수 있다.

15.2 Property

ProductCode, ProductName, ProductVersion(a.b.c, a 와 b 는 255 까지, c 는 65535 까지; 넷째 부분도 되지만 판을 비교할 때는 무시된다), Manufacturer, ProductLanguage(1033, 1042, ...), UpgradeCode, ALLUSERS=1(컴퓨터 전체), REBOOT=ReallySuppress, MSIRESTARTMANAGERCONTROL=Disable(아래 "사용 중인 파일" 참고), 그리고 아래 업그레이드 속성 둘을 나열한 SecureCustomProperties.

15.3 Directory

- 부모가 없고 DefaultDir = SourceDir 인 TARGETDIR.
- 쓰는 표준 폴더(ProgramFiles64Folder, ProgramFilesFolder, ...)는 저마다 DefaultDir = . 인 TARGETDIR 의 자식. 이 이름들은 Windows 가 스스로 푼다. x64·Arm64 패키지는 ...64Folder 이름을, x86 은 64 없는 이름을 쓴다.
- 그 아래의 자기 폴더는 DefaultDir = SHORT!Long name. 긴 이름은 어떤 유니코드 글이든 된다(코드 페이지 65001 일 때). 짧은 이름은 올바른 8.3 ASCII 이름이어야 하고, 같은 부모의 모든 파일·폴더 사이에서 달라야 한다. 긴 이름이 이미 올바른 8.3 이름이면 그것만 써도 된다.

15.4 구성 요소, 기능, 파일

- 파일 하나에 구성 요소 하나, 그 파일이 키 경로다. ComponentId 는 제품의 수명 동안 같은 자리의 같은 자원에 대해 변하지 않아야 하는 GUID 다(결정적인 정체와 재현 가능한 패키지 참고). Attributes 는 x64·Arm64 패키지면 256(64비트), x86 이면 0.
- Level 1 인 기능. Display 0 은 그것을 숨긴다. 모든 구성 요소를 FeatureComponents 에 짝짓는다. 자식 기능은 Feature_Parent 에 부모를 적는다. Level 이 INSTALLLEVEL(패키지가 정하지 않으면 1)보다 높은 기능은 기본으로 설치하지 않는다. 뒤에 MsiConfigureFeature 로 더할 수 있고, 그러면 그 파일들이 들어온다. [observed]
- File.FileName 은 DefaultDir 처럼 SHORT!Long 이다. Attributes 512 는 파일을 필수(vital)로 표시한다. Sequence 번호 1..n 은 캐비닛 속 파일의 순서다.
- 판 정보 리소스가 없는 파일에는 MsiFileHash 행이 필요하다: 파일의 MD5 를 리틀엔디언 32비트 정수 넷으로 읽은 것(부호 있는 값으로 저장). 설치된 파일은 MsiGetFileHash 로 해시하면 정확히 이 값이 나온다. 판 정보 리소스가 있는 파일은 대신 File.Version 에 판을 적는다(해시는 필요 없다).

15.5 빈 폴더

파일 없이 있어야 하는 폴더에는 제 Directory 행, Directory_ 가 그 폴더이고 KeyPath 가 null 인 구성 요소(폴더 자체가 키 경로), 둘을 가리키는 CreateFolder 행, FeatureComponents 행을 준다. CreateFolders 가 설치할 때 만들고, RemoveFolders 가 제거할 때 비어 있으면 지운다. [observed] 구성 요소 속성 16(영구)이면 다른 것은 다 지워져도 폴더는 남는다. [observed]

15.6 Media 와 캐비닛

행 하나: DiskId 1, LastSequence n, Cabinet #cab1.cab - # 는 "이 패키지 속 cab1.cab 이라는 스트림"이라는 뜻이다. 캐비닛은 파일을 File 키 이름으로 Sequence 순서대로 담는다 (캐비닛, MSZIP, deflate). 요약의 Word Count 2 는 파일이 캐비닛에 압축되어 있다는 뜻이다.

15.7 메이저 업그레이드와 다운그레이드 거부

패키지의 UpgradeCode 를 가진 Upgrade 행 둘:

VersionMin	VersionMax	Attributes	ActionProperty	뜻
이 판	-	0x102(찾기만, 최솟값 포함)	NEWER_FOUND	같거나 새로운 판이 설치됨
-	이 판	0x001(기능 옮기기)	OLDER_FOUND	옛 판이 설치됨: 지운다

Upgrade 표는 UpgradeCode, 판, 언어로만 맞춰 보고 아키텍처는 보지 않는다. x64 빌드와 x86 빌드가 UpgradeCode 를 같이 쓰면 하나를 설치할 때 다른 것이 "옛 판"으로 지워진다. 둘을 나란히 설치할 수 있어야 하면 아키텍처마다 제 UpgradeCode 를 준다(확인함: 코드가 다른 두 아키텍처는 서로 상관없이 설치·제거된다). [observed]

NEWER_FOUND 를 조건으로 한 오류 사용자 지정 동작(형식 19, Target = 메시지; 서식 문자열이라 [ProductName] 이 된다)은 같거나 새로운 판이 있을 때 오류 1603 으로 설치를 멈춘다. 판마다 새 ProductCode 가 필요하다(그리고 패키지 파일마다 새 패키지 코드).

15.8 설치한 프로그램으로 등록·해제하기

어떤 제품(입력기, 셀 확장)은 표의 행이 아니라 자기 프로그램으로 스스로를 등록해야 한다. 설치·복구·업그레이드·제거를 원자적으로 지키는 방법은 그 프로그램의 구성 요소 c 의 동작 상태(\$C = 이번 설치가 그것에 하는 일, ?C = 그 전의 상태; 2 없음, 3 로컬)를 쓰는 것이다 [spec]. 형식 18 동작(Source = 그 프로그램의 File 키, Target = 인수)을 모두 지연(0x400), 가장하지 않음(0x800)으로 둔다:

동작	형식	자리	조건	하는 일
...UndoRollback	18+0x100+0x400+0x800+0x40	RemoveFiles 앞	\$C=2 AND ?C=3	등록(다음 행의 되돌림)
...Undo	18+0x400+0x800	그 뒤, RemoveFiles 앞	\$C=2 AND ?C=3	해제
...DoRollback	18+0x100+0x400+0x800+0x40	InstallFiles 뒤	\$C>2 AND ?C<3	해제
...RedoRollback	같음	그 뒤	\$C>2 AND ?C=3	등록
...Do	18+0x400+0x800	둘 다 뒤	\$C>2	등록

되돌림 동작은 그것이 되돌리는 동작보다 앞에 놓아야 한다: 되돌림은 스크립트를 거꾸로 실행하고, 이미 스크립트에 든 되돌림 동작만 실행한다. 되돌림 동작은 종료 코드를 무시하고(0x40) 앞으로 가는 동작은 그러지 않으므로, 등록이나 해제가 실패하면 설치가 실패하고 되돌려진다. Windows 11 에서 본 것: 복구는 ...Do 를 다시 실행한다. 해제가 실패하면 제거가 되돌려지고 (아직 있는) 프로그램이 다시 등록된다. 업그레이드가 실패하면 새 패키지의 ...DoRollback 이 실행되고, 옛 판의 파일이 돌아오고, 옛 패키지 자신의 ...UndoRollback 이 옛 프로그램을 다시 등록한다. [observed]

InstallInitialize 와 RemoveExistingProducts 사이에는 스크립트에 쓰는 것이 아무것도 있어서는 안 된다: 거기 지연 동작이나 되돌림 동작이 하나라도 있으면 모든 업그레이드가 오류 2613("RemoveExistingProducts action sequenced incorrectly")으로 멈춘다. [observed]

15.9 레지스트리 값

Registry(Registry s72, Root i2, Key l255, Name L255, Value L0, Component_ s72) 는 WriteRegistryValues(5000)가 쓰고 제거할 때 되돌린다. RemoveRegistry(Value 없는 같은 열)는 RemoveRegistryValues(2600)가 처리한다. Root 0 = HKCR, 1 = HKCU, 2 = HKLM. 값의 첫 글자가 형식을 정한다: #x 이진(16진), %% 확장 문자열, # 뒤에 숫자는 DWORD(#4294967295 까지), 어디든 [~] 가 있으면 다중 문

자열([~]a[~]b[~]). # 으로 시작하는 보통 문자열은 ## 로 쓴다. RemoveRegistry 의 Name - 는 키 전체를 지운다. 설치가 실패하면 덮어쓴 값과 지운 키가 되살아난다. [observed]

키 경로가 레지스트리 값인 구성 요소는 속성 4 와 KeyPath = 그 Registry 행을 가진다. 구성 요소의 64비트 속성(256)이 레지스트리 보기를 고른다: 그것이 없으면 64비트 패키지 32비트 보기(WOW6432Node)에 쓴다. 64비트 패키지 속 32비트 구성 요소는 허용되고, 32비트 패키지 속 64비트 구성 요소는 허용되지 않는다.

REG_QWORD 는 Registry 표로 나타낼 수 없다. rubrapack 은 그것을 Binary 표의 DLL 사용자 지정 동작(형식 1)으로 쓴다: 즉시 동작이 구성 요소 동작 상태와 현재 값을 읽어 목록 둘을 CustomActionData 로 넘긴다 - 하나는 쓰거나 지우는 지연 동작에게, 하나는 원래 있던 것을 되살리는 그 되돌림 짝에게. DLL 의 비트 수는 패키지와 맞아야 한다(엔진은 그것을 패키지 아키텍처의 사용자 지정 동작 서버에 싣는다). [observed: x64 와 x86]

파일 형식과 URL 스킴은 Extension/Verb/ProgId 표(처음 쓸 때 광고와 복구를 끌고 온다)가 아니라 프로그램의 구성 요소에 든 Root 0(HKCR) 아래의 보통 Registry 행이다: .ext(기본값) = ProgId; ProgId(기본값) = 그 설명, ProgId\DefaultIcon = [#File], 0, ProgId\shell\open\command = "[#File]" "%1". 스킴은 scheme(기본값) = URL:<설명>, URL Protocol = 빈 값(Name 이 있고 Value 가 null 이면 빈 문자열을 쓴다), 그리고 같은 DefaultIcon 과 shell\open\command. Root 0 은 설치 방식을 따른다: 컴퓨터 전체면 HKLM\Software\Classes, 사용자별이면 HKCU\Software\Classes. 그러면 이 프로그램만 말하는 형식의 파일을 열 때 곧바로 이 프로그램이 시작된다. [observed]

15.10 바로가기

Shortcut(Shortcut s72, Directory_ s72, .lnk 없는 Name l128 SHORT!Long, Component_ s72, 광고 아닌 바로가기면 Target [#FileKey], Arguments(서식), Description(평문), Hotkey, Icon_, IconIndex, ShowCmd, WkDir = Directory 키)는 CreateShortcuts(4500)가 쓰고 RemoveShortcuts(3200)가 지운다. 바로가기는 대상 파일의 구성 요소에 둔다. 바로가기만을 위해 만든 폴더는 저절로 지워지지 않는다: 그 폴더와 표준 폴더 아래의 부모마다 RemoveFile 행(FileName null, DirProperty = 그 폴더, InstallMode 2)을 더한다. ALLUSERS=1 이면 ProgramMenuFolder 와 DesktopFolder 는 모든 사용자의 시작 메뉴와 공용 바탕화면이다. 설치가 실패하면 바로가기도 폴더도 남지 않고, 복구는 지운 바로가기를 다시 만든다. [observed]

15.11 파일 지우기와 복제

RemoveFile(FileKey s72, Component_ s72, */? 가 든 FileName L255 또는 폴더면 null, DirProperty s72, InstallMode i2: 1 설치, 2 제거, 3 둘 다)은 구성 요소가 설치될 때(1)나 제거될 때(2) RemoveFiles 에서 실행된다. 키 경로 파일이 없고 Directory_ 가 있는 구성 요소가 그 운반자 노릇을 한다. DuplicateFile(FileKey s72, Component_ s72 = 원본의 구성 요소, File_ s72, DestName L255 SHORT!Long, DestFolder S72)은 DuplicateFiles(4210)와 RemoveDuplicateFiles 에서 실행된다. 설치 때 지운 파일은 설치가 실패하면 돌아온다. 설치 전부터 있던 폴더는 비어 있어도 제거할 때 그대로 둔다. [observed]

15.12 환경 변수

Environment(Environment s72, Name l255, 서식 Value L255, Component_ s72)는 WriteEnvironmentStrings(5200)와 RemoveEnvironmentStrings 가 쓴다. Name 접두: * 시스템 변수(없으면 사용자 변수), = 만들거나 정함, - 구성 요소를 제거할 때 되돌림. Value [~];x 는 ;x 를 뒤에 붙이고 x;[~] 는 앞에 붙인다. - 가 있으면 제거할 때 정한 변수는 지우고, 원래 있던 변수에서는 붙인 부분만 떼어 낸다. - 가 없으면 아무것도 되돌리지 않는다. 설치가 실패하면 이전 값이 되살아난다. [observed]

15.13 INI 파일

IniFile(IniFile s72, FileName l255 SHORT!Long, DirProperty S72, Section l96, Key l128, 서식 Value l255, Action i2: 0 줄 더하기, 1 줄 만들기, 3 태그 더하기, Component_ s72)은 WriteIniValues(5100)가 적용하고 구성 요소를 제거할 때 되돌린다. RemoveIniFile(같은 열, Value null 허용, Action 2 줄 지우기, 4 태그 지우기)은 InstallFiles 전에 실행되는 RemoveIniValues 가 처리한다. 태그 더하기는 쉼표 목록에 ,value 를

붙이고, 제거하면 그 항목만 땀다. ASCII 가 아닌 파일·구역·키·값 이름도 된다. 설치가 실패하면 파일이 되살아난다. [observed]

15.14 검색과 설치 조건

두 순서 모두 50 에 있는 AppSearch(**Property, Signature_**)는 같은 서명을 가진 찾개(locator)로 속성을 정한다: RegLocator(Root, Key, Name, Type 2 = 원 값, +16 = 64비트 보기), 파일이면 Signature 행 (FileName, MinVersion)을 결들인 DrLocator(Path 는 [System64Folder] 같은 폴더 속성으로 시작해도 된다), 또는 CompLocator(ComponentId, Type 1 = 키 파일). 그 속성은 공개 이름이어야 하고, 서버 쪽까지 가려면 SecureCustomProperties 에 있어야 한다. 두 순서 모두 100 에 있는 LaunchCondition(**Condition**, 서식 Description)은 조건이 거짓이면 그 설명으로 설치를 멈춘다. [observed]

15.15 서비스, 글꼴, 권한

ServiceInstall(Name, 서식 DisplayName 과 Description, ServiceType 0x10, StartType 2/3/4, ErrorControl 1, StartName null 또는 NT AUTHORITY\LocalService, 서식 Arguments, Component_ = exe 의 구성 요소)은 InstallServices(5800)와 함께 쓴다. ServiceControl 의 Event 플래그는 0x1 설치 때 시작, 0x2 설치 때 멈춤, 0x20 제거 때 멈춤, 0x80 제거 때 지움이고 Wait 1, StopServices(1900), DeleteServices(2000), StartServices(5900)와 함께 쓴다. Font(File_, FontTitle null = 글꼴에서 읽음)는 RegisterFonts/UnregisterFonts 와 함께 쓰고, 파일은 FontsFolder 바로 안에 있어야 한다. MsiLockPermissionsEx(MsiLockPermissionsEx, LockObject, Table = CreateFolder, File, Registry, ServiceInstall 가운데 하나, **SDDLText**(열 이름), Condition)는 Windows Installer 5.0 이 필요하다(요약 page count 500). 폴더는 CreateFolder 행을 거쳐 그것을 받는다. 설치가 실패하면 서비스도 글꼴도 남지 않는다. [observed]

15.16 캐비닛, 관리 이미지, 광고

캐비닛이 여럿이면 저마다 Media 행 하나(DiskId 1..n, LastSequence = 그 안 마지막 파일의 순번). 안에 넣은 것은 Cabinet 에 # 와 함께 이름 적은 스트림이고, 밖의 것은 패키지 옆에 있는 파일로 Cabinet 에 # 없이 이름을 적는다(긴 이름도 된다). AdminExecuteSequence(CostInitialize 800, FileCost 900, CostFinalize 1000, InstallValidate 1400, InstallInitialize 1500, InstallAdminPackage 3900, InstallFiles 4000, InstallFinalize 6600)와 AdminUISequence 는 msixexec /a 가 원본처럼 설치되는 압축 없는 이미지를 쓰게 한다. AdvtExecuteSequence(CostInitialize, CostFinalize, InstallValidate, InstallInitialize, PublishFeatures 6300, PublishProduct 6400, InstallFinalize)는 msixexec /jm 이 제품을 광고하게 한다. [observed]

15.17 사용자별 패키지와 겸용 패키지

패키지 하나로 하는 방법: ALLUSERS=2 와 MSIINSTALLPERUSER=1 은 사용자별로 설치한다 - 엔진이 ProgramFilesFolder/ProgramFiles64Folder 를 %LOCALAPPDATA%\Programs 로, 메뉴와 바탕화면 폴더를 그 사용자의 것으로 돌린다 - 그리고 ALLUSERS=1 MSIINSTALLPERUSER="" 는 컴퓨터 전체로 설치한다. 요약 Word Count 비트 8 은 권한 상승이 필요 없다는 뜻이다. Registry Root -1(HKMU)은 사용자별이면 HKCU, 컴퓨터 전체면 HKLM 이다. * 없는 Environment 이름은 사용자 변수다. 사용자별 제품이 설치되고 나면 엔진은 MSIINSTALLPERUSER 를 지우므로, 그것을 보는 설치 조건은 Installed OR ... 로 써야 한다. 그러지 않으면 제품을 더는 제거할 수 없다. 일반적으로 설치 조건은 모두 Installed OR (...) 로 쓴다. [observed]

15.18 사용 중인 파일

바꾸거나 지을 파일을 실행 중인 프로그램이 쥐고 있을 때(그 안에 DLL 이 실려 있을 때):

- 기본인 Restart Manager 를 쓰면 Windows Installer 는 **조용한 설치(/qn)**에서도 그런 파일을 **권 프로그램**을 모두 **끄러** 하고 뒤에 다시 시작한다. 하나라도 닫히지 않으면 설치 전체가 실패한다(1601). [observed]
- Property 표에 MSIRESTARTMANAGERCONTROL=Disable 이 있으면 엔진은 쥐인 파일을 옆으로 옮기고(c:\Config.Msi*.rbf) 새 파일을 곧바로 제자리에 두고 0 을 돌려준다. 프로그램은 옛 사본으로 계속 돌고, 그 뒤에 시작한 프로그램은 새 파일을 싣고, 옛 사본은 다음 재시작 때 지워진다. [observed]

- 명령줄에서 준 속성은 설치하고 있는 패키지에만 간다. 업그레이드 중 옛 판의 제거는 **옛 패키지의** Property 표를 따른다 - 그 속성은 첫 판부터 넣어 둔다. [observed]

15.19 순서

InstallExecuteSequence(대괄호 안은 조건):

```
FindRelatedProducts 25, <다운그레이드 거부 동작> 30 [NEWER_FOUND], CostInitialize 800,
FileCost 900, CostFinalize 1000, MigrateFeatureStates 1200, InstallValidate 1400,
InstallInitialize 1500, RemoveExistingProducts 1501, ProcessComponents 1600,
UnpublishFeatures 1800, RemoveFiles 3500, RemoveFolders 3600, CreateFolders 3700,
InstallFiles 4000, RegisterUser 6000, RegisterProduct 6100, PublishFeatures 6300,
PublishProduct 6400, InstallFinalize 6600
```

InstallUISequence: FindRelatedProducts 25, 다운그레이드 거부 동작 30 [NEWER_FOUND], CostInitialize 800, FileCost 900, CostFinalize 1000, MigrateFeatureStates 1200, ExecuteAction 1300(그리고 대화창, 대화창 참고).

InstallInitialize 바로 뒤의 RemoveExistingProducts 는 옛 판의 제거를 새 설치의 트랜잭션 안에 넣는다. 그래서 업그레이드가 실패하면 옛 판으로 되돌아간다. [observed: InstallFiles 바로 뒤에 실패하는 지연 사용자 지정 동작이 있으면 업그레이드는 1603 으로 끝나고 옛 판이 파일까지 바이트 그대로 다시 등록된다. 첫 설치가 실패하면 파일도 등록도 남지 않는다.]

15.20 대화창

대화창이 있는 패키지는 다음 표를 가진다(형식은 위와 같다):

표	열
Dialog	Dialog s72, HCentering i2, VCentering i2, Width i2, Height i2, Attributes I4, Title L128, Control_First s50, Control_Default S50, Control_Cancel S50
Control	Dialog_ s72, Control s50, Type s20, X i2, Y i2, Width i2, Height i2, Attributes I4, Property S72, Text L0, Control_Next S50, Help L50
ControlEvents	Dialog_ s72, Control_ s50, Event s50, Argument s255, Condition S255, Ordering I2
ControlCondition	Dialog_ s72, Control_ s50, Action s50, Condition s255
EventMapping	Dialog_ s72, Control_ s50, Event s50, Attribute s50
TextStyle	TextStyle s72, FaceName s32, Size i2, Color I4, StyleBits I2
UIText	Key s72, Text L255
Binary	Name s72, Data v0(스트림)
RadioButton	Property s72, Order i2, Value s64, X i2, Y i2, Width i2, Height i2, Text L0, Help L50
ComboBox	Property s72, Order i2, Value s64, Text L64

엔진이 대화창을 보일 때 검사하는 것(저마다 그 번호를 단 오류 대화창으로 보았다):

- **Tab 순서.** Control_Next 는 그것을 가진 모든 컨트롤을 Control_First 에서 시작해 한 바퀴 도는 고리를 이루어야 한다. 컨트롤 하나를 빠뜨리거나 Control_Next 가 빈 컨트롤에서 끝나는 사슬은 2810 이나 2809 로 설치를 멈춘다. 정적 글씨는 고리에 끼지 않는다. [observed] rubrapack 은 짓는 대화창마다 검사하고 깨진 것은 쓰지 않는다(RP0010).
- **오류 대화창.** ErrorDialog 속성이 가리키는 대화창에는 ErrorText 라는 Text 컨트롤과 ErrorIcon 이라는 Icon 컨트롤이 있어야 한다(없으면 2835). Attributes 에 오류 대화창 비트(0x10000)가 있고, EndDialog 인수 ErrorAbort .. ErrorYes 를 가진 A, C, I, N, O, R, Y 누름단추가 메시지에 필요한 대로 보인다. [observed]
- **문구 속 대괄호.** Text 는 서식 문자열이다: [Next] 는 속성으로 읽혀 아무것도 보이지 않는다. [observed] 본 문에서 단추 이름은 대신 따옴표로 적는다.
- **사용 중인 파일.** Restart Manager 가 꺼져 있으면 엔진은 FilesInUse 라는 대화창(Retry, Ignore, Exit 로 끝나는 단추)을 FileInUseProcess 에 묶인 ListBox 와 함께 보인다. **ListBox** 표는 **비어 있어도 있어야 한다**: 없

으면 엔진은 2205 를 로그에 남기고 대화창을 건너뛰며, 설치하는 무시를 고른 것처럼 계속된다. 메이저 업그레이드에서는 대화창이 두 번 나온다(새 파일, 그다음 옛 판의 제거). 무시한 뒤에는 쥐인 파일을 옆으로 옮기고 설치는 0 으로 끝난다. [observed]

- **오류 아이콘.** 엔진은 컨트롤의 Binary 가 무엇을 담은 메시지 종류에 맞는 자기 아이콘(설치 조건이면 경고 삼각형)을 ErrorIcon 에 넣는다. [observed]
- **목록 순서.** ComboBox 는 컨트롤에 Sorted 속성(0x10000)이 없으면 항목을 가나다순으로 늘어놓고, 있으면 Order 열을 따른다. [observed] rubrapack 은 그것을 켜다.
- **권한 상승.** 권한 없는 사용자가 대화형으로 시작한 컴퓨터 전체 설치의 실행 순서가 시작될 때 보안 데스크톱에서 동의(UAC)를 묻는다. [observed]
- **코드 페이지.** 데이터베이스 코드 페이지가 65001 이면 한국어 제목, 문구, RTF 사용권 문구가 한국어 시스템 로캘에서도 영어(1252) 시스템 로캘에서도 제대로 보인다. observed, [msi-database.md 참고]

rubrapack 의 세트: 모든 대화창은 370 x 270 대화창 단위이고, 띠(그림이 Binary.RpBanner 인 Bitmap 컨트롤 0,0,370,44)에 굵은 제목({\RpTitle}, TextStyle)과 설명 한 줄이 있고, 44 와 234 에 Line, y 243 에 뒤로/다음/취소가 있다. InstallUISequence 는 환영(1230, NOT Installed)이나 유지 관리 대화창(1240, Installed AND NOT RESUME AND NOT Preselected), 진행 대화창(1280, 모델리스, EventMapping 으로 ActionText 와 SetProgress 를 구독), 그리고 -1(성공), -2(취소), -3(치명적 오류)의 끝 대화창 셋을 더한다. 설치 폴더 대화창은 폴더의 Directory 키를 PathEdit 에 넣고, 그 다음 단추는 NewDialog 전에 경로를 검사하는 SetTargetPath 를 실행한다. 사용권 문구는 RTF 를 담은 ScrollableText 컨트롤이다: 평문은 \uN? 이스케이프(U+FFFF 너머의 글자는 대리쌍)로, 줄마다 \par 하나로 바뀐다.

작성자가 만든 페이지는 같은 틀 속의 보통 Dialog 행이다. Control_Next 는 컨트롤들을 위치 순서대로 지나 뒤로, 다음, 취소로 간다. 라디오 묶음은 단추가 RadioButton 행(묶음 기준 위치)인 RadioButtonGroup 컨트롤이고, 드롭다운 목록은 ComboList(0x20000)와 Sorted 가 있는 ComboBox 컨트롤로 ComboBox 행에서 채운다. 그 속성들은 SecureCustomProperties 에 더해지므로, 대화창에서 고르거나 명령줄에서 준 값이 실행 순서까지 간다. [observed]

15.21 이 조리법이 아직 다루지 않는 것

위의 오류 형식, 등록 쌍, REG_QWORD 도우미 말고 다른 사용자 지정 동작, 그리고 _Validation(검증 도구에는 필요하지만 설치 엔진에는 필요 없다). rubrapack 이 구현하는 대로 이 문서들도 자란다.

16 프로그램 파일: 머신 형식과 판

이 장의 차례

16.1	머리 찾기 [spec]	70
16.2	판 정보 리소스 [spec]	70
16.3	패키지에 드는 것 [observed]	70
16.4	안전하게 읽기	71

패키지 작성기는 담는 PE 파일(.exe, .dll 등)마다 두 가지를 읽는다: 잘못된 아키텍처용 파일을 거부하기 위한 머신 형식, 그리고 Windows Installer 가 판이 있는 파일과 없는 파일을 다르게 다루기 때문에 판 정보 리소스. 배치는 Microsoft PE/COFF 명세와 VS_VERSIONINFO 문서를 따른다. [spec]

16.1 머리 찾기 [spec]

오프셋 0	"MZ"
오프셋 0x3C	u32 e_lfanew: PE 서명의 오프셋
e_lfanew	"PE\0\0"
+4	COFF 머리: u16 machine, u16 섹션 수, ..., u16 섹션 머리 크기(+16)
+24	선택 머리: u16 magic 0x10B(PE32) 또는 0x20B(PE32+); 데이터 디렉터리는 +96(PE32) 또는 +112(PE32+)에서 그 개수 다음에 시작
그다음	섹션 표: 섹션마다 40바이트(가상 크기 +8, 가상 주소 +12, 원 크기 +16, 원 오프셋 +20)

머신 값: 0x8664 x64, 0xAA64 Arm64, 0x014C x86. MZ 로 시작하지만 올바른 PE\0\0 머리가 없는 파일은 보통 데이터 파일이다.

16.2 판 정보 리소스 [spec]

- 데이터 디렉터리 2 가 리소스 표다(RVA 와 크기). RVA 는 섹션 표를 거쳐 파일 오프셋으로 바꾼다.
- 리소스 디렉터리는 세 단계다: 형식 -> 이름 -> 언어. 디렉터리마다 16바이트 머리(이름 항목 수와 ID 항목 수가 +12, +14)와 그 뒤 8바이트 항목(u32 이름 또는 ID, u32 오프셋; 오프셋의 최상위 비트는 하위 디렉터리 표시)이 있다.
- 형식 16(RT_VERSION), 그 첫 이름, 그 첫 언어: 앞은 데이터 항목(u32 RVA, u32 크기)이다.
- 데이터는 VS_VERSIONINFO 다: u16 길이, u16 값 길이, u16 형식, NUL 까지의 UTF-16 키 VS_VERSION_INFO, 4바이트 채움, 그다음 첫 u32 가 0xFEEF04BD 인 VS_FIXEDFILEINFO. dwFileVersionMS(+8)와 dwFileVersionLS(+12)가 판 HIWORD(MS).LOWORD(MS).HIWORD(LS).LOWORD(LS) 를 준다.

16.3 패키지에 드는 것 [observed]

- 판이 있는 파일: File.Version = 네 부분 판, File.Language = MsiGetFileVersion 이 알려 주는 언어, 곧 VarFileInfo\Translation 의 첫 언어(예 1033, 1042). MsiFileHash 행은 없다. 설치된 파일을 다시 읽으면 같은 판이다.
- 판이 없는 파일: 판 없음, MsiFileHash 행 있음(설치·복구·업그레이드·제거가 되는 가장 작은 패키지).
- 머신 형식: x64 패키지는 x64 파일을, Arm64 패키지는 Arm64 파일을, x86 패키지는 x86 파일을 받는다. 일부러 하는 예외(64비트 패키지 속 32비트 도우미)는 조용히 받지 말고 선언하게 한다. 구성 요소 자체의 비트 수는 바뀌지 않는다.

16.4 안전하게 읽기

오프셋과 크기는 쓰기 전에 모두 파일 길이와 대조하고, RVA 는 그 범위 전체를 담은 섹션을 거쳐서만 바꾸며, 기대한 세 디렉터리 단계에서 멈춘다 - 고리나 범위 밖 포인터를 가진 조작된 파일은 흔하다.

17 캐비닛, MSZIP, deflate

이 장의 차례

17.1	캐비닛 배치 [spec: MS-CAB]	72
	블록 체크섬 [spec]	72
17.2	MSZIP [spec: MS-MCI]	72
17.3	deflate 인코더 간단히 [spec: RFC 1951]	73

17.1 캐비닛 배치 [spec: MS-CAB]

CFHEADER (예약 필드가 없으면 36바이트):

"MSCF", u32 0, u32 캐비닛 크기, u32 0, u32 첫 CFFILE 오프셋, u32 0,
u8 부판 3, u8 주판 1, u16 폴더 수, u16 파일 수, u16 플래그(0),
u16 세트 id, u16 캐비닛 번호(0)

CFFOLDER (각 8바이트):

u32 폴더의 첫 CFDATA 오프셋, u16 CFDATA 블록 수, u16 압축
(0 = 없음, 1 = MSZIP)

CFFILE (파일마다):

u32 압축 전 크기, u32 폴더의 압축 전 데이터 속 오프셋, u16 폴더 번호,
u16 날짜, u16 시각, u16 속성, NUL 로 끝나는 이름

CFDATA (블록마다):

u32 체크섬, u16 압축 바이트 수, u16 압축 전 바이트 수(최대 32768), 데이터

- MSI 에서 캐비닛 속 파일 이름은 File 표의 키이고 ASCII 다.
- DOS 날짜 (year - 1980) << 9 | month << 5 | day, 시각 hour << 11 | minute << 5 | second / 2. 고정된 날짜(rubrapack 은 1980-01-01 00:00 을 쓴다)가 출력을 결정적으로 지킨다. 속성은 0x20.
- 폴더 하나에는 블록이 65535개(약 2 GiB)까지 든다. 데이터가 더 많으면 폴더나 캐비닛을 더 쓴다.

블록 체크섬 [spec]

checksum(bytes, seed):

바이트 속 온전한 리틀엔디언 u32 를 모두 seed 에 xor 한다;

남은 1~3바이트는 u32 하나를 이루되 남은 바이트 가운데 첫째가 쓰인 자리 가운데 가장 높은 곳에 온다
(3개 남은: b0<<16 | b1<<8 | b2; 2개: b0<<8 | b1; 1개: b0). 그것도 xor 한다.

블록 체크섬 = checksum(저장된 대로의 4바이트 "압축 크기, 압축 전 크기",
checksum(블록 데이터, 0))

체크섬 0 은 "계산하지 않음"을 뜻하고 판독기는 그것을 받는다.

17.2 MSZIP [spec: MS-MCI]

압축 형식이 1 인 폴더의 CFDATA 블록마다 다음이 든다:

"CK" (43 4B), 그다음 이 블록의 압축 전 바이트(최대 32768)에 대한 온전한 원시 deflate 스트림
(RFC 1951) 하나, 마지막 블록으로 끝남.

압축 해제기는 같은 폴더의 앞 블록들을 사전으로 간직하므로, 인코더는 앞 블록을 되돌아 가리켜도 된다. 그러지 않는 인코더(블록마다 따로 압축)도 올바른 MSZIP 을 만든다. rubrapack 은 블록마다 앞 블록의 압축 전 32 KiB 를 미리 준 사전으로 쓴다(섞인 자료에서 블록을 따로 압축할 때보다 약 9% 작다). 그 사전은 출력이 아니라 입력이므로 블록을 여러 스레드에서 동시에 압축할 수 있고, 스레드 수와 상관없이 바이트가 같다. 폴더는 65535블록이 되기 전에 파일 경계에서 새로 시작하고, 사전은 폴더를 넘지 않는다. Windows 는 이것을 모두 풀어낸다: 폴더 열두 개짜리 패키지가 설치·복구·제거되고 모든 파일이 바이트까지 같다. [observed]

17.3 deflate 인코더 간단히 [spec: RFC 1951]

1. **일치 찾기(LZ77):** 다음 3바이트를 해시하고, 32 KiB 안에서 같은 해시를 가진 앞 자리들의 체인을 따라가 가장 긴 일치(3..258바이트)를 고른다. 체인을 길게 따라갈수록 압축이 좋아진다. 다음 자리도 해 보고 그 일치가 더 길면 리터럴을 내보내는 "게으른" 단계가 눈에 띄게 도움이 된다.
2. **블록:** 기호를 수만 개까지 묶는다. 블록마다 세 가지 인코딩의 크기를 계산해 가장 작은 것을 쓴다:
 - 고정 허프만 부호(리터럴/길이 0-143: 8비트, 144-255: 9, 256-279: 7, 280-287: 8; 거리 5비트);
 - 동적 허프만 부호: 부호 길이는 15비트로 제한(부호 길이 알파벳은 7). package-merge 가 제한된 최적 길이를 준다. HLIT, HDIST, HLEN, 16 17 18 0 8 7 9 6 10 5 11 4 12 3 13 2 14 1 15 순서의 부호 길이 부호, 그다음 런LENGTH로 부호화한 길이들(16 = 앞 것 3-6번 반복, 17 = 0 을 3-10개, 18 = 0 을 11-138개)을 보낸다;
 - 저장(바이트 정렬한 LEN, NLEN, 원 바이트; 저장 블록 하나에 최대 65535).
3. 허프만 부호는 최상위 비트부터, 그 밖의 것은 최하위 비트부터 쓴다.
4. 늘 기호 둘 이상에 부호 길이를 준다(외톨이 기호는 가짜 기호와 짝지운다). 그래야 모든 부호가 완전하다 - 불완전한 부호를 거부하는 디코더가 있다.

인코더는 독립된 구현(예: zlib 라이브러리의 원시 deflate, window bits -15)으로 출력을 풀어 보고, 그것으로 지은 MSI 는 설치해서 설치된 파일의 해시를 비교해 확인한다(출력을 Windows 와 대조하기).

18 결정적인 정체와 재현 가능한 패키지

이 장의 차례

18.1	이름에서 만드는 GUID(UUID 8판) [spec: RFC 9562]	74
18.2	키	74
18.3	재현 가능한 패키지	74

Windows Installer 는 **변하지 않아야 하는** GUID 와 키로 것들을 식별한다: 같은 구성 요소는 모든 판에서 같은 ComponentId 를 지켜야 하고, 그러지 않으면 복구와 업그레이드가 어긋난다. 손으로 적어 두는 대신 내용에 서 만들어 내면 값이 변하지 않고 빌드도 재현 가능해진다. 이 문서는 rubrapack 의 규칙을 적는다. 변하지 않 기만 하면 다른 규칙도 된다.

18.1 이름에서 만드는 GUID(UUID 8판) [spec: RFC 9562]

```
digest = SHA-256( u32le(len(tag)) || tag
                  || u32le(1)                (형식 판)
                  || 필드마다: u32le(len(field)) || field ) (필드는 UTF-8)
uuid   = digest[0..16], 그다음 바이트 6 = (바이트 6 & 0x0F) | 0x80 (8판)
                  바이트 8 = (바이트 8 & 0x3F) | 0x80 (변종 10)
text   = "{" + hex(바이트 0-3) "-" hex(4-5) "-" hex(6-7) "-" hex(8-9) "-" hex(10-15) + "}"
          대문자, 바이트 순서대로
```

길이 접두는 ("ab","c") 와 ("a","bc") 를 가르고, 꼬리표(tag)는 서로 다른 종류의 정체를 가른다.

정체	꼬리표	필드
ComponentId	rubrapack.component	업그레이드 코드, 범위(machine), 구성 요소 아키텍처, 논리 대상 경로 (<dir id>/<파일 이름>), 키 경로 종류(file), 키 경로 이름(File 키)
ProductCode	rubrapack.product	업그레이드 코드, 아키텍처, 범위, 판의 앞 세 부분
패키지 코드(재현 빌드)	rubrapack.package	패키지 코드를 모두 0 으로 두고 지은 패키지의 SHA-256 16진

ProductCode 는 세 부분 판과 함께 바뀌고 - 메이저 업그레이드에 필요한 것 - ComponentId 는 바뀌지 않는다.

18.2 키

표 키(File, Component, Directory)는 식별자다: [A-Za-z_][A-Za-z0-9_]*, 최대 72자. rubrapack 은 파일과 폴 더에는 작성자의 ID 를 그대로 쓰고 나머지는 끌어낸다:

```
key = 접두 + "_" + SHA-256(논리 이름)의 소문자 16진 앞 20자리
      구성 요소는 접두 C(논리 이름: File 키), 중간 폴더는 D
      (논리 이름: "<표준 폴더>/<부분>/<부분>..."), 와일드카드에서 온 파일은 F
```

18.3 재현 가능한 패키지

시계, 컴퓨터, 해시 표의 순서에 기대는 것이 하나도 없으면 패키지는 바이트까지 재현된다: 고정된 CFB 디렉 터리 시각(0), 고정된 캐비닛 날짜, 문자열의 바이트 순서로 매긴 문자열 id, 키로 정렬한 행, 고정된 순서의 스트림, 무작위 GUID 없음. 패키지 파일마다 달라야 하는 정체는 패키지 코드 하나뿐이다. 재현이 중요하면 (위 처럼) 내용에서 끌어내고, 아니면 무작위로 쓴다.

19 Authenticode 서명

이 장의 차례

19.1	PE 파일에서 서명이 사는 곳	75
19.2	PE 다이제스트	75
19.3	CMS SignedData	75
19.4	MSI 패키지	76
19.5	타임스탬프	77
19.6	ECDSA	77
19.7	Windows 가 알려 주는 것	77

Windows 가 서명된 PE 파일(.exe, .dll)이나 MSI 패키지를 받아들이게 하려면 프로그램이 무엇을 써야 하는가. 출처 표시는 구현하는 사람을 위한 패키지 형식과 같다: **[spec]** 은 Microsoft 의 "Windows Authenticode Portable Executable Signature Format", PE/COFF 명세, RFC 5652(CMS), **[observed]** 는 Windows 자신의 서명기 (mssign32.dll 을 쓰는 PowerShell Set-AuthenticodeSignature)가 서명하고 Windows 11 에서 Get-AuthenticodeSignature / WinVerifyTrust 로 확인한 파일이다.

19.1 PE 파일에서 서명이 사는 곳

- 인증서 표: 선택 머리의 데이터 디렉터리 항목 4(항목은 8바이트: 파일 오프셋, 크기). 디렉터리는 PE32(magic 0x10B)면 선택 머리 + 96, PE32+(magic 0x20B)면 + 112 에서 시작하고, NumberOfRvaAndSizes 는 5 이상이어야 한다. 다른 모든 디렉터리 항목과 달리 그 "주소"는 RVA 가 아니라 **파일 오프셋**이다. **[spec]**
- 그 오프셋, 파일 끝에: WIN_CERTIFICATE { u32 dwLength, u16 wRevision = 0x0200, u16 wCertificateType = 0x0002 (PKCS_SIGNED_DATA), bCertificate[] }. bCertificate 는 DER CMS ContentInfo 다. dwLength 는 8바이트 머리를 포함하고 8의 배수로 올리며 늘어난 바이트는 0 이다. 디렉터리 항목의 크기는 dwLength 와 같다. **[spec]** **[observed]**
- 표를 붙이기 전에 파일을 8의 배수까지 0 으로 채운다. 채운 바이트도 해시한다(아래 "섹션 뒤의 데이터"). **[spec]**
- 선택 머리의 CheckSum(선택 머리 + 64)을 고친다. Windows 의 서명기는 다시 계산하지 않고 있던 값을 조정한다: 체크섬이 PE 알고리즘 값이 아니던 파일이면 Windows 는 그 차이를 지키고, 새로 계산하면 그렇지 않다. CheckSum 은 다이제스트 밖이라 둘 다 검증된다. **[observed]**

19.2 PE 다이제스트

SHA-256 으로 이 순서대로: **[spec]**

1. 머리, 0 부터 SizeOfHeaders 까지. CheckSum(4바이트)과 인증서 표 디렉터리 항목(8바이트)은 뺀다.
2. SizeOfRawData > 0 인 모든 섹션을 PointerToRawData 가 커지는 순서로, PointerToRawData 부터 SizeOfRawData 바이트.
3. 그 뒤의 데이터: SizeOfHeaders + sum(SizeOfRawData) 부터 파일 끝에서 인증서 표를 뺀 곳까지.

구멍 둘을 두고 파일을 앞에서부터 해시해도 섹션 사이에 틈이 없을 때만 같은 결과가 나온다. 위 단계가 정의다.

19.3 CMS SignedData

Windows 의 서명기가 쓰고 받아들이는 것: **[observed]**

```

ContentInfo { signedData (1.2.840.113549.1.7.2), [0] SignedData {
  version 1,
  digestAlgorithms SET { sha256 + NULL },
  encapContentInfo { SPC_INDIRECT_DATA (1.3.6.1.4.1.311.2.1.4),
    [0] SpcIndirectDataContent } -- SEQUENCE 그대로, OCTET STRING 없음
  certificates [0] { 서명자의 인증서, 그다음 중간 인증서; 자체 서명 루트 없음 },
  signerInfos SET { SignerInfo {
    version 1, 서명자의 IssuerAndSerialNumber,
    digestAlgorithm sha256 + NULL,
    signedAttrs [0] { SpcSpOpusInfo, contentType, SpcStatementType, messageDigest },
    signatureAlgorithm rsaEncryption + NULL,
    signature (RSA PKCS#1 v1.5) } } } }

```

- SpcIndirectDataContent ::= SEQUENCE { data, messageDigest DigestInfo }. PE 파일이면 data 는 SEQUENCE { SPC_PE_IMAGE_DATA (1.3.6.1.4.1.311.2.1.15), SpcPeImageData } 이고 Windows 는 SpcPeImageData 를 30 09 03 01 00 A0 04 A2 02 80 00 으로 쓴다: 플래그 없음, 그리고 빈 유니코드 문자열인 파일 링크. messageDigest 가 PE 다이제스트를 담는다. [observed]
- **messageDigest** 속성은 SpcIndirectDataContent 의 내용을 그 SEQUENCE 꼬리표와 길이 없이 해시한 것이다 - 요소 전체도 아니고 PE 파일도 아니다. [observed]
- 서명 속성은 꼬리표 [0](0xA0)로 저장하지만 서명은 같은 바이트를 꼬리표 SET(0x31)으로, DER SET OF 로 정렬해 계산한다. Windows 가 쓰는 속성 넷: SpcSpOpusInfo(1.3.6.1.4.1.311.2.1.12) = SEQUENCE { [0] { [0] "" }, [1] { [0] "" } }, contentType = SPC_INDIRECT_DATA, SpcStatementType(1.3.6.1.4.1.311.2.1.11) = SEQUENCE { individual code signing (1.3.6.1.4.1.311.2.1.21) }, messageDigest. 서명 시각은 없다. [observed]
- RSA PKCS#1 v1.5 는 결정적이다: 같은 키로 정확히 이 구조를 쓰는 프로그램은 Windows 와 같은 바이트를 만든다. rubrapack 이 그렇다(위의 CheckSum 차이만 빼고). [observed]

19.4 MSI 패키지

Microsoft 는 MSI 패키지를 어떻게 해시하는지 공개하지 않는다. 아래 규칙은 Windows 의 서명기로 패키지에 서명하고 비교해서 알아냈다: 같은 패키지과 키에 대해 rubrapack 의 서명과 Ex 스트림은 Windows 의 것과 바이트까지 같았고, 디렉터리 항목에 0 아닌 상태 비트·시각·CLSID 를 준 패키지들로 필드를 가려냈다. [observed]

- 서명은 복합 파일 루트의 \005DigitalSignature 스트림이다: PE 파일과 같은 CMS ContentInfo 에 data = SEQUENCE { SPC_SIPINFO (1.3.6.1.4.1.311.2.1.30), SpcSipInfo }. Windows 는 SpcSipInfo 를 SEQUENCE { 2, OCTET STRING <MSI SIP GUID {000C10F1-0000-0000-C000-000000000046} 을 리틀엔디언 바이트 순서로>, 0, 0, 0, 0, 0 } 으로 쓴다.
- Windows 는 \005MsiDigitalSignatureEx 도 쓴다: 32바이트, 복합 파일 디렉터리의 "사전 해시"에 대한 SHA-256: 루트 자신의 시각은 들지 않는다(Windows 는 서명한 파일을 저장할 때 루트의 고친 시각을 정한다).
 - ▶ 루트 항목의 CLSID(16바이트)와 상태 비트(4바이트, 리틀엔디언);
 - ▶ 그다음 루트의 모든 스트림 - 서명 스트림 둘은 빼고 - 을 이름의 UTF-16LE 바이트 비교 순서로: 이름 (UTF-16LE, 끝 NUL 없음), 스트림 크기 8바이트 리틀엔디언, 항목의 만든 시각과 고친 시각을 저장된 그대로(8 + 8바이트). 스트림의 CLSID 와 상태 비트는 들지 않는다.
- SpcIndirectDataContent 의 다이제스트는 다음의 해시다: 32바이트 MsiDigitalSignatureEx 값(그 스트림을 쓸 때), 그다음 같은 이름 순서의 모든 스트림 내용(역시 서명 스트림 둘은 빼고), 그다음 루트 CLSID.
- 루트의 스트림만 관찰했다. 저장소가 있는 패키지(안에 넣은 변환, 하위 데이터베이스)는 여기서 다루지 않고, rubrapack 은 그런 패키지에 서명하지 않는다.
- 서명은 .msi 안에 든 것만 덮는다: 밖의 캐비닛(# 으로 시작하지 않는 Media 표 Cabinet 값)은 다이제스트에 들지 않는다.

19.5 타임스탬프

타임스탬프 없는 서명은 서명 인증서가 만료되면 더는 유효하지 않다. RFC 3161 타임스탬프가 있으면 Windows 는 대신 찍힌 시각에 인증서를 확인한다. `SIGNER_TIMESTAMP_RFC3161` 을 준 `mssign32!SignerTimeStampEx2` 가 쓰는 것: [observed]

- 요청의 메시지 지문은 `SignerInfo` 의 `signature` 값 - OCTET STRING 의 내용, 꼬리표와 길이 없이 - 의 해시 (여기서는 SHA-256)다.
- 서버의 `TimeStampToken(TSTInfo` 위의 CMS SignedData, RFC 3161)은 서명 뒤에 `SignerInfo` 의 서명 안 된 속성으로 들어간다: [1] { SEQUENCE { 1.3.6.1.4.1.311.3.3.1, SET { TimeStampToken } } }.
- 서명의 다른 것은 바뀌지 않는다: 서명 속성, 서명, 바깥 인증서는 전과 같은 바이트이고, 서버의 인증서는 토큰 안에 실려 다닌다. MSI 패키지에서는 `\005DigitalSignature` 스트림이 그만큼 커지고(몇 킬로바이트 토큰이면 미니 스트림에서 보통 섹터로 옮겨 간다), `\005MsiDigitalSignatureEx` 와 다이제스트는 바뀌지 않는다 - 서명 스트림은 거기 들지 않기 때문이다.
- `rubrapack` 은 정확히 이것을 쓴다. 같은 키와 토큰이면 그 PE 파일은 Windows 의 것과 바이트까지 같고, MSI 서명 스트림도 그렇다.

`Set-AuthenticodeSignature -TimestampServer` 는 대신 옛 형태를 쓴다: PKCS#9 `counterSignature` 속성 (1.2.840.113549.1.9.6)과, 바깥 인증서에 더한 서버의 인증서. Windows 는 둘 다 받아들인다. [observed]

19.6 ECDSA

ECDSA P-256 서명자는 PE 파일, MSI 패키지, MSIX 패키지 모두에 통한다(Windows 11 26100 은 루트를 신뢰하면 셋 모두 `Valid` 라고 한다). Windows 의 서명기는 `SignerInfo` 의 `signatureAlgorithm` 을 `ecdsa-with-SHA256` 이 아니라 `NULL` 매개변수의 `id-ecPublicKey(1.2.840.10045.2.1)`로 쓰고, 서명 값은 DER SEQUENCE { INTEGER `r`, INTEGER `s` } 로 쓴다. `rubrapack` 도 결정적 `nonce`(RFC 6979)로 같게 쓰고, 두 OID 를 모두 읽는다. [observed]

19.7 Windows 가 알려 주는 것

`Get-AuthenticodeSignature` 는 서명하는 쪽에 필요한 경우들을 가려 준다(타임스탬프가 있으면 `TimeStamperCertificate` 에 타임스탬프 인증서를 적는다): `Valid`(다이제스트, 서명, 신뢰하는 루트까지의 사슬), "the certificate chain ... not trusted" 를 단 `UnknownError`(다이제스트와 서명은 맞고, 루트를 이 컴퓨터가 신뢰하지 않음), `HashMismatch`(서명 뒤에 파일이 바뀐), `NotSigned`. 그러니 어떤 루트도 신뢰시키기 전에 서명이 맞는지 확인할 수 있다. [observed]

20 출력을 Windows 와 대조하기

이 장의 차례

20.1	데이터베이스가 열리고 풀린다	78
20.2	패키지가 설치된다	78
20.3	쓰기 전에 하는 검사	79
20.4	Microsoft 의 ICE 규칙 [관찰]	79
20.5	멀쩡해 보이지만 아닌 것	79

Microsoft 는 MSI 표 인코딩을 공개하지 않으므로 기준은 Windows 자신이다. 이 확인에는 Windows 컴퓨터와 Windows Installer API(msi.dll)만 있으면 된다. SDK 도구는 필요 없다.

20.1 데이터베이스가 열리고 풀린다

1. 파일을 MsiOpenDatabase 로 읽기 전용으로 연다 - 복합 파일과 문자열 풀이 받아들여진다는 뜻이다.
2. 모든 표(와 _ForceCodepage, _SummaryInformation)를 MsiDatabaseExport 로 IDT 파일로 내보낸다.
3. 자기 판독기가 내보낸 것과, 또는 msi.dll 로 만든 기준 데이터베이스(MsiOpenDatabase 만들기 모드, 65001 이 든 _ForceCodepage.idt 의 MsiDatabaseImport, 뷰를 통한 SQL CREATE TABLE/INSERT)의 IDT 와 비교한다. 바이트까지 같은 IDT 는 인코딩이 맞다는 뜻이다.

20.2 패키지가 설치된다

설치 엔진은 msixexec 보다 프로그램에서 부른다. 그래야 단계마다 동기로 돌고 오류 코드를 돌려준다:

단계	API	기대
조용한 UI	MsiSetInternalUI(INSTALLUILEVEL_NONE)	
자세한 로그	MsiEnableLog(모든 모드)	무엇이 실패하면 읽는다
설치	MsiInstallProduct(path, "")	0
이름	설치된 경로마다 FindFirstFile, 항목 자신의 UTF-16 이름을 정확히 비교	같음
내용	설치된 파일마다 MsiGetFileHash 와 자기 MsiFileHash 행	같음
등록	MsiGetProductInfo(ProductCode, InstalledProductName)	자기 ProductName
복구	설치된 파일 하나를 지우고 MsiReinstallProduct(ProductCode, ...)	파일이 같은 이름으로 돌아옴
업그레이드	판 N 위에 판 N+1 설치	옛 ProductCode 없어지고 새것이 설치됨
다운그레이드	판 N 을 다시 설치	거부(1603, 로그에 거부한 동작의 이름)
기능	MsiQueryFeatureState; MsiConfigureFeature(..., INSTALLSTATE_LOCAL)	INSTALLLEVEL 보다 높은 수준은 켜 때까지 없음
되돌림	InstallFiles 뒤에 실패하는 지연 사용자 지정 동작(형식 34 + 1024, "[SystemFolder]cmd.exe" /c exit 1)과 새 패키지 코드를 가진 판 N+1 의 사본을 N 위에 설치	1603(로그: 오류 1722); N 은 그대로 등록되어 있고 파일도 그대로
제거	MsiConfigureProduct(ProductCode, ..., INSTALLSTATE_ABSENT)	0; 폴더, 키, 등록이 사라짐

컴퓨터 전체 설치에는 관리자 권한 프로세스가 필요하다. 시험 제품은 제 UpgradeCode 를 쓰고 같은 실행 안에서 지운다.

20.3 쓰기 전에 하는 검사

msi.dll 은 깨진 데이터베이스를 많이 받아들이고, 실패하더라도 설치 때에야 실패한다(열 너비조차 강제하지 않는다). 작성기는 제 표를 먼저 검사해야 한다. rubrapack 의 build 는 다음 가운데 하나라도 여기는 패키지는 쓰지 않는다(진단 RP2001-RP2015):

- 칸마다 열의 종류가 맞고, null 을 허용하지 않는 열에 null 이 없고, 문자열은 UTF-16 단위로 센 열 너비보다 길지 않으며(BMP 밖 글자는 둘로 센다) 올바른 UTF-8 이다. 16비트 정수는 -32767..32767, 32비트 정수는 -2147483648 이 아니다(그 값들은 null 을 뜻한다);
- 기본 키가 겹치지 않고, 참조(Component.Directory_, File.Component_, FeatureComponents, CreateFolder, MsiFileHash.File_, Directory_Parent, Feature_Parent)가 있는 행을 가리키며, 순서 표의 동작은 모두 표준 동작, CustomAction, Dialog 가운데 하나다;
- ProductCode, ProductName, ProductVersion, Manufacturer, ProductLanguage 가 있다. GUID 는 중괄호 속 대문자, 구성 요소 GUID 는 서로 다르며, ProductCode 는 UpgradeCode 와 다르다;
- File.Sequence 는 저마다 다르고 어떤 Media.LastSequence 가 덮는다;
- 표준 동작의 순서를 지킨다(비용 계산은 InstallValidate 전, 파일 동작은 InstallInitialize 와 InstallFinalize 사이);
- 64비트 구성 요소 속성(256)이 요약 Template 의 플랫폼(x64/Arm64 대 Intel)과 맞는다;
- 파일 키 경로는 같은 구성 요소의 파일이다;
- Upgrade.ActionProperty 는 저마다 대문자(공개)이고 SecureCustomProperties 에 있다 - 그렇지 않으면 컴퓨터 전체 설치가 그것을 서버 쪽에 넘기지 않는다.

20.4 Microsoft 의 ICE 규칙 [관찰]

Windows SDK 의 "MSI Tools" 에는 MsiVal2 와 darice.cub 가 들어 있다. Microsoft 의 작성 도구들이 돌리는 내부 일관성 검사기(ICE)다: MsiVal2 package.msi darice.cub -f 가 실패한 것을 찍는다. Windows 에서만 돌고, 위의 검사보다 한 걸음 더 간다 - 표의 모양만이 아니라 뜻을 안다. rubrapack 의 시험 패키지(시험에 쓰는 모든 원본, 39개)에서 나오는 것과 그 까닭:

- **ICE03, "Missing specifications in _Validation"** 이 모든 열에: _Validation 표는 검사기만을 위해 열을 설명한다. Windows Installer 는 읽지 않고, rubrapack 은 쓰지 않는다.
- **ICE82**, 겹친 순서 번호: 대화창 언어가 여럿인 패키지의 언어 동작들이 번호 하나를 함께 쓴다. 서로 무관한 속성 설정 동작이라 순서가 상관없다.
- **ICE34**, 언어 페이지 라디오 속성에 Property 행이 없다: 일부러다. 명령줄(RPLANGUAGE=ko)이 감지보다 이겨야 하고, 감지는 NOT RPLANGUAGE 일 때만 돈다.
- **ICE43, ICE57**, 파일을 키 경로로 쓰는 컴포넌트의 광고되지 않은 바로가기: 이 규칙들은 바로가기 폴더를 사용자별로 가정한다. 컴퓨터 전체 패키지(ALLUSERS=1)에서 시작 메뉴와 바탕화면은 모든 사용자 폴더이고, 사용자별·점용 패키지는 사용자마다 따로 설치되므로, 이 규칙이 막으려는 "첫 사용자만" 문제는 생기지 않는다.
- **ICE52**, AppSearch 의 비공개 속성(RpFound_<ID>, 기억하는 폴더): 명령줄이 정하지 못하게 일부러 비공개다. AppSearch 는 두 순서에서 모두 돌므로 서버 쪽에 넘길 필요가 없다.

셋은 진짜 결함이었고 고쳤다(0.4.3): 완료·취소·실패 페이지가 AdminUISequence 에 없어 전체 UI 의 관리 설치(msiexec /a)가 그 페이지 없이 끝났다(ICE20). 폴더를 키 경로로 쓰는 컴포넌트(INI 항목, [remove])가 CreateFolder 에 없었다(ICE18). 대화창이 있는 패키지에 ControlCondition 표가 없었는데, ICE17 은 대화창마다 이 표를 읽는다(검사 대신 오류 2228 로 멈췄다).

20.5 멀쩡해 보이지만 아닌 것

- msi.dll 이 만든 데이터베이스가 완전한 명세는 아니다: msi.dll 은 65001 요약 문자열을 제대로 저장하고도 다시 읽지 못한다(요약 정보).
- MsiDatabaseExport 는 요약 날짜를 지역 시각으로 찍는다. 날짜는 UTC 로 비교한다.
- MsiOpenDatabase 가 성공했다고 행 순서나 참조 수가 맞다는 뜻은 아니다 - 그것은 설치만이 말해 준다. 열어 보기만 하지 말고 설치해서 시험한다.

21 MSIX 패키지

이 장의 차례

21.1	ZIP 압축 파일	80
21.2	블록 맵(AppxBlockMap.xml)	80
21.3	[Content_Types].xml	81
21.4	매니페스트(AppxManifest.xml)	81
21.5	가상 레지스트리(Registry.dat, User.dat)	81
21.6	가상 파일 시스템(VFS\...)	82
21.7	확장	82
21.8	묶음(.msixbundle)	83
21.9	서명(AppxSignature.p7x)	83
21.10	서명 없는 패키지 설치하기	84

Windows 가 MSIX 패키지를 읽고, 설치하고, 실행하게 하려면 프로그램이 무엇을 써야 하는가. 출처 표시는 구현하는 사람을 위한 패키지 형식과 같다: **[spec]** 은 Microsoft Learn(패키지 매니페스트와 블록 맵 스키마)과 ECMA-376 Part 2(Open Packaging Conventions), **[observed]** 는 Windows 자신의 패키징 API(Windows 의 일부인 AppxPackaging.dll 의 IAppxFactory/IAppxPackageWriter)가 쓴 패키지, 그리고 IAppxPackageReader 로 다시 읽고 Windows 11 에서 Add-AppxPackage 로 설치한 rubrapack 의 패키지다. rubrapack 은 여기 적은 것을 쓴다.

21.1 ZIP 압축 파일

- 페이로드 파일이 먼저, 그다음 AppxManifest.xml, AppxBlockMap.xml, [Content_Types].xml 이 그 순서로 온다. [observed]
- 모든 항목은 크기와 상관없이 ZIP64 다: 로컬 머리는 필요 판 4.5, 플래그 0x0008(데이터 서술자), CRC 와 크기 0, extra 필드 없음. 데이터 뒤에는 ZIP64 데이터 서술자(pk\7\8, CRC-32, 압축 크기와 원 크기 각 8 바이트)가 온다. 중앙 디렉터리 항목은 만든 판과 필요 판이 4.5(MS-DOS), 크기와 오프셋이 0xFFFFFFFF 이고 ZIP64 extra 필드(0x0001, 24바이트: 원 크기, 압축 크기, 로컬 머리 오프셋)를 가진다. 압축 파일은 ZIP64 끝 레코드(크기 44), 그 로케이터, 그리고 개수와 오프셋이 모두 0xFFFF / 0xFFFFFFFF 인 끝 레코드로 끝난다. [observed]
- 방식: 저장(0)과 deflate(8). AppxManifest.xml, AppxBlockMap.xml, [Content_Types].xml 은 페이로드가 저장되어도 deflate 한다. [observed]
- 항목 이름은 OPC 부분 이름이다: 폴더 사이는 /, A-Z a-z 0-9 - . _ ~ 밖의 모든 바이트는 UTF-8 바이트까지 퍼센트 인코딩한다 - data\<U+C790> %%(1).txt(한글 음절, 공백, %, #, 괄호)는 data/%EC%9E%90%20%25%23%281%29.txt 로 저장한다 - 그리고 UTF-8 이름 플래그는 켜지 않는다. [Content_Types].xml 은 대괄호를 그대로 둔다. [observed]
- 머리의 날짜는 읽지 않는다. Windows 는 쓰는 시각을, rubrapack 은 1980-01-01 00:00 을 써서 빌드마다 패키지가 바뀌지 않게 한다. [observed]

21.2 블록 맵(AppxBlockMap.xml)

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<BlockMap xmlns="http://schemas.microsoft.com/appx/2010/blockmap"
  xmlns:b4="http://schemas.microsoft.com/appx/2021/blockmap" IgnorableNamespaces="b4"
  HashMethod="http://www.w3.org/2001/04/xmlenc#sha256">
  <File Name="data\text.txt" Size="218890" LfhSize="43">
```

```

    <Block Hash="(원 바이트 65536개의 base64 SHA-256)" Size="(이 블록의 압축 바이트 수)"/>
    ...
    <b4:FileHash Hash="(파일 전체의 base64 SHA-256)"/>
  </File>
  ...
</BlockMap>

```

- 페이로드 파일마다 File 하나, AppxManifest.xml 에도 하나. 블록 맵과 [Content_Types].xml 은 심지 않는다. Name 은 적은 그대로의 경로로 \ 를 쓰고 퍼센트 인코딩하지 않는다. Size 는 원 크기, LfhSize 는 로컬 머리의 크기(30 + ZIP 이름 길이)다. 빈 파일에는 Block 이 없다. [spec] [observed]
- 원 데이터 65536바이트마다 Block 하나. Hash 는 그 원 블록의 base64 SHA-256 이다. [spec]
- deflate 한 파일은 블록마다 독립된 원시 deflate 조각 하나다: 조각마다 새 디코더로 풀리고(앞 블록을 되돌아 가리키는 것이 없다) 빈 저장 블록(00 00 FF FF)으로 끝나며, 마지막 조각 뒤에는 빈 마지막 블록(03 00)이 온다. 블록의 Size 는 그 조각의 바이트 수라서, 파일의 압축 크기는 Size 합 더하기 2 다(빈 파일은 2바이트, 블록 없음). 저장 파일의 블록에는 Size 가 없다. 압축되지 않는 데이터도 (저장 블록으로) deflate 한 다. [observed]
- b4:FileHash 는 블록이 둘 이상인 파일에만 나온다. [observed]
- Windows 의 판독기는 파일을 읽으면서 블록마다 해시를 확인하고, 파일이 블록 맵과 다른 패키지는 읽을 때도 설치할 때도 거부한다. [observed]

21.3 [Content_Types].xml

한 줄: 처음 쓰인 순서대로의 파일 확장자(소문자)마다 Default 하나, xml 은 application/vnd.ms-appx.manifest+xml, 그리고 application/vnd.ms-appx.blockmap+xml 인 Override PartName="/AppxBlockMap.xml". rubrapack 은 확장자 없는 파일마다 Override 를, 그리고 페이로드의 .xml 파일이 xml 기본을 차지했으면 / AppxManifest.xml 에 Override 를 더한다. [observed]

21.4 매니페스트(AppxManifest.xml)

Windows 가 설치하고 시작하는 가장 작은 데스크톱 앱(네임스페이스 foundation/windows10, uap/windows10, restrictedcapabilities): [spec] [observed]

- Identity: Name(A-Z a-z 0-9 . - 로 3~50자), Publisher(서명 인증서의 주체), Version(네 부분, 각 65535 이하), ProcessorArchitecture(x64, arm64, x86).
- Properties: DisplayName, PublisherDisplayName, Logo(패키지 속 PNG).
- MinVersion 과 MaxVersionTested 를 가진 Dependencies/TargetDeviceFamily Name="Windows.Desktop".
- Resources/Resource Language.
- uap:VisualElements(DisplayName, Description, BackgroundColor, Square150x150Logo, Square44x44Logo)를 가진 Application Id Executable EntryPoint="Windows.FullTrustApplication", 그리고 제한된 기능 runFullTrust.
- 매니페스트 속 경로는 \ 를 쓰고 패키지의 파일을 가리킨다.

21.5 가상 레지스트리(Registry.dat, User.dat)

패키지 뿌리의 레지스트리 하이브(REGF 형식: 레지스트리 하이브 파일(REGF)). Windows 11(26100)에서 패키지된 앱으로 켜진 것: [observed]

- Registry.dat: 그 REGISTRY\MACHINE\SOFTWARE 키가 HKLM\Software 를 나타낸다(하이브의 뿌리 자체는 아니다). REGISTRY\MACHINE\SOFTWARE\WOW6432Node\... 는 앱이 32비트 보기에서 보는 것이다. Microsoft 의 설명("registry.dat serves as the logical equivalent of HKLM\Software")은 앱이 보는 것에 대한 말이지 하이브의 배치에 대한 말이 아니다.
- User.dat: 그 뿌리가 HKCU 를 나타낸다(그 아래 Software\... 가 HKCU\Software\...).
- 앱은 패키지의 키를 실제 레지스트리와 합쳐 읽는다. 컴퓨터의 레지스트리에는 아무것도 쓰지 않고, 제거한 뒤에는 아무것도 남지 않는다.

- Windows의 패키징 도구는 이 하이브를 오프라인 레지스트리 라이브러리로 쓴다(그 표시 "OfRg"가 기본 블록에 있다).

21.6 가상 파일 시스템(VFS\...)

VFS\<폴더> 아래의 파일은 앱에게 실제 자리에 있는 것으로 보이고, 실제 폴더는 바뀌지 않는다. 켄 것: ProgramFilesX64(%ProgramFiles%), SystemX64(System32), Common AppData(%ProgramData%) - 그리고 Microsoft Learn이 말하는 대로 AppData나 Local AppData에는 없다. rubrapack이 쓰는 다른 이름(ProgramFilesX86, ProgramFilesCommonX64/X86, SystemX86, Windows)은 Microsoft Learn이 늘어놓은 것이다. [spec] [observed]

21.7 확장

rubrapack이 앱의 <Extensions>(uap:VisualElements 뒤)에 쓰는 것, 그리고 이것뿐이다: 여기 행이 없는 원본 기능은 오류이고, 필요한 Windows 빌드가 패키지의 MinVersion보다 높은 것은 min-version을 올리라고 요구한다. [spec] Microsoft Learn, "Integrate your desktop app with Windows using packaging extensions"와 요소 문서들. 마지막 열은 Windows 11(21H2)에 패키지를 설치해 확인한 것이다. [observed]

원본	요소(범주)	네임스페이스	최소 빌드	기능	Windows 11에서 확인한 것
[assoc]	uap:Extension windows. fileTypeAssociation > uap3: FileTypeAssociation(Name, Parameters) > uap:DisplayName, uap: SupportedFileTypes > uap:FileType	uap, uap3	14393	runFullTrust	.rpx/.rpy 파일을 열면 프로그램이 그 파일과 함께 시작된다
[protocol]	uap3:Extension windows.protocol > uap3:Protocol(Name, Parameters)	uap3	14393	runFullTrust	scheme: URI를 시작하면 프로그램이 그것과 함께 시작된다
[msix-extension] 별칭	uap3:Extension windows. appExecutionAlias(Executable, EntryPoint) > uap3: AppExecutionAlias > desktop: ExecutionAlias(Alias)	uap3, desktop	14393	runFullTrust	별칭이 %LOCALAPPDATA%\Microsoft\WindowsApps에 생기고 프로그램을 시작한다
[msix-extension] 시작 작업	desktop:Extension windows. startupTask(Executable, EntryPoint) > desktop: StartupTask(TaskId, Enabled, DisplayName)	desktop	14393	runFullTrust	첫 시작 뒤에 등록된다
[shortcut] Desktop	desktop7:Extension windows.shortcut > desktop7:Shortcut(File \$(Desktop)\<이름>.lnk, Icon, Arguments, Description)	desktop7	19645	runFullTrust	사용자의 바탕화면에 바로가기 생성되고 프로그램을 시작한다
[shortcut] Programs, StartMenu	없음: 앱 자신의 시작 메뉴 항목	-	-	-	시작 메뉴에 앱이 있다
[font]	uap4:Extension windows.sharedFonts > uap4:SharedFonts > uap4:Font(File Fonts\<이름>), 첫 앱에	uap4	15063	-	패키지가 설치되어 있는 동안 다른 프로그램이 글꼴을 보고, 제거하면 보지 못한다

- 네임스페이스는 쓸 때만 Package 에 선언하고 무시 가능(ignorable)으로 둔다. 그래서 확장 없는 패키지는 위의 매니페스트를 그대로 지킨다.
- Parameters, Arguments: 리터럴 글(MSI 의 [...]) 는 거부). Windows 는 %1 자리에 파일이나 URI 를 넣는다.
- FileTypeAssociation 의 Name: prog-id 를 소문자로. prog-id 를 같이 쓰는 [assoc] 표들은 FileType 이 여러 인 연결 하나가 된다.
- 모든 것은 원본이 대상으로 적은 실행 파일을 가진 앱에 들어간다. 앱의 실행 파일이 아닌 대상은 오류다.

21.8 묶음(.msixbundle)

Windows 의 묶음 작성기(IAppxBundleWriter)가 만드는 대로, 패키지처럼 놓인 ZIP 압축 파일(데이터 서술자를 가진 ZIP64 항목): [observed]

- 패키지들이 먼저, 저장(방식 0)으로, 그 파일 이름으로, 더한 순서대로. 그다음 AppxMetadata/AppxBundleManifest.xml, AppxBlockMap.xml, [Content_Types].xml 을 deflate 로.
- 블록 맵은 묶음 매니페스트(AppxMetadata\AppxBundleManifest.xml)만 싣는다. 패키지들은 제 블록 맵을 가진다.
- [Content_Types].xml: Default msix = application/vnd.ms-appx, Default xml = application/vnd.ms-appx.bundlemanifest+xml, 그리고 /AppxBlockMap.xml 의 Override.
- 묶음 매니페스트, 줄 끝은 CRLF, 들여쓰기는 탭, 마지막 꼬리표 뒤에는 줄 끝이 없다:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<Bundle xmlns="http://schemas.microsoft.com/appx/2013/bundle" SchemaVersion="5.0" xmlns:b4="http://schemas.microsoft.com/appx/2018/bundle" xmlns:b5="http://schemas.microsoft.com/appx/2019/bundle" IgnorableNamespaces="b4 b5">
  <Identity Name="Example.App" Publisher="CN=Example" Version="1.0.0.0"/>
  <Packages>
    <Package Type="application" Version="1.0.0.0" Architecture="x64" FileName="Example.App_1.0.0.0_x64.msix" Offset="66" Size="61340">
      <Resources>
        <Resource Language="en-US"/>
      </Resources>
      <b4:Dependencies>
        <b4:TargetDeviceFamily Name="Windows.Desktop" MinVersion="10.0.17763.0" MaxVersionTested="10.0.26100.0"/>
      </b4:Dependencies>
    </Package>
  </Packages>
</Bundle>
```

- Offset 은 묶음에서 패키지의 바이트가 시작하는 곳(로컬 머리 다음), Size 는 그 길이다. Resources 와 Dependencies 는 패키지 매니페스트의 Resource 와 TargetDeviceFamily 요소를 되풀이한다.
- 묶음의 Version 은 작성기의 인수(부분마다 16비트인 64비트 수)다. rubrapack 은 패키지들의 판을 준다. 모든 패키지는 묶음의 Name 과 Publisher, 한 가지 판, 그리고 저마다 다른 아키텍처를 가져야 한다.
- 같은 패키지들로 만든 rubrapack 의 묶음은 같은 항목을 같은 오프셋에 가지고, 매니페스트·블록 맵·콘텐츠 형식이 바이트까지 같다. ZIP 날짜만 다르다(rubrapack 은 1980). Windows 는 그 가운데 자기 아키텍처의 패키지를 설치한다: x64 컴퓨터는 x86 과 arm64 가 있어도 x64 를, x86 만 든 묶음에서는 x86 을. [observed]

21.9 서명(AppxSignature.p7x)

Windows 의 서명기(APPX_SIP_CLIENT_DATA 를 준 mssign32!SignerSignEx2; PowerShell 명령은 패키지에 서명하지 못한다)와 대조해 알아내고, rubrapack 이 서명한 패키지를 설치해 확인했다. [observed]

- 매니페스트의 Publisher 는 Windows 가 보여 주는 방식으로 쓴 서명 인증서의 주체와 같아야 한다: RDN 을 마지막부터 처음으로, A=value 를 , 로 이어서 - /CN=Example/O=Example Ltd 로 만든 인증서는 O=Example Ltd, CN=Example 이 필요하다. 그렇지 않으면 서명이 0x8007000B 로 실패한다.
- 서명은 서명기가 하는 대로 압축 파일을 다시 쓴다: 페이로드의 로컬 레코드, 매니페스트, 블록 맵은 그대로 두고, [Content_Types].xml 에 <Override PartName="/AppxSignature.p7x" ContentType="application/vnd.ms-

appx.signature"/> 를 더하고, 서명을 맨 끝에 deflate 로, 크기를 로컬 머리에 담아(필요 판 2.0, 데이터 서술자 없음)더한다. 중앙 디렉터리는 크기와 오프셋이 허락하는 한 ZIP32 방식(ZIP64 extra 필드 없음)으로 쓰고, 끝 레코드의 디스크 번호는 0 이다.

- AppxSignature.p7x 는 PKCX 뒤에 Authenticode 서명과 같은 CMS SignedData 가 오되, 다음이 다르다: data 는 SEQUENCE { SpcSipInfo (1.3.6.1.4.1.311.2.1.30), SEQUENCE { INTEGER 0x01010000, OCTET STRING <SIP GUID>, INTEGER 0, INTEGER 0, INTEGER 0, INTEGER 0, INTEGER 0 } } 이고 패키지 SIP GUID 바이트는 4B DF C5 0A 07 CE E2 4D B7 6E 23 C8 39 A0 9F D1, 묶음 SIP GUID 바이트는 B3 58 5F 0F DE AA 9A 4B A4 34 95 74 2D 92 EC EB 다. 서명 속성은 contentType 과 messageDigest 뿐이다.
- DigestInfo 의 다이제스트는 해시 하나가 아니라 APPX 뒤에 4바이트 꼬리표와 SHA-256 의 기록들이 오는 것이다:
 - ▶ AXPC: 압축 파일 처음부터 서명 항목의 로컬 머리까지;
 - ▶ AXCD: 서명 항목을 뺀 중앙 디렉터리, 그다음 ZIP64 끝 레코드, 그 로케이터, 끝 레코드 - 모두 서명 항목이 없는 것처럼(중앙 디렉터리가 서명의 로컬 머리 자리에서 시작하는 것처럼). Windows 는 서명 항목의 자리를 그 ZIP32 필드에서 읽는다: ZIP64 방식으로 기술한 서명 항목(거기 0xFFFFFFFF)은 HashMismatch 이고, 서명한 뒤 중앙 디렉터리를 그렇게 다시 쓴 패키지도 그렇다;
 - ▶ AXCT: [Content_Types].xml(원 바이트); AXBM: AppxBlockMap.xml;
 - ▶ AXCI: AppxMetadata/CodeIntegrity.cat, 패키지에 그것이 있을 때만.
- Windows 의 서명기는 패키지에 프로그램 파일이 있으면 AppxMetadata/CodeIntegrity.cat - 같은 키로 서명한, 패키지 프로그램 파일의 카탈로그 - 도 더한다. 그것은 선택이다: 그것 없이 서명한 패키지도 설치되고 실행된다. rubrapack 은 그것을 쓰지 않고, 있으면 AXCI 를 확인한다.
- 묶음: 안의 모든 패키지를 먼저 서명하고(저마다 제 AppxSignature.p7x), 그 둘레에 묶음을 쓰고, 묶음 SIP GUID 로 AXCI 없이 서명한다.
- -AllowUnsigned 와 publisher OID(아래)가 필요한 서명 없는 패키지는 서명하지 않는다: 서명은 그 OID 를 가진 publisher 를 거부한다.

21.10 서명 없는 패키지 설치하기

- Publisher 가 OID.2.25.311729368913984317654407730594956997722=1 로 끝나야 한다. 그러면 Add-AppxPackage -AllowUnsigned 가 Windows 11 에 설치한다. [spec]
- 실행 파일이 든 패키지는 관리자 권한 프로세스가 필요하다(아니면 0x80073D2B: 서명 없는 패키지는 실행 파일 활성화를 가질 수 없다). 개발자 모드는 필요 없다. [observed]
- 네트워크 로그온(SSH 세션)에서 Add-AppxPackage 는 "PLM initialization" 에서 0x80070005 로 실패하고, 대화형 세션에서는 된다. [observed]
- 파일 형식이 든 패키지를 제거하면 HKCU\Software\Classes\.<ext> 아래에 빈 OpenWithProgids 키가 남는다 - 패키지가 아니라 Windows 가 하는 일이다. [observed]

22 레지스트리 하이브 파일(REGF)

이 장의 차례

22.1	배치	85
22.2	기본 블록	85
22.3	키(nk)	86
22.4	하위 키 목록(lh, ri)	86
22.5	값(vk)과 값 목록	86
22.6	보안 칸(sk)	86
22.7	재현 가능한 하이브	86

MSIX 패키지 속 Registry.dat 와 User.dat 의 파일 형식이다. Microsoft 는 이것을 공개하지 않는다. 여기의 모든 사실은 **[observed]** 다: 고른 키/값 집합으로 Windows 의 오프라인 레지스트리 라이브러리(offreg.dll, 10.0.26100, Windows 패키징 도구가 쓰는 작성기 - 그 표시가 기본 블록에 있다)가 쓴 하이브, 그리고 같은 집합으로 rubrapack 이 쓴 하이브를 Windows 에서 OROpenHive 와 RegLoadAppKey 로 읽어 Windows 가 자기 하이브에서 늘어놓는 것과 정확히 같은 키와 값이 나오는 것을 확인했다. 새로 쓴 하이브에 필요한 것만 적는다: 트랜잭션 로그, 휘발성 키, 빈 칸 재사용은 없다.

22.1 배치

- 4096바이트 기본 블록, 그다음 하이브 bin 들. 칸 속의 오프셋("칸 오프셋")은 기본 블록 끝에서부터 센다 (파일 오프셋 - 4096). "없음"은 0xFFFFFFFF 다.
- bin: hbin, u32 자신의 칸 오프셋, u32 크기(4096 이나 그 배수), 0 인 8바이트, u64 시작(0), u32 여분(0); 모두 32바이트. 그 뒤에 칸이 온다. 지금 bin 에 들어가지 않는 칸은 그것을 담을 크기의 새 bin 을 시작하고, bin 의 나머지는 빈 칸 하나다.
- 칸: i32 크기(쓰는 중이면 음수, 8의 배수, 크기 필드 포함), 그다음 내용. 칸은 제 bin 의 끝을 넘지 않는다.

22.2 기본 블록

오프셋	필드	값
0	서명	regf
4, 8	순서 번호	1, 1 (다르면 = 깨끗이 닫히지 않음; 판독기는 거부하거나 로그를 다시 적용)
12	u64 시작	0 도 받는다
20, 24	판	1, 5
28, 32	종류, 형식	0, 1
36	루트 키의 칸 오프셋	
40	모든 bin 의 크기	
44	클러스터링	1
0xB0	offreg 의 표시	0fRg, 그다음 u32 1
508	체크섬	앞의 dword 127개의 XOR; 0 은 1 로, 0xFFFFFFFF 는 0xFFFFFFFFE 로 쓴다

나머지는 모두 0 이다.

22.3 키(nk)

nk, u16 플래그, u64 시각, u32 접근 비트(0), u32 부모(루트면 없음), u32 하위 키 수, u32 휘발성 하위 키 수(0), u32 하위 키 목록, u32 휘발성 목록(없음), u32 값 수, u32 값 목록, u32 보안 칸, u32 클래스(없음), u32 가장 긴 하위 키 이름, u32 가장 긴 클래스(0), u32 가장 긴 값 이름, u32 가장 큰 값 데이터, u32 작업 변수(0), u16 이름 길이, u16 클래스 길이(0), 그다음 이름.

- 플래그: 이름을 Latin-1 로 저장하면("압축": 모든 UTF-16 단위가 256 미만) 0x20, 아니면 이름은 UTF-16LE 다. offreg 하이브의 루트 키 이름은 ROOT 이고 플래그는 0x20 뿐이다.
- 가장 긴 이름들은 저장 형태와 상관없이 UTF-16 바이트로 센다.
- 이름은 대소문자 없이 비교한다: UTF-16 단위마다 유니코드 단순 대문자로 바꾼다.

22.4 하위 키 목록(lh, ri)

- lh, u16 수, 그다음 하위 키마다: u32 키 칸, u32 해시. 대문자로 바꾼 이름순 정렬.
- 해시: $h = 0$; 이름의 UTF-16 단위마다 $h = h * 37 + \text{upper}(\text{unit})(32\text{비트에서 넘침})$.
- 하위 키가 507개보다 많으면: 507개까지 담은 lh 목록들의 ri(u16 수, lh 목록마다 u32).

22.5 값(vk)과 값 목록

- 값 목록은 u32 값 칸 오프셋을 값을 정한 순서대로 담은 보통 칸이다.
- vk, u16 이름 길이, u32 데이터 크기, u32 데이터 오프셋, u32 형식, u16 플래그, u16 여분, 그다음 이름. 플래그 1 = Latin-1 이름. 이름 없는 기본값은 이름이 비고 플래그가 0 이다.
- 4바이트 이하의 데이터(빈 값 포함)는 데이터 오프셋 필드에 두고 크기의 최상위 비트를 켜다.
- 16344바이트까지: 데이터 칸 하나. 16345바이트부터: db 칸(db, u16 조각 수, u32 조각 목록), u32 조각 칸들의 목록 칸, 저마다 16344바이트인 조각들 - 마지막 것까지 모든 조각 칸을 온 크기로 잡는다: 마지막 조각 칸이 더 짧으면 Windows 는 그 값을 읽다 멈춘다.

22.6 보안 칸(sk)

모든 키가 나눠 쓰는 sk 칸 하나: sk, u16 0, u32 다음과 u32 이전(둘 다 자기 자신), u32 참조 수(키의 수), u32 서술자 길이, 그다음 자기 상대(self-relative) 보안 서술자. offreg 의 것: 소유자와 그룹은 Administrators; SYSTEM 과 Administrators 모든 권한, Everyone 과 RESTRICTED 읽기, 모두 컨테이너 상속.

22.7 재현 가능한 하이브

시각 필드를 모두 0 으로 두고 칸을 깊이 우선으로 놓으면(키; 루트 키 다음에는 보안 칸; 키의 하위 키 목록, 값 목록, 값마다 그 뒤에 데이터; 그다음 하위 키들을 정렬 순서로) 같은 키와 값은 어느 호스트에서나 같은 바이트가 된다. Windows 는 그런 하이브를 아무 불평 없이 읽는다.